

WEBOVÉ A DATABÁZOVÉ APLIKACE V PHP

URČENO PRO VZDĚLÁVÁNÍ V AKREDITOVANÝCH STUDIJNÍCH PROGRAMECH

ROSTISLAV MIARKA

ČÍSLO OPERAČNÍHO PROGRAMU: CZ.1.07
NÁZEV OPERAČNÍHO PROGRAMU:
VZDĚLÁVÁNÍ PRO KONKURENCESCHOPNOST
OPATŘENÍ: 7.2
ČÍSLO OBLASTI PODPORY: 7.2.2

INOVACE VÝUKY INFORMATICKÝCH PŘEDMĚTŮ VE STUDIJNÍCH PROGRAMECH OSTRAVSKÉ UNIVERZITY

REGISTRAČNÍ ČÍSLO PROJEKTU: CZ.1.07/2.2.00/28.0245

OSTRAVA 2014

Tento projekt je spolufinancován Evropským sociálním fondem a státním rozpočtem České republiky

Recenzent: Ing. Radek Sztefek

Název: Webové a databázové aplikace v PHP
Autor: RNDr. Rostislav Miarka, Ph.D.
Vydání: první, 2014
Počet stran: 160

Jazyková korektura nebyla provedena, za jazykovou stránku odpovídá autor.

© RNDr. Rostislav Miarka, Ph.D.
© Ostravská univerzita v Ostravě

INVESTICE DO ROZVOJE VZDĚLÁVÁNÍ

OBSAH

1 SÍŤOVÝ PROTOKOL HTTP.....5

1.1 PROTOKOL HTTP A JEHO VLASTNOSTI. 5
1.2 URI 8
1.3 METODY POŽADAVKU STRANY KLIENTA 10
1.4 ODPOVĚĎ STRANY SERVERU.....15

2 FORMULÁŘE V JAZYCE HTML.....19

2.1 ZNAČKA FORM 20
2.2 ZNAČKA INPUT.....21
2.3 ZNAČKA BUTTON 31
2.4 ZNAČKA TEXTAREA 32
2.5 ZNAČKA SELECT.....33
2.6 FORMULÁŘE V HTML5.....35
2.7 PŘÍKLAD TVORBY REGISTRAČNÍHO FORMULÁŘE 35

3 JAZYK PHP, JEHO HISTORIE A HLAVNÍ RYSY 40

4 PRVNÍ APLIKACE V PHP, PRÁCE S PROMĚNNÝMI A KONSTANTAMI, OPERÁTORY.....43

4.1 PRVNÍ PHP SKRIPT 44
4.2 PROMĚNNÉ A DATOVÉ TYPY 48
4.2.1 Jednoduché datové typy 48
4.2.2 Složitější datové typy 51
4.2.3 Přetypování proměnných.....54
4.2.4 Funkce pro práci s proměnnými.....55
4.3 KONSTANTY 56
4.4 OPERÁTORY A VÝRAZY 57
4.4.1 Unární operátory.....57
4.4.2 Binární operátory.....58
4.4.3 Ternární operátor.....61
4.4.4 Výrazy.....62
4.4.5 Zkrácené přiřazení 62

5 PŘÍKAZY 64

5.1 VĚTVENÍ PROGRAMU 64
5.1.1 Příkaz if 64
5.1.2 Příkaz switch 69
5.2 CYKLY.....72
5.2.1 Cyklus while 72
5.2.2 Cyklus do…while.....74
5.2.3 Cyklus for 74
5.2.4 Cyklus foreach.....77
5.2.5 Příkazy break a continue.....79
5.3 VKLÁDÁNÍ SOUBORŮ DO STRÁNEK PHP 79
5.4 OPUŠTĚNÍ STRÁNKY PHP 79

6 ZPRACOVÁNÍ VSTUPU OD UŽIVATELE 81

6.1 PARAMETRY V RÁMCI URL ADRESY 81
6.2 FORMULÁŘE 85
6.3 SUPERGLOBÁLNÍ PROMĚNNÉ 88

7 FUNKCE 91

7.1 DEKLARACE, DEFINICE A VOLÁNÍ FUNKCE 91
7.2 PŘEDÁVÁNÍ PARAMETRŮ 93
7.3 ROZSAH PLATNOSTI PROMĚNNÝCH 97
7.4 VNOŘENÉ FUNKCE A REKURZE 99
7.5 PŘEDDEFINOVANÉ FUNKCE 100

8 PRÁCE SE SOUBORY 102

8.1 OTEVŘENÍ A ZAVŘENÍ SOUBORU 102
8.2 PRÁCE S OBSAHEM SOUBORU 105
8.2.1 Čtení ze souboru 105
8.2.2 Zápis do souborů 109
8.2.3 Pohyb uvnitř souborů 109
8.3 MANIPULACE SE SOUBORY 110
8.4 PRÁCE SE ADRESÁŘI 112

9 PRÁCE S ŘETĚZCI 115

9.1 VYPSÁNÍ ŘETĚZCE 115
9.2 PROCHÁZENÍ ŘETĚZCŮ, VYHLEDÁVÁNÍ V ŘETĚZCÍCH 118
9.3 ÚPRAVA ŘETĚZCŮ 120
9.4 ROZDĚLOVÁNÍ A SPOJOVÁNÍ ŘETĚZCŮ 123

10 PRÁCE S POLEM, PRÁCE S DATEM A ČASEM 126

10.1 PRÁCE S POLEM 126
10.2 PRÁCE S DATEM A ČASEM 132

11 OBJEKTOVĚ ORIENTOVANÉ PROGRAMOVÁNÍ V PHP 137

11.1 VYTVOŘENÍ TŘÍDY A JEJÍ INSTANCE 138
11.2 ČLENSKÉ PROMĚNNÉ A METODY, ZAPOUZDŘENÍ TŘÍD 140
11.3 PŘETĚŽOVÁNÍ PROMĚNNÝCH A METOD 142

12 DATABÁZE MYSQL 146

12.1 TYPY TABULEK A DATOVÉ TYPY 146
12.1.1 Celá čísla 147
12.1.2 Desetinná čísla 148
12.1.3 Datum a čas 148
12.1.4 Řetězce znaků 148
12.1.5 Binární data 148
12.2 PŘÍSTUP K DATABÁZI MYSQL PROSTŘEDNICTVÍM PHP 149
12.2.1 Připojení k databázi a výběr databáze 149
12.2.2 Spouštění příkazů SQL 150
12.2.3 Zpracování výsledků příkazu SELECT 153
12.2.4 Uvolnění výsledku a uzavření připojení k databázi. 156

13 SEMESTRÁLNÍ PROJEKT 158

LITERATURA 159

VYSVĚTLIVKY K POUŽÍVANÝM SYMBOLŮM 160

1 Síťový protokol HTTP

V této kapitole se dozvíte:

- Jaké jsou základní vlastnosti protokolu HTTP.
- Co je to URI.
- Jaké jsou metody požadavku strany klienta.
- Jaké odpovědi může generovat strana serveru.

Po jejím prostudování byste měli být schopni:

- Znát základní vlastnosti protokolu HTTP.
- Znát strukturu URI nebo URL identifikátoru.
- Rozlišovat jednotlivé metody požadavku strany klienta.
- Poznat, zda bylo zpracování požadavku na serveru úspěšné a jaký je jeho výsledek.

Klíčová slova této kapitoly:

HTTP, HTML, klient, server, síťový protokol, URL, URI.

Doba potřebná ke studiu: 4 hodiny

Průvodce studiem



Pro studium této kapitoly je vhodné znát alespoň základní pojmy z oblasti počítačových sítí. Studium kapitoly není příliš náročné, seznamuje čtenáře se základními vlastnostmi protokolu HTTP, s URI adresami, s metodami požadavku strany klienta a s druhy odpovědí strany serveru.

Nejznámější a nejpoužívanější službou mezinárodní počítačové sítě Internet je služba World Wide Web (WWW). Mezi další služby Internetu pak patří například FTP, e-mail, instant messaging (IM), VoIP nebo DNS. Služba WWW kombinuje text, grafiku a ostatní multimédia a propojuje je pomocí hypertextových odkazů. Pojmy WWW a Internet bývají někdy používány jako ekvivalentní, ale ve skutečnosti je WWW pouze jednou ze služeb Internetu.

Služba WWW je realizována pomocí speciálních programů, kdy mezi nejznámější patří WWW prohlížeč (MS Internet Explorer, Firefox, Opera, Google Chrome atd.) a WWW server (Apache, MS Internet Information Server atd.). WWW prohlížeč bývá nejčastěji umístěn na straně pracovní stanice a v komunikaci vystupuje v roli klienta. WWW server pak plní roli serveru služby WWW. Komunikace mezi klientem a serverem je zajišťována pomocí protokolu HTTP – HyperText Transfer Protocol.

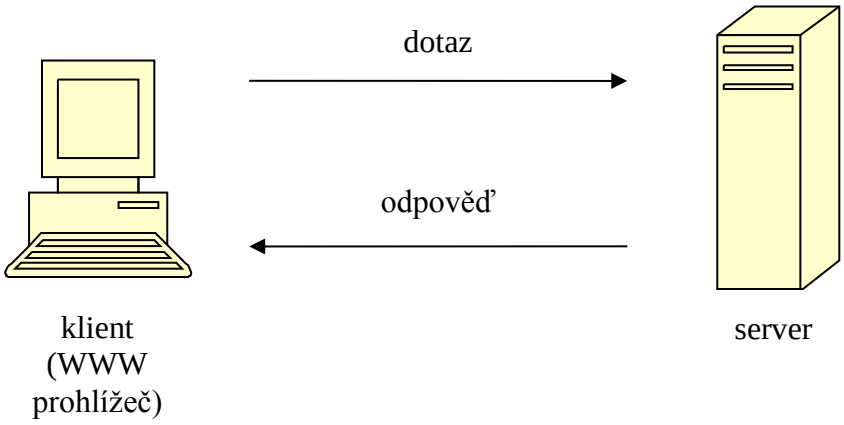
1.1 Protokol HTTP a jeho vlastnosti.

Protokol HTTP se poprvé objevil kolem roku 1990. Protokol původně vznikl v organizaci CERN (Evropská organizace pro jaderný výzkum), která sídlí

v Ženevě. Předchůdcem protokolu HTTP byl protokol Gopher, který se dnes již nepoužívá. Velmi významná byla verze 0.9 protokolu HTTP. V dnešní době se používá protokol HTTP ve verzi 1.1 z roku 1999, která je specifikovaná v RFC-2616 [1]. Protokol HTTP, stejně jako vše týkající se webu je dnes zaštiťován konsorciem W3C (www.w3.org).

Protokol HTTP je jedním z aplikačních protokolů TCP/IP. Své služby poskytuje nad transportním protokolem TCP, při jeho použití je komunikace spolehlivá a potvrzovaná. Pro použití protokolu HTTP je standardně používán port s číslem 80.

Komunikace pomocí protokolu HTTP je založena na architektuře klient-server. Klient, který bývá obvykle reprezentován WWW prohlížečem, odešle přesně strukturovaný požadavek na server. Server na základě požadavku zašle klientovi odpověď. V případě, že odeslání odpovědi není možné, odešle server chybové hlášení. Obecně platí, že pomocí protokolu HTTP lze přenášet libovolný obsah odpovídající doporučení MIME – Multipurpose Internet Mail Extension [2]. Nejčastěji bývá odpověď ve formě WWW stránky vytvořené v jazyce HTML [3]. Kromě toho ale může být odpovědí PDF soubor, obrázek, zvukový soubor, video soubor atd.



Obrázek 1.1 Komunikace klient-server.

Komunikace nejčastěji probíhá tak, že uživatel napíše do adresního řádku WWW prohlížeče URI identifikátor (adresu) objektu, který chce prohlížet. Prohlížeč nejprve z identifikátoru vyřízne jméno serveru a pomocí DNS jej přeloží na IP adresu. Poté klient naváže s touto IP adresou spojení pomocí protokolu TCP. Do takto vytvořeného kanálu vloží prohlížeč HTTP dotaz a server na něj reaguje odesláním HTTP odpovědi ve stejném kanálu. Nakonec zobrazí prohlížeč odpověď uživateli. V případě, že se výsledná webová stránka

skládá z více objektů, je nutné každý objekt stáhnout jedním HTTP dotazem. Ve starších verzích protokolu HTTP pro každý dotaz vždy navazovalo nové TCP spojení. Prvním dotazem se stáhl základní text stránky, který na místech objektů (obrázků atd.) obsahoval odkazy a poté se postupně stahovaly jednotlivé objekty. V protokolu HTTP v1.1 se již implicitně předpokládá, že TCP spojení mezi klientem a serverem bude navázáno pro celou sadu dotazů.

Odpověď serveru může být statická nebo dynamická. V případě statické odpovědi klient požaduje konkrétní soubor, který se nachází v adresářové struktuře serveru. V tom případě odešle server klientovi obsah tohoto souboru. Dynamická odpověď může být dvojího typu:

- Dynamická odpověď generovaná na straně serveru – v tomto případě obvykle server komunikuje s externí aplikací, která na základě parametrů zadaných klientem vygeneruje obsah odpovědi. Nejčastěji je výsledkem dynamicky vygenerovaná WWW stránka, která je poté serverem odeslaná klientovi.

- Dynamická odpověď generovaná na straně klienta – v tomto případě server vrací klientovi WWW stránku, která obsahuje dynamické prvky, např. Java Applet. Běh dynamického prvku je pak realizován v prohlížeči na straně klienta.

Komunikace protokolem HTTP tedy probíhá následujícím způsobem:

1. HTTP klient naváže prostřednictvím protokolu TCP komunikační spojení se stranou HTTP serveru.

2. Klient odešle požadavek (dotaz) na server.

3. Server zpracuje požadavek klienta a odešle odpověď, která má obecně multimediální charakter. V případě, že z důvodu nějaké chyby není možné odeslat klientovi požadovanou odpověď, server odešle klientovi chybovou zprávu.

4. Během jednoho spojení je možné poslat více dotazů a odpovědí.

5. Spojení mezi klientem a serverem je ukončeno.

1.2 URI



K identifikaci objektů ve webovém prostředí se používají tzv. URI (Uniform Resource Identifier) identifikátory. Speciálnějším případem URI je URL (Uniform Resource Locator) [3], které jsou v praxi používanější. Obecný tvar URL je:

```
schéma://server[:port][cesta[?dotaz#[fragment]]]
```

kde

- `schéma` – označuje použité schéma. V našem případě zde bude uvedeno http. Kromě toho zde může být ftp, mailto, telnet atd.
- `server` – označuje jedinečné jméno serveru, na který je odesílán požadavek ze strany klienta. Server může být identifikován svým doménovým jménem nebo IP adresou.
- `port` – číslo služby v rozsahu 0 až 65535, nad kterou je na daném serveru aktivní aplikace. Jak již bylo uvedeno, standardně je při komunikaci protokolem HTTP používán port s číslem 80. Pokud chceme použít tento port, není potřeba jej uvádět.
- `cesta` – označuje cestu v adresářové struktuře serveru, která určuje, jaký soubor je požadován. Začíná znakem lomítka „`/`“
- `dotaz` – zde jsou uvedeny parametry dotaz, může jich být několik. Parametr se zadává ve formě

  ```
  jmeno_parametru=hodnota_parametru
  ```

  Pokud je potřeba použít více parametrů jsou odděleny znakem „`&`“. Hodnota parametru se zadává bez uvozovek nebo apostrofů.
- `fragment` – začíná znakem „`#`“ a určuje konkrétní umístění na výsledné webové stránce. Po načtení stránky se okno prohlížeče přesune na umístění fragmentu.

Části URL identifikátoru umístěné mezi znaky „`[`“ a „`]`“ jsou nepovinné.

Příklady:

1. `http://www.osu.cz`



URL představuje požadavek klienta na obsah kořenového adresáře serveru *www.osu.cz*.

2. `http://www.osu.cz:8080/studium/index.html`

   URL obsahuje specifikaci portu, jedná se o port s číslem *8080*. Klient požaduje stránku *index.html*, která se nachází v adresáři *studium*.

3. `http://www.osu.cz/index.php?kategorie=10&id=1`

   URL obsahuje dva parametry. První parametr s názvem *kategorie* má hodnotu *10*, druhý parametr s názvem *id* má hodnotu *1*.

4. `http://www.osu.cz/index.php?kategorie=1#odstavec3`

   URL obsahuje jeden parametr s názvem *kategorie*, který má hodnotu *1*, a fragment s názvem *odstavec3*.

Pokud URI nezačíná názvem schématu, hovoříme o tzv. relativním URI. Relativní URI je vždy vztaženo k nějakému absolutnímu URI – k nějaké bázi. Bází může být URI aktuálního dokumentu, předešlého dokumentu nebo může být báze nastavena implicitně pro celou aplikaci. Relativní URI může ve své cestě obsahovat znak „`.`“ pro reprezentaci aktuálního adresáře nebo znak „`..`“ pro reprezentaci nadřízeného adresáře.

Příklady:

Mějme soubor s adresou *http://www.osu.cz/adresar/soubor.htm*. Tento soubor může obsahovat relativní odkazy:

- *#odstavec1* – tomuto odkazu odpovídá absolutní URI *http://www.osu.cz/adresar/soubor.htm#odstavec1*
- *soubor2.htm* – tomuto odkazu odpovídá absolutní URI *http://www.osu.cz/adresar/soubor2.htm*
- *../soubor3.htm* – tomuto odkazu odpovídá absolutní UR*I* *http://www.osu.cz/soubor3.htm*

1.3 Metody požadavku strany klienta



Požadavek strany klienta, kterým se odesílá HTTP dotaz na stranu serveru má pevně danou strukturu, viz. obrázek.

Metoda	`GET / HTTP/1.1`
Hlavičky požadavku	`Host: www.osu.cz` `Connection: keep-alive`
Prázdný řádek	
Data (volitelná část)	`kategorie=10&id=10`

Obrázek 1.2 Struktura HTTP dotazu.

První částí je specifikace metody požadavku. Dále následuje záhlaví, které je tvořeno jednotlivými hlavičkami. Na konci každé hlavičky je vždy umístěn konec řádku CRLF. Za poslední hlavičkou je vložen prázdný řádek, opět pomocí CRLF. Poslední částí jsou přenášená data. Tato část je volitelná.

Metoda

Protokol HTTP ve verzi 1.1 podporuje metody GET, POST, HEAD, OPTIONS, TRACE, PUT a DELETE. Každý WWW server musí povinně podporovat metody GET a HEAD. Ostatní metody nemusí být podporován. Specifikace metody požadavku je vždy ve tvaru

```
Metoda_požadavku URI_požadavku Protokol/Verze
```

Jako `Metoda_požadavku` může být uvedena jakákoliv z metod podporovaných protokolem HTTP. `URI_požadavku` představuje relativní cestu ke zdroji (WWW stránce), který klient požaduje. V poslední části bude vždy uvedeno `HTTP/1.1`.

Metoda GET slouží k získání statických zdrojů ze strany serveru, které mohou být obecně multimediálního charakteru. Těmito zdroji mohou být statické WWW stránky, obrázky, zvukové soubory atd. Metodu GET je možné použít i pro získání dynamického obsahu a to jak na straně serveru, tak na straně klienta. Obecně platí, že veškeré adresy, které klient napíše do adresního řádku WWW prohlížeče se odesílají metodou GET. Stejně tak se metodou GET odesílají všechny hypertextové odkazy, tj. vše, co je v HTML kódu webové stránky označeno tagem `<a> … </a>`. Důležitou vlastností metody GET je, že veškeré zdroje získané touto metodou jsou bookmarkovatelné. Uživatel si

může adresu zdroje uložit a navštívit ji znovu. Pokud jsou součástí URL i parametry, je možné navštívit odkaz se stejnými parametry. To může být za určitých okolností výhodou. Nevýhodou je, že délka URL je omezená, většinou je to 255 znaků. Další nevýhodou je, že parametry, které jsou součástí URL, jsou viditelné. Používat metodu GET pro přihlašování, kdy by adresa byla např.

```
http://www.stranka.cz/index.php?jmeno=pepa&heslo=123
```

by tedy nebylo zrovna bezpečné.

Metoda HEAD je velmi podobná metodě GET a stejně jako metoda GET musí být povinně implementována každým WWW serverem. Hlavním rozdílem mezi těmito metodami je, že metoda HEAD vrací pouze hlavičku odpovědi, al nevrací tělo odpovědi. Metoda HEAD se používá zejména v případech, kdy potřebujeme ověřit, zda požadovaný zdroj vůbec existuje, jaký má formát atd.

Další metodou je metoda POST, která bývá implementována prakticky na každém WWW serveru, i když patří mezi nepovinné metody. Společně s metodou GET patří mezi nejpoužívanější metody. Metoda POST slouží jednak získání dynamické odpovědi ze strany serveru a také k odesílání dat na server (upload na server). Na rozdíl od metody GET, nejsou dotazy odesílané metodou POST bookmarkovatelné. Samozřejmě je možné si uložit URL dotazu, který se odesílá metodou post, ale při zadání této adresy do prohlížeče může být výsledek dotazu úplně jiný nebo nemusí URL vůbec fungovat. Výhodou této metody ovšem je, že parametry odesílané na server nejsou viditelné v URL, ale jsou ukryty v těle dotazu. Takže zejména v případě odesílání citlivých dat je tato metoda výhodnější. Druhou výhodou je, že objem dat odesílaných na server není prakticky omezen. Na server je možné odesílat (uploadovat) soubory o velikosti několika GB (giga bajtů).

Metoda PUT slouží k uložení statických informací na WWW server. Při použití této metody je součástí dotazu URI zdroje, který má být uložen na server. Z bezpečnostních důvodů ale tato metoda nebývá povolena. Provozovatel WWW serveru určitě nemá zájem na tom, aby mohl kdokoliv ukládat soubory do adresářů na serveru.

Metoda DELETE umožňuje smazání zdroje, jehož URI je specifikováno v HTTP dotazu. Stejně jako metoda PUT bývá z bezpečnostních důvodů zakázána.

Metoda OPTIONS se používá ke zjištění komunikačních vlastností serveru nebo požadovaného URI. Pokud chceme zjistit vlastnost celého serveru, uvedeme místo URI znak hvězdičky „*“.

Metoda TRACE se používá v případě, kdy komunikace mezi klientem a serverem probíhá pomocí proxy serverů nebo bran. Odpovědí na dotaz metodou TRACE pak je seznam mezilehlých systémů (proxy serverů nebo bran), pomocí nichž byla provedena komunikace.

Hlavičky požadavku

Hlavičky požadavku obsahují jeden nebo více řádků, které poskytují serveru dodatečné informace o požadavku. Každý řádek hlavičky je ve tvaru:

```
Klíčové_slovo: Hodnota
```

Seznam klíčových slov je pevně dán. Povinná je pouze hlavička označená klíčovým slovem *Host*. Tato hlavička určuje adresu WWW serveru, na který je odesílán dotaz a dále určuje port, pomocí něhož je dotaz odesílán. Standardní port s číslem 80 nemusí být uveden. Ostatní hlavičky jsou nepovinné. Mezi nejpoužívanější hlavičky patří následující:

- *Accept* – určuje MIME typy, které smí strana klienta akceptovat, např. *text/html*.
- *Accept-Charset* – určuje znakové sady, které smí klient použít, např. *ISO-8859-2*.
- *Accept-Language* – záhlaví určuje, jaké jazyky akceptuje strana klienta, hodnotou záhlaví jsou standardní jazykové kódy podle specifikace RFC 1766, např. *en*, *en-us* atd.
- *Connection* – pomocí tohoto záhlaví je možné určit, že klient požaduje persistentní spojení protokolem HTTP, tj. že při navázání jediného spojení je možné obdržet více WWW stránek. Pro vytvoření persistentního spojení se používá hodnota *keep-alive*.
- *Content-Length* – záhlaví udává velikost datové části požadavku v bytech, používá se pouze u metody POST.

- *Content-Type* – záhlaví se používá spíše v odpovědi ze strany serveru, ale je možné ho použít i v požadavku. Používá se zejména při požadavcích metodou POST a PUT.

- *Cookie* – záhlaví se používá pro odeslání hodnot cookies na server.

- *If-Modified-Since* – záhlaví se používá v případě, kdy klient požaduje zaslání WWW stránky pouze v případě, pokud se změnila po uvedeném datu. Toto záhlaví umožňuje prohlížečům ukládat si stránky do paměťové cache a nenačítat je vždy znovu ze serveru.

- *Pragma* – toto záhlaví může mít pouze jednu hodnotu a tou je *no-cache*. Tato hodnota určuje, že server má odeslat WWW stránku, i když je uložena v cache paměti prohlížeče.

- *Referer* – záhlaví udává URL adresu WWW stránky, kterou klient požaduje. Většina WWW prohlížečů toto záhlaví odesílá v hlavičce požadavku.

- *User-Agent* – záhlaví slouží k identifikaci WWW prohlížeče nebo jiného typu klienta, např. Firefox, MS Internet Explorer atd.

Příklady HTTP dotazů



1. Dotaz metodou GET na kořenový adresář serveru *www.osu.cz*

```
GET / HTTP/1.1
Host: www.osu.cz
Accept: text/html, application/xhtml+xml,
 application/xml
Accept-Encoding: gzip, deflate
Accept-Language: cs, en-us, en
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 6.1; rv:16.0)
 Gecko/20100101 Firefox/16.0
```

2. Dotaz metodou GET na adresu

   http://prf.osu.cz/kip/index.php?kategorie=502

```
GET /kip/index.php?kategorie=502 HTTP/1.1
Host: prf.osu.cz
```

```
Accept: text/html, application/xhtml+xml,
 application/xml
Accept-Encoding: gzip, deflate
Accept-Language: cs, en-us, en
Connection: keep-alive
```

3. Dotaz metodou POST na adresu

   http://www.osu.cz/index.php?kategorie=892

```
POST /index.php?kategorie=892 HTTP/1.1
Host: www.osu.cz
Accept: text/html, application/xhtml+xml,
 application/xml
Accept-Language: cs, en-us, en
Connection: keep-alive

zpravas=a&lidis=a&kategs=a&search=ahoj&go=Hledat
```

4. Dotaz metodou PUT, který realizuje vložení souboru test.html do adresáře /testovaci na serveru www.osu.cz

```
PUT /testovaci/test.html HTTP/1.1
Host: www.osu.cz
Connection: keep-alive
Content-Length: 50

<HTML>
  <HEAD>
    <TITLE>Testovací dotaz metodou PUT</TITLE>
  </HEAD>
  <BODY>
    Obsah WWW stránky
  </BODY>
</HTML>
```

5. Požadavek na zrušení souboru *test.html*, který se nachází ve složce */testovaci* na serveru *www.osu.cz*

```
DELETE /testovaci/test.html HTTP/1.1
Connection: keep-alive
Host: www.osu.cz
```

1.4 Odpověď strany serveru



Poté, co WWW server obdrží požadavek ze strany klienta, generuje odpověď na tento požadavek. Struktura odpovědi je podobná jako struktura požadavku strany klienta.

Stavový řádek	`HTTP/1.1 200 OK`

Hlavičky odpovědi	`Server: Apache`
	`Content-Type: text/html`
	`Connection: close`

Prázdný řádek	

Tělo odpovědi	`<html>`
	`<head>`
	`<title>Stránka</title>`
	`</head>`
	`<body>`
	`ahoj`
	`</body>`
	`</html>`

Obrázek 1.3 Struktura odpovědi strany serveru.

První částí odpovědi strany serveru je stavový řádek, který říká, zda bylo zpracování požadavku úspěšné či nikoliv. V případě neúspěšného zpracování požadavku je zde uveden popis chyby, která nastala. Další částí jsou hlavičky, které jsou velmi podobné hlavičkám HTTP dotazu. Za poslední hlavičkou následuje prázdný řádek. Poslední částí je Tělo odpovědi, které obsahuje samotný obsah odpovědi. V případě WWW stránky je zde uveden její HTML kód, v případě obrázku je to obrázek v daném formátu (JPEG, PNG, GIF).

Stavový řádek

Stavový řádek odpovědi ze strany serveru je vždy v následujícím tvaru:

```
Protokol/Verze Kód_stavu Popis
```

První část, `Protokol/Verze` bude v našem případě vždy `HTTP/1.1`. `Kód_stavu` je trojmístné přirozené číslo v rozsahu 100 – 599 a dělí se do pěti kategorií:

- 1xx – informativní stavové kódy, označující, že zpracování pokračuje dále.

- 2xx – stavové kódy označující úspěšné splnění požadavku ze strany klienta.

- 3xx – stavové kód pro přesměrování, další akce se bude týkat jiného URI.

- 4xx – stavové kódy pro identifikaci chyby na straně klienta, např. syntaktická chyba v dotazu, dotaz na neexistující WWW stránku.

- 5xx – stavové kódy pro identifikaci chyby na straně serveru, např. chyba v CGI skriptu.

Poslední část stavového řádku, `Popis`, pak obsahuje slovní popis stavu.

Hlavičky odpovědi

Hlavičky odpovědi jsou podobné hlavičkám požadavku, také obsahují jeden nebo více řádků, které poskytují klientovi dodatečné informace o odpovědi nebo o serveru samotném. Každý řádek hlavičky je ve tvaru:

```
Klíčové_slovo: Hodnota
```

Stejně jako v případě požadavku, i zde je seznam klíčových slov je pevně dán. Mezi nejpoužívanější záhlaví patří:

- *Allow* – záhlaví určuje, které metody požadavků podporuje WWW server. Toto záhlaví je vygenerováno, pokud je v požadavku použita metoda OPTIONS.

- *Cache-control* – záhlaví určuje, zda a za jakých podmínek smí klient uložit WWW stránku do své cache paměti. Přípusné hodnoty jsou *public, private, no-cache, no-store, must-revalidate, proxy-revalidate, max-age=xxx, s-max-age=xxx* (*xxx* označuje čas v sekundách)

- *Connection* – pokud je hodnotou tohoto záhlaví slovo close, je tím klient informován, že persistentní konektivita byla ukončena.

- *Content-Language* – záhlaví udává, v jakém jazyce je dokument odeslaný klientovi jako odpověď. Hodnotami jsou, stejně jako v případě požadavku, standardní jazykové kódy dle RFC 1766, např. *en, en-us* atd.

- *Content-Length* – záhlaví udává velikost odpovědi v bytech, která byla odeslána klientovi. Tato hlavička je povinná v případě, že má být zachováno persistentní spojení.

- *Content-Type* – hodnotou záhlaví je MIME typ odpovědi, která byla odeslána klientovi. Je vždy ve tvaru *maintype/subtype* (pro registrované typy) a *maintype/x-subtype* (pro neregistrované typy). Příkladem hodnoty tohoto záhlaví je *text/html*, *image/png* atd.

- *Last-Modified* – záhlaví udává datum a čas poslední změny dokumentu.

- *Location* – toto záhlaví je povinné v případě, že kód stavu odpovědi je 3xx. Hodnotou záhlaví je adresa dokumentu, na kterou vede přesměrování.

- *Pragma* – stejně jako v případě požadavku, i zde udává hodnota *no-cache* informaci, že dokument nemá být uložen do cache paměti prohlížeče.

- *Server* – záhlaví obsahuje informaci o identifikaci WWW serveru, hodnotou může být např. *Apache*.

- *Set-Cookie* – záhlaví specifikuje cookie, které souvisí s danou WWW stránkou

Příklady odpovědí strany serveru

1. Odpověď při úspěšném zpracování požadavku



```
HTTP/1.1 200 OK
Content-Type: text/html
Server: Apache
Connection: close
Pragma: no-cache
Date: Wed, 10 Apr 2013 13:19:05 GMT

<HTML>
… Obsah WWW stránky …
</HTML>
```

2. Odpověď generující kód 301

```
HTTP/1.1 301 Moved Permanently
```

```
Server: Apache
Connection: close
Location: http://prf.osu.cz/kip/

<HTML/>
  Obsah WWW stránky
</HTML>
```

3. Odpověď generující chybovou zprávu s kódem 404 – stránka nenalezena

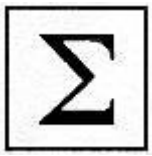
```
HTTP/1.1 404 Not Found
Server: Apache
Connection: close

<HTML/>
  Obsah chybové WWW stránky
</HTML>
```



Kontrolní otázky:

1. Jaký je princip komunikace klient-server?
2. Jaké jsou metody požadavků protokolu HTTP?
3. Jaká je struktura požadavku strany klienta?
4. Jaká je struktura odpovědi strany serveru?



Shrnutí obsahu kapitoly

V této kapitole jste se seznámili s protokolem HTTP, který se používá při komunikaci v rámci služby WWW. Základní vlastnosti tohoto protokolu, struktura URI adresy (identifikátoru), metody požadavku strany klienta a druhy odpovědi strany serveru jsou důležité znalosti potřebné pro tvorbu webových aplikací.

2 Formuláře v jazyce HTML

V této kapitole se dozvíte:

- K čemu slouží formuláře v jazyce HTML.
- Jaké jsou nejdůležitější formulářové prvky.
- Jaké vlastnosti je možné nastavit těmto prvkům.

Po jejím prostudování byste měli být schopni:

- Vytvořit formulář v jazyce HTML.
- Umět používat jednotlivé formulářové prvky a umět jim nastavovat parametry.

Klíčová slova této kapitoly:

HTML, formulář, parametry.

Doba potřebná ke studiu: 4 hodiny

Průvodce studiem



Studium této kapitoly je velmi důležité pro tvorbu webových stránek nebo aplikací v jazyce PHP. Dynamické webové stránky nebo aplikace se neobejdou bez formulářů, protože pomocí nich můžeme získávat vstupy od uživatelů. Studium kapitoly není náročné, čtenáře seznamuje s jednotlivými formulářovými prvky a s jejich vlastnostmi.

Při tvorbě dynamických webových stránek hrají velmi důležitou roli tzv. formuláře. Pomocí formulářů je možné získávat vstupy od uživatele. Při „surfování“ na webu jste se s formuláři už jistě setkali. Pomocí formulářů se přihlašujete do systémů, které vyžadují autorizaci (např. přihlášení do e-mailu). Dále je pomocí formulářů realizováno vyhledávání na webu, registrace do různých systémů (např. elektronické obchody) apod. Nejčastěji formuláře obsahují textová políčka, políčka pro heslo, zaškrtávací políčka, roletkové menu nebo přepínače. Pro odeslání dat z formuláře na server se nejčastěji používají tlačítka (ale je možné, aby bylo řešeno i pomocí obrázku – graficky zpracovaného tlačítka. Většina formulářů bývá na server odesílána metodou POST, ale někdy se používá i metoda GET. V případě použití metody GET se data z formuláře přidají do URL adresy, která je uvedená v adresním řádku prohlížeče. Všechny prvky, které se vyskytují ve formuláři, jsou po odeslání formuláře přidány jako parametry HTTP dotazu ve tvaru

```
jmeno_parametru=hodnota_parametru
```

kde název formulářového prvku představuje `jmeno_parametru` a `hodnota_parametru` představuje hodnotu tohoto prvku. Rozdíl je pouze v umístění parametrů. V případě odeslání metodou GET jsou tyto parametry

součástí URL adresy zobrazené v adresním řádku prohlížeče. V případě metody POST jsou uvedeny v datové oblasti (v těle) HTTP dotazu.

Nyní si popíšeme nejdůležitější značky (tagy) pro tvorbu formulářů a jejich parametry (atributy).

2.1 Značka FORM



Nejdůležitější značkou pro tvorbu formulářů je párová značka `FORM`. Tvoří formulář samotný a vše, co má tvořit jeden formulář musí být uvedeno v této značce, tj. mezi `<FORM>` a `</FORM>`. Na jedné stránce může být více formulářů, ale neměly by se navzájem překrývat. Uvedení značky `<FORM> ... </FORM>` se na vzhledu stránky nijak neprojeví, tato značka není na první pohled viditelná.

Parametry

- `action` – v tomto parametru je uvedena absolutní nebo relativní adresa, na kterou má být obsah formuláře odeslán. Pokud by tento parametr nebyl uveden, po stisknutí tlačítka pro odeslání formuláře se nic nestane.

- `name` – jméno formuláře, jednotlivé formuláře můžeme pojmenovat a potom se na ně odkazovat.

- `method` – metoda, kterou má být obsah formuláře odeslán na WWW server. Může zde být uvedeno GET nebo POST. Pokud není tento parametr uveden, odešle se obsah formuláře metodou GET.

- `enctype` – hodnota tohoto parametru udává typ odesílaných dat. Výchozí hodnotou, kterou není potřeba uvádět, je hodnota `application/x-www-form-urlencoded`. Pokud je potřeba pomocí formuláře odeslat na server binární data (např. obsah souboru), je potřeba hodnotu parametru nastavit na `multipart/form-data`. Odesílání binárních dat na server je možné pouze pomocí metody POST, metoda GET to neumožňuje.

Příklad použití značky `FORM` na WWW stránce. Formulář odesílá metodou POST binární data na server, obsah formuláře je odeslán na soubor *index.php* (tento soubor musí být ve stejné složce jako soubor, ve kterém je formulář uveden).

Výpis 2.1 Použití značky FORM

```
<HTML>
  <HEAD>
    <TITLE>Příklad formuláře</TITLE>
  </HEAD>
  <BODY>
    <H1>Dotaz metodou POST</H1>
    <FORM METHOD="POST" ACTION="index.php"
     ENCTYPE="multipart/form-data">
      Obsah formuláře
    </FORM>
  </BODY>
</HTML>
```



2.2 Značka INPUT



Značka `INPUT` je nepárová a umožnuje zadání vstupů od uživatele, který může mít rozličný charakter. Nejdůležitějším parametrem je `type`. Parametr `type` určuje druh vstupního pole a můžu nabývat následujících hodnot:

- `text` – textové políčko představující jeden řádek textu. Ve formulářích bývá tento typ používán např. pro zadání uživatelského jména při přihlašování, pro zadání hledaného textu apod.

- `password` – políčko pro vložení hesla. Při zadávání znaků se nezobrazují znaky, ale nějaký zástupný symbol (hvězdička apod.). Ve formulářích se používá pro zadání hodnot, které by neměly být přímo čitelné na stránce, nejčastěji pro zadání hesla při přihlašování.

- `hidden` – skrytý prvek, není na stránce viditelný, ale je možné si do něj uložit hodnotu. Nejčastěji se používá pro vložení pomocných hodnot na stránku (např. hodnota nějakého nastavení apod.)

- `checkbox` – zaškrtávací políčko, které umožňuje vybrat více hodnot současně. Používá se například pro zadání zájmů uživatele, pro zadání dovedností uživatele apod.

- `radio` – přepínač, který umožňuje vybrat pouze jednu z možností. Typickým příkladem použití je zadání pohlaví uživatele, který se registruje do systému (např. do e-shopu).

- `file` – pole pro výběr souboru, který má být odeslán na server. Na stránce je prvek zobrazen jako textové políčko s cestou k souboru a tlačítko, které umožňuje najít soubor v adresářové struktuře disku.

- `submit` – tlačítko, po jehož stisknutí se obsah formuláře odešle na server. Při vkládání tlačítka do formuláře můžeme nastavit, jaký text se na tlačítku zobrazí.

- `reset` – tlačítko pro „resetování“ (vynulování) formuláře. Po stisknutí tohoto tlačítka se všechny prvky nastaví na hodnotu, kterou měly po načtení stránky.

- `button` – tlačítko, které nemá žádnou speciální funkci. Je možné mu přiřadit funkčnost například zavoláním funkce v JavaScriptu (což je možné i u ostatních tlačítek).

- `image` – umožnuje vložení obrázku na WWW stránku, která se chová jako tlačítko, tj. po kliknutí odešle data z formuláře na server.

Některé parametry prvku `INPUT` jsou společné pro všechny druhy vstupního pole, u některých se chování prvku po nastavení parametru liší. Společné parametry, jejichž chování je stejné u všech druhů prvku `INPUT` jsou:

- `type` – hodnota parametru určuje, jakého druhu je vstupní pole nebo tlačítko. Jak již bylo uvedeno, přípustnými hodnotami parametru jsou `text`, `password`, `hidden`, `submit`, `reset`, `button`, `checkbox`, `radio`, `file` a `image`.

- `name` – název formulářového prvku, při odeslání formuláře na server je název uveden jako parametr v HTTP dotazu.

- `disabled` – uvedením tohoto parametru znemožníme editaci pole. Hodí se například v situacích, kdy má být v textovém poli uživateli zobrazen nějaký text, ale uživatel nesmí tento text editovat.

U následujících parametrů se liší jejich vliv na formulářový prvek a nebo mají smysl pouze u některých druhů prvku `INPUT`, popis bude uveden dále:

- `value`

- `size`

- maxlength
- checked
- src

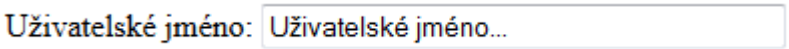
Textové pole

Nastavením parametru type na hodnotu text je na stránce zobrazeno textové políčko, do kterého je možné zadat jeden řádek textu. Kromě typu a názvu prvku mají u tohoto prvku smysl parametry value, size a maxlength. Hodnota parametru value určuje text, který bude zobrazen v textovém poli po načtení stránky. Pokud je součástí formuláře tlačítko typu reset, je po jeho stisknutí v textovém poli zobrazena hodnota nastavená v parametru value. Parametr size určuje šířku (velikost) textového pole. Hodnotou parametru je číslo, které udává počet znaků viditelných v prvku. Pokud není nastaven parametr size, má textové pole výchozí délku nastavenou prohlížečem. Parametrem maxlength je možné omezit délku řetězce zadaného do textového pole. Hodnotou parametru je opět číslo, které udává maximální počet znaků, které je možné do pole vepsat. Omezení počtu znaků zadaných do pole je možné využít například ve formuláři, do kterého uživatel zadává svou adresu. Pole pro zadání PSČ by mělo obsahovat pouze 5 znaků (případně 6 znaků, pokud se zadává PSČ včetně mezery).

Výpis 2.2 Ukázka použití textového pole.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="text" name="uzivatel"
       value="Uživatelské jméno..." size="50"
       maxlength="30">
    </FORM>
  </BODY>
</HTML>
```





Obrázek 2.1 Zobrazení textového pole na stránce.

Heslo

Pole pro vložení hesla vytvoříme nastavením parametru `type` na hodnotu `password`. U tohoto typu vstupního pole je možné použít stejné parametry jako u textového pole, tj. `value`, `size` a `maxlength`. Jediný rozdíl oproti textovému poli je v tom, že při zadávání znaků do pole nejsou znaky viditelné, ale jsou místo nich zobrazeny zástupné symboly, např. hvězdička nebo plné kolečko (záleží na nastavení prohlížeče nebo na použitém operačním systému). Text zadaný v tomto poli tedy není přímo viditelný na monitoru. Při odeslání formuláře se ale samozřejmě odešle text ve své původní podobě. Použití tohoto typu je typické pro citlivá data, tedy kromě hesel různé PIN kódy apod.

Výpis 2.3 Ukázka použití pole pro heslo.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="password" name="heslo"
       value="Heslo..." size="25" maxlength="25">
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.2 Zobrazení pole pro heslo na stránce.

Skryté pole

Pokud prvku `INPUT` nastavíme parametr `type` na hodnotu `hidden`, není tento prvek nijak viditelný na stránce. Kromě názvu prvku má u tohoto typu prvku význam pouze parametr `value`. Nejčastější použití je pro uložení pomocných hodnot, jejichž zobrazení by uživatele mátlo a bylo by pro něj zbytečné. Například ve formuláři týkajícího se výrobků by zde mohla být uložena sazba DPH, ID výrobku z databáze atd.

Výpis 2.4 Ukázka použití skrytého prvku.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="hidden" name="dph" value="0.21">
    </FORM>
  </BODY>
```

```
</HTML>
```

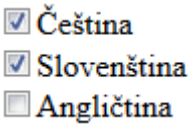
Zaškrtávací pole

Pro vytvoření zaškrtávacího pole je nutné nastavit parametr `type` na hodnotu `checkbox`. Jednotlivá zaškrtávací pole jsou navzájem nezávislá, zaškrtnutí jednoho z nich nemá vliv na ostatní pole. U tohoto typu vstupního pole mají smysl pouze parametry `type`, `name`, `value` a `checked`. Uvedením parametru `checked` docílíme toho, že po načtení je příslušné zaškrtávací pole zaškrtnuté. Hodnota nastavená prvku pomocí parametru `value` je po odeslání formuláře přenesena na server.

Výpis 2.5 Ukázka použití zaškrtávacího pole - různé pojmenování prvků

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="checkbox" name="cestina"
       value="cz" checked>Čeština<BR>
      <INPUT type="checkbox" name="slovenstina"
       value="sk" checked>Slovenština<BR>
      <INPUT type="checkbox" name="anglictina"
       value="en">Angličtina<BR><BR>
    </FORM>
  </BODY>
</HTML>
```





Obrázek 2.3 Zobrazení zaškrtávacího pole na stránce.

Při uvedeném použití zaškrtávacího pole se jednotlivá zaškrtávátka jmenují různě. Po odeslání formuláře na server budou součástí HTTP dotazu parametry ve tvaru

```
cestina=cz&slovenstina=sk
```

Zaškrtávací pole je ale možné použít i jiným způsobem, a to tak, že všechna zaškrtávátka patřící do stejné skupiny (v tomto případě to je jazyk) se jmenují stejně. Tato možnost je zobrazena v následujícím výpisu.

Výpis 2.6 Ukázka použití zaškrtávacího pole – shodné pojmenování prvků.



```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="checkbox" name="jazyk" value="cz"
       checked>Čeština<BR>
      <INPUT type="checkbox" name="jazyk" value="sk"
       checked>Slovenština<BR>
      <INPUT type="checkbox" name="jazyk"
       value="en">Angličtina<BR><BR>
    </FORM>
  </BODY>
</HTML>
```

Při použití druhého způsobu pojmenování jednotlivých prvků bude formulář na WWW stránce zobrazen úplně stejně. Rozdíl ale bude v parametrech HTTP dotazu. Po odeslání formuláře budou ve tvaru

```
jazyk=cz&jazyk=sk
```

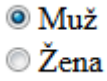
Jeden parametry je tedy odeslán 2x, ale pokaždé s jinou hodnotou. Rozhodnutí zda použít první nebo druhý způsob pojmenování prvků typu checkbox záleží pouze na tvůrci WWW stránky. Podle vybraného způsobu je pak nutné přizpůsobit zpracování formuláře na straně serveru. Pokud je použitý druhý způsob, je potřeba počítat s tím, že některý parametr (v tomto případě parametr s názvem jazyk) může nabývat více hodnot.

Přepínací pole

Nastavením parametru `type` na hodnotu `radio` vytvoříme na WWW stránce radio boxy, které umožňují výběr pouze jedné z nabízených možností. Stejně jako v předchozím případě, i zde mají dále smysl jen parametry `name`, `value` a `checked`. Pokud prvky typu `radio` pojmenujeme stejně, tj. hodnota parametru `name` je u všech stejná, tvoří jednu skupinu, v níž lze vybrat pouze jednu z nabízených možností. Rozdíl je pouze v hodnotě (parametru `value`) jednotlivých možností. Použitím parametru `checked` u některé z hodnot má za efekt výběr dané hodnoty. Pokud je tento parametr uveden u více různých možností v rámci jedné skupiny, je vybrána poslední z nich (stránka se načítá postupně, postupně se „vybírají“ možnosti v pořadí, v jakém jdou za sebou, ale po výběru poslední hodnoty už tento výběr není zrušen další hodnotou). Při odeslání na server je pak odeslána hodnota vybraného prvku, takže poznáme, kterou možnost uživatel zvolil.

Výpis 2.7 Ukázka použití radio boxu.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="radio" name="pohlavi" value="muz"
       checked>Muž<BR>
      <INPUT type="radio" name="pohlavi"
       value="zena">Žena<BR>
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.4 Zobrazení radio boxu na stránce.

Po odeslání uvedeného formuláře je součástí HTTP dotazu parametr ve tvaru

```
pohlavi=muz
```

U prvku typu radio nemůže nastat, že nějaký parametr nabývá v rámci jednoho HTTP dotazu více hodnot.

Pole pro výběr souboru

Tento formulářový prvek vložíme do WWW stránky nastavením parametru `type` na hodnotu `file`. Standardně se tento prvek skládá ze dvou částí – z textového pole pro vložení cesty k souboru (v rámci adresářové struktury disku) a z tlačítka Procházet (Browse), pomocí něhož může uživatel vybrat příslušný soubor na disku počítače. Použití tohoto formulářového prvku vyžaduje nastavení parametru *enctype* u značky FORM na `multipart/form-data` a pro odeslání formuláře musí být použita metoda POST. Kromě parametru name je možné u tohoto prvku použít parametry size, value a maxlength. Jejich chování je stejné jako v případě textového pole.

Výpis 2.8 Příklad použití pole pro vložení názvu souboru.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="file" name="soubor" size="50"
       maxlength="50"><BR>
    </FORM>
```

```
  </BODY>
</HTML>
```



Obrázek 2.5 Zobrazení pole pro výběr souboru na stránce.

Obrázek

Nastavením parametru `type` na hodnotu `image` je na WWW stránce vytvořeno odesílací tlačítko, které může být tvořeno obrázkem. Kromě typu je možné použít parametry `name` a `src`. Parametr `src` určuje cestu k obrázku, který má tvořit tlačítko. Při odeslání formuláře jsou navíc odeslány souřadnice kliknutí.

Výpis 2.9 Ukázka použití obrázkového tlačítka.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="image" name="obrazek"
       src="tlacitko.gif"><BR>
    </FORM>
  </BODY>
</HTML>
```

obrázek

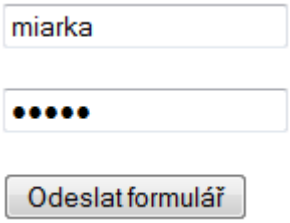
Tlačítko submit

Prvek, kterému parametr `type` nastavíme na hodnotu `submit`, představuje tlačítko, které slouží k odeslání dat z formuláře na server. Formulář se odešle metodou, která je nastavená v parametru `method` u značky `FORM`. Tlačítko je možné pojmenovat pomocí parametru `name`. Pomocí parametru `value` je možné nastavit popisek zobrazený na tlačítku. Pokud není tento parametr uveden, je zobrazen výchozí popis nastavený WWW prohlížečem.

Výpis 2.10 Ukázka použití tlačítka typu submit

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="text" name="uzivatel"
       value="miarka" size="20"
       maxlength="20"><BR><BR>
```

```
      <INPUT type="password" name="heslo"
       value="heslo" size="20"
       maxlength="20"><BR><BR>
      <INPUT type="submit" name="tlacitko"
       value="Odeslat formulář"><BR>
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.6 Zobrazení tlačítka typu submit na stránce.

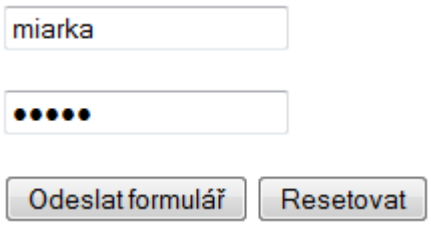
Resetovací tlačítko

Tento typ tlačítka vytvoříme nastavením parametru `type` na hodnotu `reset`. Toto tlačítko není určeno k odeslání obsahu formuláře na server, ale slouží k vynulování prvků ve formuláři. Hodnota tlačítka typu reset se po odeslání nepředává WWW serveru. Pokud mají formulářové prvky nastavenou hodnotu pomocí parametru `value`, po stisknutí tohoto tlačítka se daným prvků nastaví tato hodnota.

Výpis 2.11 Ukázka použití resetovacího tlačítka.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="text" name="uzivatel"
       value="miarka" size="20"
       maxlength="20"><BR><BR>
      <INPUT type="password" name="heslo"
       value="heslo" size="20"
       maxlength="20"><BR><BR>
      <INPUT type="submit" name="tlacitko"
       value="Odeslat formulář">
      <INPUT type="reset" name="tlacitko"
       value="Resetovat"><BR>
    </FORM>
  </BODY>
</HTML>
```





Obrázek 2.7 Zobrazení tlačítek typu submit a reset na stránce.

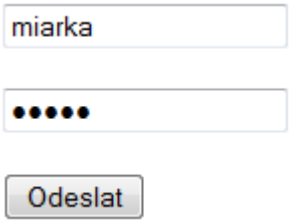
Tlačítko button

Posledním typem tlačítka, které lze vytvořit pomocí značky `INPUT` je tlačítko, jehož typ je `button`. Tento typ tlačítka je určen pro obecné akce a primárně slouží k použití v kombinaci s jazykem JavaScript. Nastavením parametrů `onclick`, `ondblclick`, `onfocus` nebo `onblur` je možné provést kód v jazyce JavaScript. Typickým příkladem použití je validace dat zadaných ve formuláři před odesláním na server. Stejně jako v případě tlačítka typu reset, ani u tohoto tlačítka, se při odeslání formuláře nepředávají hodnoty parametrů `name` a `value`.

Výpis 2.12 Ukázka použití tlačítka typu button.



```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <SCRIPT>
        function kontrolaFormulare() {
        ...
        }
      </SCRIPT>
      <INPUT type="text" name="uzivatel"
       value="miarka" size="20"
       maxlength="20"><BR><BR>
      <INPUT type="password" name="heslo"
       value="heslo" size="20"
       maxlength="20"><BR><BR>
      <INPUT type="button" name="button"
       value="Odeslat"
       onclick="kontrolaFormulare();"><BR>
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.8 Zobrazení tlačítka typu button na stránce.

2.3 Značka BUTTON



Značka `BUTTON` slouží k vytvoření obecného typu tlačítka, které umožňuje zadání náročnější formy popisku, případně vložení obrázku. Značka `BUTTON` je párová, mezi počáteční a koncovou značkou může být uveden textový popisek tlačítka, vložen obrázek (pomocí značky `<IMG>`), případně další komponenty.

Parametry

- `type` – parametr určuje typ tlačítka, může nabývat hodnot `submit`, `reset` a `button`. Jednotlivé typy tlačítka mají stejnou funkcionalitu jako v případě značky `INPUT`.
- `name` – jméno tlačítka, které je odesláno jako název parametru v rámci HTTP dotazu.
- `value` – hodnota, která ale nepředstavuje text zobrazený na tlačítku a není při odeslání formuláře hodnotou parametru v rámci HTTP dotazu.

Výpis 2.13 Ukázka použití značky BUTTON.



```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <INPUT type="text" name="uzivatel"
       value="miarka" size="20"
       maxlength="20"><BR><BR>
      <INPUT type="password" name="heslo"
       value="heslo" size="20"
       maxlength="20"><BR><BR>
      <BUTTON type="submit" name="tlacitko"
       value="Odeslat"><IMG alt="Obrázek"
       src="tlacitko.gif" width="20"
       height="20"><B>Odeslat data</B></BUTTON>
    </FORM>
```

```
  </BODY>
</HTML>
```

obrázek

2.4 Značka TEXTAREA



Značka `TEXTAREA` slouží k vytvoření víceřádkového textového pole. Používá se zejména v případě, kdy má uživatel zadat delší text. Tato značka nemá parametr `value`, pokud chceme v poli zobrazit po načtení stránky výchozí text, je nutné ho umístit mezi značky `<TEXTAREA>` a `</TEXTAREA>`.

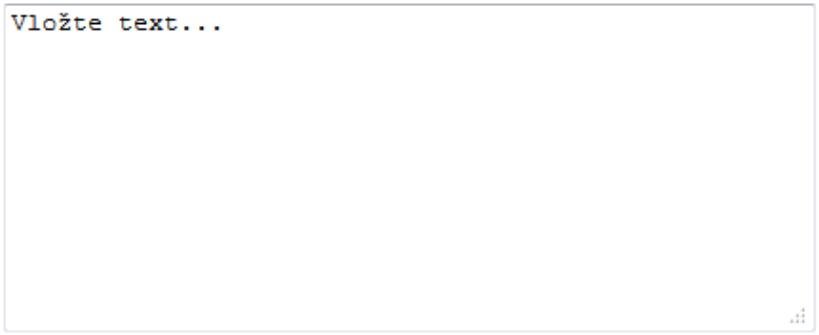
Parametry

- `name` – název formulářového prvku, po odeslání formuláře tvoří název parametru v rámci HTTP dotazu.
- `rows` – výška textového pole, hodnotou parametru je číslo udávající počet viditelných řádků v textovém poli. Pokud je do pole vloženo více řádků než udává tento parametr, je automaticky zobrazení vertikální posuvník.
- `cols` – šířka textového pole, hodnotou parametru je číslo udávající počet znaků, které se vejdou na jeden řádek. Pokud je do pole vložen text obsahující více znaků bez mezer, než udává tento parametr, je automaticky zobrazen horizontální posuvník.

Výpis 2.14 Ukázka použití značky TEXTAREA.



```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <TEXTAREA name="text" rows="10"
       cols="50">Vložte text...</TEXTAREA>
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.9 Zobrazení víceřádkového textového pole na stránce.

2.5 Značka SELECT



Značka `SELECT` slouží k vložení roletkového menu na WWW stránku. Vhodným nastavením parametrů lze tento formulářový prvek zobrazit jako posuvný seznam. Značka je párová.

Parametry

- `name` – název formulářového prvku. Po odeslání formuláře na server je součástí HTTP dotazu jako název parametru.
- `size` – velikost prvku, hodnotou je číslo udávající počet viditelných prvků seznamu. Výchozí hodnotou je číslo 1, tuto hodnotu není nutné uvádět. Prvek je zobrazen jako rozbalovací seznam, ze kterého uživatel vybírá hodnotu.
- `multiple` – zadáním tohoto parametru umožníme uživateli vybrat ze seznamu více hodnot.

Jednotlivé prvky seznamu v rámci značky `SELECT` specifikujeme pomocí značky `OPTION` (je možné ji použít jako párovou i jako nepárovou).

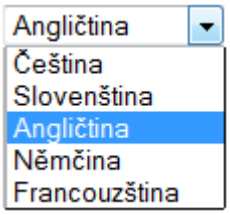
Parametry

- `value` – hodnota daného prvku v rámci seznamu. Je-li tato hodnota zvolena a formulář je odeslán na server, je tato hodnota součástí HTTP dotazu.
- `selected` – uvedením tohoto parametru u hodnoty docílíme toho, že po načtení stránky je tato hodnota zvolena.

- `disabled` – parametr má stejnou funkčnost jako v případě prvku `INPUT`, znemožní výběr dané možnosti.

Výpis 2.15 Ukázka použití prvku SELECT jako roletkového menu.

```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <SELECT name="jazyk">
        <OPTION value="cz">Čeština</OPTION>
        <OPTION value="sk">Slovenština</OPTION>
        <OPTION value="en"
         selected>Angličtina</OPTION>
        <OPTION value="de">Němčina</OPTION>
        <OPTION value="fr">Francouzština</OPTION>
      </SELECT>
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.10 Zobrazení rozbalovacího seznamu na stránce.

Při odeslání dat z formuláře na server bude součástí HTTP dotazu parametr ve tvaru

```
jazyk=en
```

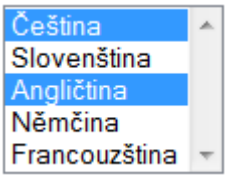
Následující ukázka demonstruje použití parametrů size a multiple u prvku `SELECT`.

Výpis 2.16 Ukázka použití prvku SELECT jako seznamu.



```
<HTML>
  <BODY>
    <FORM METHOD="POST" ACTION="index.php">
      <SELECT name="jazyk" size="5" multiple>
        <OPTION value="cz" selected>Čeština</OPTION>
        <OPTION value="sk">Slovenština</OPTION>
        <OPTION value="en"
         selected>Angličtina</OPTION>
```

```
        <OPTION value="de">Němčina</OPTION>
        <OPTION value="fr">Francouzština</OPTION>
      </SELECT>
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.11 Zobrazení seznamu na stránce.

Odeslání formuláře, který obsahuje tuto variantu prvku `SELECT` bude mít za následek přidání dvou hodnot parametru jazyk do HTTP dotazu:

```
jazyk=cz&jazyk=en
```

2.6 Formuláře v HTML5

V souvislosti s uvedením verze 5 jazyka HTML byla rozšířena a upravena i sada dostupných formulářových prvků. Uvedeme si pouze nejdůležitější změny. Podpora nových formulářových prvků se v různých webových prohlížečích může lišit, proto je prozatím jejich použití na zvážení.

Prvek INPUT je v HTML5 rozšířen o nové typy vstupních polí. Jedná se například o pole pro zadání barvy, data a času, e-mailu nebo URL adresy. Prvek TEXTAREA je rozšířen o parametr `maxlength`, jehož funkce je stejná jako v případě textového pole. Dalším novým parametrem tohoto prvku je `required`, pomocí něhož je vyžadováno vyplnění pole (pokud není pole vyplněno, formulář se neodešle).

Dalším novým prvkem je DATALIST, který slouží k vytvoření našeptávače pro prvek INPUT.

2.7 Příklad tvorby registračního formuláře

Následující výpis představuje HTML kód pro vytvoření registračního formuláře do webové aplikace, např. do e-shopu. Za výpisem pak následuje obrázek zobrazující formulář v prohlížeči.

Výpis 2.17 Příklad registračního formuláře.



```
<HTML>
  <HEAD>
    <TITLE>Registrační formulář</TITLE>
  </HEAD>
  <BODY>
    <H1>Registrační formulář</H1>
    <FORM METHOD="POST" ACTION="index.php">
    <TABLE border="0" cellspacing="0">
      <TR>
        <TD>E-mail: </TD>
        <TD><INPUT type="text" name="uzivatel"
         size="25" maxlength="25"></TD>
      </TR>
      <TR>
        <TD>Heslo: </TD>
        <TD><INPUT type="password" name="heslo1"
         size="25" maxlength="25"></TD>
      </TR>
      <TR>
        <TD>Potvrzení hesla: </TD>
        <TD><INPUT type="password" name="heslo2"
         size="25" maxlength="25"></TD>
      </TR>
      <TR>
        <TD>Jméno: </TD>
        <TD><INPUT type="text" name="jmeno" size="25"
         maxlength="25"></TD>
      </TR>
      <TR>
        <TD>Příjmení: </TD>
        <TD><INPUT type="text" name="prijmeni"
         size="25" maxlength="25"></TD>
      </TR>
      <TR>
        <TD>Ulice a č.p.: </TD>
        <TD><INPUT type="text" name="ulice" size="25"
         maxlength="25"></TD>
      </TR>
      <TR>
        <TD>Město: </TD>
        <TD><INPUT type="text" name="mesto" size="25"
         maxlength="25"></TD>
      </TR>
      <TR>
        <TD>PSČ: </TD>
```

```
  <TD><INPUT type="text" name="psc" size="6"
   maxlength="5"></TD>
</TR>
<TR>
  <TD>Kraj: </TD>
  <TD><SELECT name="kraj">
    <OPTION value="praha">Hlavní město
     Praha</OPTION>
    <OPTION
     value="stredocesky">Středočeský</OPTION>
    <OPTION
     value="liberecky">Liberecký</OPTION>
    <OPTION value="ustecky">Ústecký</OPTION>
    <OPTION
     value="karlovarsky">Karlovarský</OPTION>
    <OPTION value="plzensky">Plzeňský</OPTION>
    <OPTION
     value="jihocesky">Jihočeský</OPTION>
    <OPTION
     value="pardubicky">Pardubický</OPTION>
    <OPTION
     value="kralovehradecky">Královéhradecký</O
     PTION>
    <OPTION value="vysocina">Vysočina</OPTION>
    <OPTION
     value="jihomoravsky">Jihomoravský</OPTION>
    <OPTION
     value="olomoucky">Olomoucký</OPTION>
    <OPTION
     value="moravskoslezsky">Moravskoslezský</O
     PTION>
    <OPTION value="zlinsky">Zlínský</OPTION>
  </SELECT></TD>
</TR>
<TR>
  <TD>Pohlaví: </TD>
  <TD><INPUT type="radio" name="pohlavi"
   value="muz" checked>Muž <INPUT type="radio"
   name="pohlavi" value="zena">Žena</TD>
</TR>
<TR>
  <TD>Vzkaz: </TD>
  <TD><TEXTAREA name="vzkaz" rows="6"
   cols="40"></TEXTAREA></TD>
</TR>
<TR>
```

```
        <TD>Souhlasím s obchodními podmínkami: </TD>
        <TD><INPUT type="checkbox" name="obchpodm"
         value="ano"></TD>
      </TR>
      <TR>
        <TD colspan="2" align="center"><INPUT
         type="submit" name="submit" value="Odeslat">
         <INPUT type="reset" name="reset"
         value="Smazat formulář"></TD>
      </TR>
    </TABLE>
    </FORM>
  </BODY>
</HTML>
```



Obrázek 2.12 Zobrazení registračního formuláře na stránce.



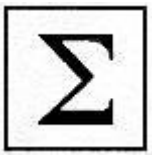
Kontrolní otázky:

1. K čemu slouží značka FORM?
2. Jaké formulářové prvky můžeme vytvořit pomocí značky INPUT?
3. K čemu slouží značka SELECT?

4. K čemu slouží značka TEXTAREA?

5. Jaký je rozdíl mezi tlačítkem typu SUBMIT a RESET?

Shrnutí obsahu kapitoly



V této kapitole jste se seznámili s formulářovými prvky, které je možné použít při tvorbě formulářů v jazyce HTML. Formuláře umožňují získat vstup od uživatele. U jednotlivých formulářových prvků jsou důležité jejich vlastnosti. Dostupné vlastnosti se liší podle druhu formulářového prvku. Tato kapitola je pro vytváření webových aplikací velmi důležitá. Nemusíte znát zpaměti všechny možné parametry jednotlivých formulářových prvků, ale je důležité mít o nich alespoň základní představu a vědět, kde můžete nalézt podrobnější informace.

3 Jazyk PHP, jeho historie a hlavní rysy

V této kapitole se dozvíte:

- Jaká je historie jazyka PHP a jaké jsou jeho hlavní rysy.

Po jejím prostudování byste měli být schopni:

- Znát základní rysy jazyka PHP a jeho stručnou historii

Klíčová slova této kapitoly:

PHP, Apache.

Doba potřebná ke studiu: 1 hodina



Průvodce studiem

Tato krátká kapitola seznamuje čtenáře se stručnou historií jazyka PHP a s jeho hlavními rysy. Jedná se pouze o doplňující přehledovou kapitolu, jejíž studium je jednoduché.

Zkratka PHP znamená „PHP Hypertext Preprocessor“, původně zkratka označovala slovní spojení „Personal Home Page“. Jazyk PHP je skriptovací jazyk fungující na straně serveru. Tento jazyk umožňuje tvorbu dynamických webových stránek nebo aplikací. Nejčastěji se používá v kombinaci s jazykem HTML nebo XHTML, případně XML.

Historie jazyka PHP začíná v roce 1994, kdy dánský programátor Rasmus Lerdorf vytvořil kombinaci skriptů v Perlu, které byly později vydány jako balíček označený nástroje „Personal Home Page“. Protože o tento balíček byl zájem, napsal v roce 1995 skriptovací jádro a dále další nástroj, který umožňoval analyzovat vstupy z formulářů v HTML. Tento celek byl označen jako PHP2. Postupem času přešel vývoj z jednoho člověka na celý tým programátorů. Kromě Rasmuse Lerdorfa se na vývoji podíleli Andi Gutmans, Zeev Suraski, Stig Bakken, Shane Caraveo a Jim Winstead). Rozšířením programátorského týmu vznikl jazyk označovaný jako PHP3. V této verzi bylo zlepšeno a rozšířeno skriptovací jádro, bylo přidáno jednoduché API, které umožnilo jiným programátorům vytvářet moduly. Byla zdokonalena syntaxe jazyka, byly přidány konstrukce známé z objektově orientovaných nebo procedurálních jazyků. Další verze, PHP4, je založena na jádru Zend, což je skriptovací jádro, které bylo navrženo tak, aby bylo jednoduše implementovatelné do různých aplikací. V současné době je již k dispozici 5. verze jazyka PHP, v březnu 2013 byla vydána verze 5.4.13.

Jazyk PHP je vyvíjen jako Open source a je nezávislý na použité platformě. V jazyce PHP jsou dostupné všechny konstrukce obvyklé v jiných jazycích.

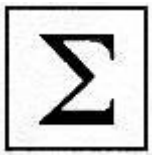
Kromě možnosti používat proměnné nebo konstanty se jedná o podmínky, cykly, tvorbu vlastních funkcí atd. Syntaxe jazyka byla inspirována jazyky C, Perl, Pascal a Java. Jazyk PHP obsahuje mnoho knihoven pro různé účely použití. Jedná se o knihovny pro zpracování grafiky, práci se soubory, přístup k databázovým systémům, podpora internetových protokolů atd. V současnosti je jazyk PHP nejrozšířenějším skriptovacím jazykem používaným na webu. Rozšíření tohoto jazyka bylo způsobeno hlavně jednoduchostí použití a velkému množství funkcí. Pro tvorbu webových aplikací bývá nejčastěji používán v kombinaci s operačním systémem Linux, webovým serverem Apache a databázovým serverem MySQL nebo PostgreSQL. Zkratka LAMP (Linux, Apache, MySQL a PHP) pak označuje použití uvedené kombinace technologií.

Používat jazyk PHP pro tvorbu vlastních webových stránek nebo aplikací je dnes velmi snadné. Prakticky každý webhosting nabízí podporu jazyka PHP spolu se serverem Apache. Podpora databázového systému MySQL je také velmi rozšířená.

Jazyk PHP je dynamicky typovaný, při použití proměnné je datový typ vázán na hodnotu a ne na proměnnou. Jinými slovy, jazyk PHP nepoužívá pro deklaraci proměnných datové typy. Datový typ proměnné je automaticky určen na základě hodnoty, kterou do ní vložíme. Pole v PHP jsou asociativní, tzn. že prvky pole jsou tvořeny dvojicí klíč – hodnota. Klíčem může být celé číslo nebo řetězec.

V počátcích použití jazyka PHP bylo poměrně náročné zprovoznit vše potřebné pro tvorbu dynamických webových stránek na lokálním počítači. Uživatel musel nainstalovat a nakonfigurovat server Apache, přidat do něj jako modul PHP, nainstalovat a nakonfigurovat databázový server. V dnešní době jsou již k dispozici balíčky, které umožňují provést tyto kroky najednou a jejich použití je snadné.

Jazyk PHP je použit pro tvorbu mnoha významných a velkých projektů. Za všechny uveďme software MediaWiki (je použitý například v encyklopedii Wikipedie), phpBB (webové diskusní fórum), WordPress (publikační systém pro tvorbu blogů), phpMyAdmin (webová aplikace pro správu databázového systému MySQL) a mnoho dalších.



Shrnutí obsahu kapitoly

V této krátké kapitole jste seznámili s historií jazyka PHP a s jeho hlavními rysy. Studium kapitoly není náročné, jedná se pouze o doplňující kapitolu.

4 První aplikace v PHP, práce s proměnnými a konstantami, operátory

V této kapitole se dozvíte:

- Jakým způsobem pracovat s proměnnými a konstantami v PHP.
- Jaké datové typy existují v PHP.
- Jaké operátory je možné použít v PHP.

Po jejím prostudování byste měli být schopni:

- Umět vypsat text na obrazovku (do okna prohlížeče).
- Umět pracovat s proměnnými v PHP.
- Znát operátory dostupné v PHP.

Klíčová slova této kapitoly:

Proměnná, konstanta, datový typ, echo, operátory.

Doba potřebná ke studiu: 6 hodin

Průvodce studiem



Studium této kapitoly představuje úvod do tvorby aplikací v PHP. Pro výpis textu na obrazovku (do okna prohlížeče) slouží příkaz echo. Seznámíte se s různými způsoby použití tohoto příkazu. Dále se seznámíte s dostupnými datovými typy v PHP, naučíte se pracovat s proměnnými a konstantami. V neposlední řadě se seznámíte s operátory, které je možné použít v PHP.

Jazyk HTML používaný pro tvorbu webových stránek umožňuje vytvářet pouze statické webové stránky. Kód stránky v tomto jazyce je uložen v souboru s příponou .htm nebo .html a při požadavku klienta na daný soubor je mu odeslán obsah tohoto souboru. WWW prohlížeč na straně klienta se pak postará o to, aby se obsah souboru zobrazil správně, tj. aby se texty naformátovaly podle použitých kaskádových stylů, aby byl funkční kód v jazyce JavaScript.

Skripty v jazyce PHP jsou uloženy jako soubory s příponou .php a samotný kód PHP je čistý text ve formátu ASCII. Kód PHP tedy můžeme vytvářet v libovolném textovém editoru (notepad v MS Windows, PSPad atd.). Soubor s příponou PHP může obsahovat i HTML kód. HTML kód se odešle klientovi ve stejné podobě v jaké je uveden v PHP souboru. Pokud je potřeba vložit kód PHP, je nutné jej označit pomocí speciálních značek. K tomu se v zásadě používají dvě značky, obě jsou párové. První značka začíná `<?php` a končí `?>`, druhá začíná `<?` a končí taktéž `?>`. Použití druhé značky nemusí vždy spolehlivě fungovat, proto je lepší používat raději první značku. Také proto bude v dalším textu používána tato značka.

Výpis 4.1 Ukázka použití značky pro označení kódu PHP.

```
<?php
  …kód php…
?>
```

Struktura kódu PHP je velmi podobná struktuře kódu v Javě nebo v C/C++. Každý příkaz musí končit středníkem `;`. Výjimkou je případ, kdy soubor PHP obsahuje pouze jeden příkaz, v tom případě se jeho vykonání provede i bez uvedení středníku za tímto příkazem. Kromě klíčových slov, identifikátorů proměnných, konstant nebo funkcí můžeme v kódu používat i komentáře. Při zpracování PHP skriptu je komentář přeskočen. Používání komentářů bývá často programátory opomíjeno, ale při větších projektech je vhodné používat komentáře, protože údržba velkého projektu bývá náročná. Navíc programátoři se mohou měnit a nově příchozí programátor má ze začátku problém se v kódu svého předchůdce vůbec zorientovat. PHP nabízí použití dvou typů komentářů – jednořádkový nebo víceřádkový komentář. Syntaxe pro použití obou typů komentářů je stejná jako v jazyce Java.

Výpis 4.2 Ukázka použití komentáře v PHP.

```
<?php
  // jednořádkový komentář
  // další jednořádkový komentář

  /*
  toto je
  víceřádkový
  komentář
  */
?>
```

Jednořádkový komentář začíná za znaky `//` a končí na konci aktuálního řádku. Víceřádkový komentář je tvořen všemi řádky textu, které jsou uvedeny mezi `/*` a `*/`. Kromě doplnění dodatečných informací ke kódu PHP je možné využít komentáře i k ladění programů. Postupným „zakomentováním“ jednotlivých částí kódu můžeme odhalit, která část způsobuje chybu.

4.1 První PHP skript

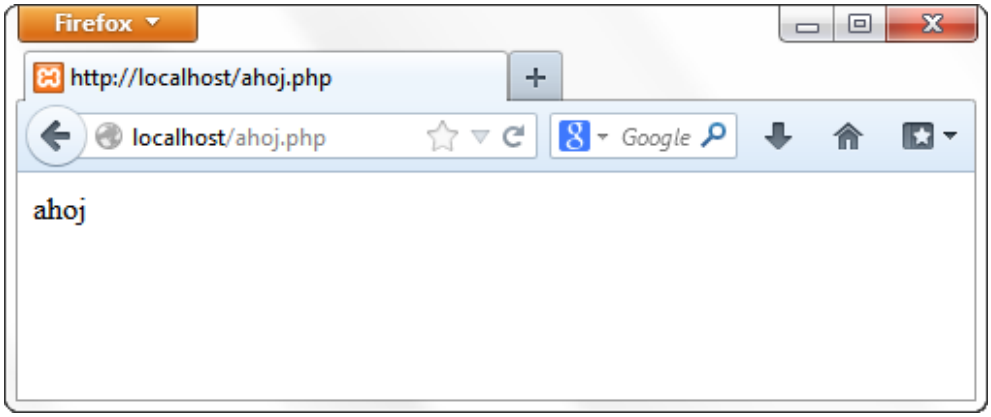
Ve většině programovacích jazyků bývá zvykem, že první vytvořený program vypíše na obrazovku pozdrav. Ani v tomto případě to nebude jinak, náš první PHP skript vypíše slovo „ahoj“ do okna prohlížeče. Následující kód je uložen v souboru `ahoj.php`, který je uložen v kořenovém adresáři serveru (v tomto

případě byl použit server Apache). Pro všechny následující příklady v této opoře bude použit webový server Apache nainstalovaný lokálně na počítači. Adresa tohoto serveru je `localhost`, všechny zobrazené PHP skripty tedy budou umístěny na adrese `http://localhost`.

Výpis 4.3 Obsah souboru ahoj.php

```
<?php
  echo "ahoj";
?>
```

Následující obrázek zobrazuje výsledek vykonání uvedeného PHP kódu.



Obrázek 4.1 Zobrazení souboru ahoj.php v prohlížeči.

Jak vidíme ve Výpisu 4.3, kromě značek pro označení začátku a konce PHP kódu, je zde uveden pouze jeden příkaz. Příkaz `echo` vypíše text do výstupu, který je odeslán ze serveru klientovi. V tomto případě je vypsán řetězec znaků uvedený v uvozovkách. Pro vytvoření řetězcové konstanty je v PHP možné použít jak klasické uvozovky (viz. Výpis 4.3) nebo apostrofy – jednoduché uvozovky. Příkaz



```
echo 'ahoj';
```

by tedy měl úplně stejný efekt. Kromě toho je možné u příkazu echo použít ještě závorku. Výpis by tedy mohl vypadat i následovně:

`echo "ahoj";` nebo `echo 'ahoj';`

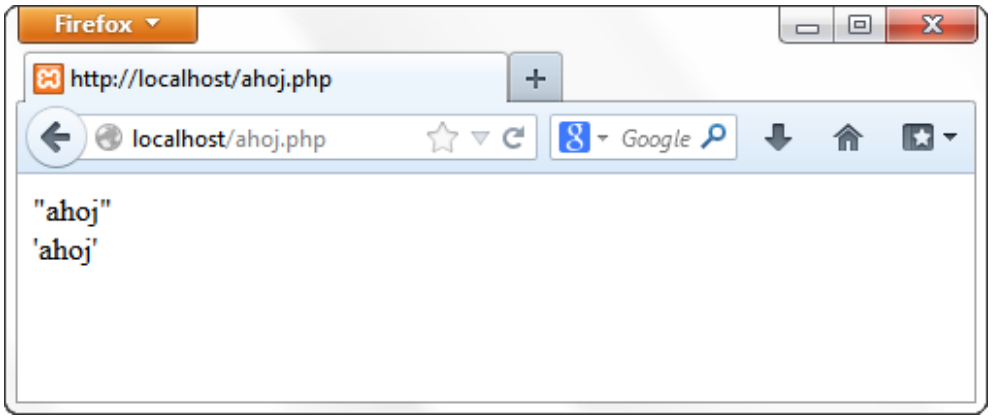
Pokud bychom v rámci vypisovaného řetězce chtěli vypsat uvozovky a pro ohraničení vypisovaného řetězce použijeme také uvozovky, je nutné použít pro vypsání uvozovek escape sekvenci – před uvozovky umístit obrácené lomítko `\`. Totéž platí v případě použití apostrofů, tj. jednoduchých uvozovek.

Následující výpis ukazuje použití escape sekvence a na obrázku je pak zobrazen výsledný výstup v prohlížeči. Protože standardně je generovaným výstupem HTML kód, pro odřádkování je nutné použít značku `<br>`.

Výpis 4.4 Použití escape sekvence.



```
<?php
  echo "\"ahoj\"<br>";
  echo '\'ahoj\'';
?>
```



Obrázek 4.2 Ukázka použití escape sekvence v prohlížeči.

Kromě uvedených escape sekvencí existují ještě další escape sekvence, které jsou známé i z jiných programovacích jazyků. Escape sekvence `\n` představuje přechod na nový řádek. Pokud tuto escape sekvenci použijeme v předchozím výpisu, tj. jeho tvar bude:

```
<?php
  echo "\"ahoj\"<br>\n";
  echo '\'ahoj\'';
?>
```

v okně prohlížeče neuvidíme žádný rozdíl. Rozdíl ale bude patrný, pokud si zobrazíme zdrojový kód stránky. Bez použití escape sekvence `\n` je ve zdrojovém kódu stránky text

```
"ahoj"<br>'ahoj'
```

Pokud escape sekvenci použijeme, zdrojový kód stránky bude

```
"ahoj"<br>

'ahoj'
```

Další escape sekvencí je tabulátor, který vytvoříme pomocí `\t`. Escape sekvence `\r` představuje návrat vozíku na začátek řádku. Pokud potřebujeme do výstupu vypsat obrácené lomítko, je nutné použít escape sekvenci `\\`. A pokud potřebujeme vypsat znak dolaru, je k tomuto účelu nutné použít escape sekvenci `\$`. Znak dolaru má totiž v PHP speciální význam.

K tomu, abychom do výstupu vypsali uvozovky nebo apostrofy, můžeme použít i jiný postup. Stačí zkombinovat použití uvozovek a apostrofů ve výstupním řetězci. Vnější znaky vždy ohraničují výstupní řetězec, vnitřní znaky lišící se od vnějších znaků se vypíší tak, jak jsou. Takže pomocí následujícího kódu vypíšeme text s uvozovkami nebo apostrofy i bez použití escape sekvence.

Výpis 4.5 Výpis uvozovek nebo apostrofů bez escape sekvence.

```
<?php
  echo '"ahoj"<br>';
  echo "'ahoj'";
?>
```



Výstup tohoto PHP kódu bude stejný jako na obrázku 4.2. Ostatní escape sekvence (přechod na nový řádek, tabulátor atd.) fungují ale pouze pokud jsou uvedeny v klasických uvozovkách. Pokud je uvedeme v apostrofech, zobrazí se ve výstupu v prohlížeči a nemají na zdrojový kód stránky žádný vliv.

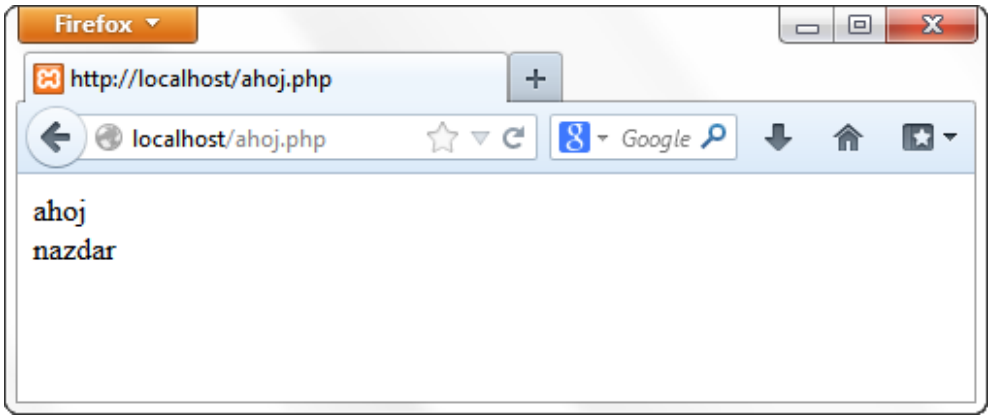
Možnost použití kombinace apostrofů a klasických uvozovek je výhodná zejména při generování HTML kódu do výstupu. HTML kód totiž zpravidla obsahuje i parametry jednotlivých značek (např. pro značku INPUT je to parametr type). Hodnoty parametrů se zpravidla uvádějí v uvozovkách. Pokud bychom pro označení začátku a konce výstupu použili klasické uvozovky, musíme před všechny uvozovky ve výstupním HTML kódu uvést znak obráceného lomítka. Pokud ovšem na začátku a na konci HTML kódu použijeme apostrof, nemusím se tímto zdržovat. HTML kód je navíc bez escape sekvencí u každých uvozovek o mnoho přehlednější.

Pokud je potřeba spojit více řetězců dohromady nebo spojit řetězce spolu s hodnotami proměnných, je možné použít operátor `.` (tečka), který spojení provede.

Výpis 4.6 Ukázka spojení řetězců.

```
<?php
  echo "ahoj" . "<br>" . "nazdar";
```

```
?>
```



Obrázek 4.3 Výsledek spojení řetězců zobrazený v prohlížeči.

4.2 Proměnné a datové typy



Jazyk PHP umožňuje používání proměnných, stejně jako je tomu v jiných programovacích jazycích. Proměnná má svůj identifikátor (název) a hodnotu. Identifikátor proměnné vždy začíná znakem `$` (dolar). Proto je nutné použít escape sekvenci, pokud chceme vypsat znak dolaru. Pro zbytek identifikátoru (za znakem dolaru) platí, může obsahovat písmena anglické abecedy, čísla a podtržítko. Začínat může písmenem nebo podtržítkem. Identifikátory proměnných jsou case senzitivní, tj. rozlišují se v nich malá a velká písmena. Zpravidla se identifikátory píší malými písmeny. Následují ukázky identifikátorů.

Výpis 4.7 Ukázka identifikátorů.

```
<?php
  $cislo;
  $_promenna;
  $hodnota123;
  $Hodnota123; /* nová proměnná, identifikátory
   $hodnota123 a $Hodnota123 se liší */
  $3abc; // chybný identifikátor - začíná číslem
?>
```

Pro hodnoty proměnných jsou v PHP různé datové typy, celkem existuje 6 hlavních datových typů – logická hodnota, celé číslo, desetinné číslo, řetězec znaků, pole a objekt.

4.2.1 Jednoduché datové typy

Logická hodnota

Logický typ se v PHP nazývá Boolean. Slouží k uchování hodnoty „pravda“ nebo „nepravda“. Hodnoty tohoto typu jsou samozřejmě v angličtině, tj. true a false. Hodnoty nejsou case sensitivní, takže můžeme napsat i TRue nebo True.

Celé číslo

Název celočíselného typu je Integer. Slouží k uchování celých čísel. Čísla mohou být kladná nebo záporná. Rozsah tohoto typu se liší v závislosti na použité platformě, ale obvykle bývá zhruba od -2 miliard do +2 miliard. Hodnota může být zadána v desítkové, osmičkové, šestnáctkové nebo binární soustavě.

Desetinné číslo

Datový typ pro uložení reálné hodnoty se v PHP nazývá Float nebo Real. Rozsah a přesnost typu je závislá na platformě, ale obvykle platí, že přesnost tohoto typu je 14 desetinných míst, rozsah je velmi velký, zhruba 1.8e308. Při zadávání hodnoty je nutné použít desetinnou tečku, je možné použít i matematický zadání pomocí exponentu.

Řetězec znaků

Tento datový typ se nazývá String a slouží k uchování posloupnosti znaků. Jeho délka je v PHP prakticky neomezená. K vytvoření hodnoty typu String je možné použít buď klasické uvozovky nebo apostrofy (jednoduché uvozovky).

Na rozdíl od jiných programovacích jazyků, v PHP neexistuje deklarace proměnné. Datový typ proměnné je určen v okamžiku uložení hodnoty do proměnné. Přiřazení hodnoty do proměnné je realizováno pomocí znaku = (rovná se). Pokud tedy do proměnné přiřadíme hodnotu `10`, automaticky se její typ nastaví na celé číslo. Pro výpis hodnoty proměnné se používá příkaz `echo`, např. `echo $promenna;`. Název proměnné může být uveden i v rámci vypisovaného řetězce. Následují ukázky vytvoření proměnných jednotlivých datových typů.

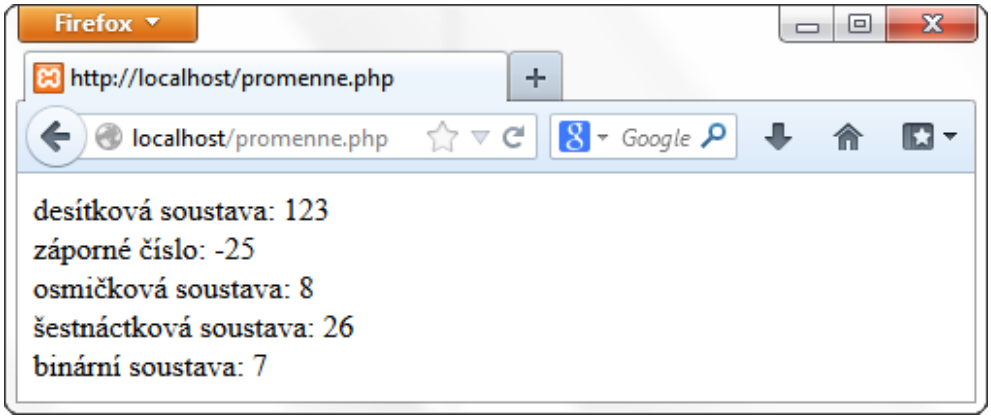
Výpis 4.8 Ukázka vytvoření logické proměnné.

```
<?php
  // logická proměnná
  $logicka1 = true;
  $logicka2 = false;
?>
```

Výpis 4.9 Ukázka vytvoření celočíselné proměnné.

```
<?php
  // celé číslo
  // desítková soustava
  $cislo = 123;
  echo "desítková soustava: " . $cislo . "<br>";
  // záporné číslo
  $cislo = -25;
  echo "záporné číslo: $cislo<br>";
  // osmičková soustava
  $cislo = 010;
  echo "osmičková soustava: " . $cislo . "<br>";
  // šestnáctková soustava
  $cislo = 0x1A;
  echo "šestnáctková soustava: " . $cislo . "<br>";
  // binární soustava
  $cislo = 0b111;
  echo "binární soustava: " . $cislo . "<br>";
?>
```

Výsledkem v prohlížeči bude výstup zobrazený na následujícím obrázku.



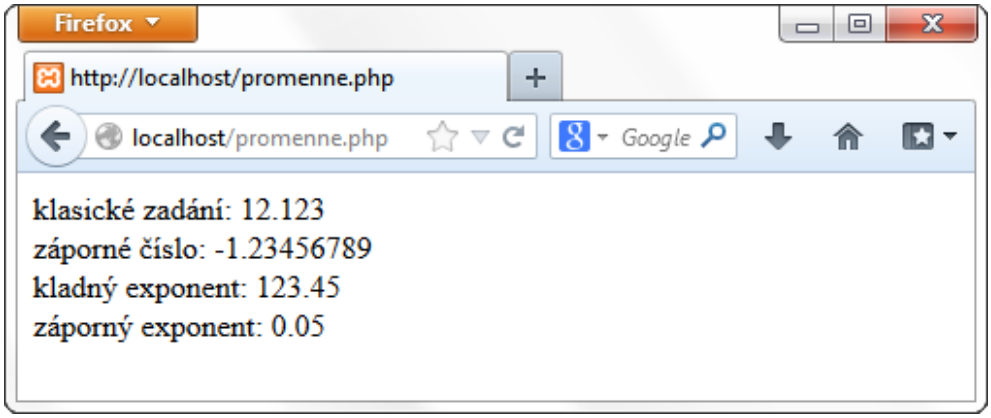
Obrázek 4.4 Výsledek zobrazený v prohlížeči.

Výpis 4.10 Ukázka použití reálných proměnných.

```
<?php
  // realné číslo
  // klasické zadání
  $cislo = 12.123;
  echo "klasické zadání: " . $cislo . "<br>";
  // záporné číslo
  $cislo = -1.23456789;
  echo "záporné číslo: " . $cislo . "<br>";
  // zadání s kladným exponentem
  $cislo = 1.2345e2;
```

```
  echo "kladný exponent: " . $cislo . "<br>";
  // zadání se záporným exponentem
  $cislo = 5e-2;
  echo "záporný exponent: " . $cislo . "<br>";
?>
```

Výsledek použití reálné proměnné zobrazený v prohlížeči je zobrazen na následujícím obrázku.



Obrázek 4.5 Výsledek zobrazený v prohlížeči.

Uvedené datové typy patří mezi jednoduché typy. Složitějším datovým typem je pole.

4.2.2 Složitější datové typy

Pole

Datový typ pole představuje strukturu, v PHP je pole v podstatě mapou, ve které jsou jednotlivé prvky. Každý prvek má svůj klíč a svou hodnotu. Klíč můžeme nazývat také jako index. Klíčem (indexem) prvku může být celé číslo nebo řetězec. Pokud je klíčem řetězec hovoříme o tzv. asociativním poli. Pole v PHP nemusí být homogenní, tj. každý prvek pole může mít jiný datový typ. Vytvoření pole je v PHP možné několika způsoby. První způsob je vyjmenování prvků, které do pole patří.

Výpis 4.11 Vytvoření pole vyjmenováním prvků.

```
<?php
  // klíč není uveden
  $cisla[] = 10;
  $cisla[] = 20;
  $cisla[] = 30;
  $cisla[] = 40;
```



```
  // klíčem v poli je číslo, ale čísla netvoří řadu
  $teploty[0] = 10.7;
  $teploty[4] = 10.3;
  $teploty[8] = 12.8;
  $teploty[12] = 16.8;

  // klíčem v poli je řetězec
  $jmena["prvni"] = "Petr";
  $jmena["druhe"] = "Jana";
  $jmena["treti"] = "Martin";
  $jmena["ctvrte"] = "Pavla";
?>
```

Při vyjmenování prvků pole `$cisla` nejsou uvedeny indexy (klíče). Prvkům jsou tedy indexy přiřazeny automaticky, jejich číslování začíná od nuly. První prvek pole bude mít index `0`, druhý index `1` atd. PHP umožňuje, aby číselné indexy netvořily souvislou řadu čísel. Ukázka vytvoření takového pole je pole `$teploty`. Indexy prvků by mohly představovat například hodinu, kdy byla daná teplota naměřena. Poslední pole, `$jmena`, obsahuje jako klíče řetězce, v tomto případě už hovoříme o asociativním poli.

Druhou možností, jak vytvořit pole je použití funkce `array()`. Této funkci předáme jako parametry hodnoty prvků v poli. Indexy prvků budou tvořeny čísly, první prvek bude mít index `0`. Pokud chceme změnit indexy prvků, je potřeba použít operátor `=>`. Následující výpis zobrazuje ukázku vytvoření stejných polí jako v předchozím výpisu.

Výpis 4.12 Ukázka vytvoření pole pomocí funkce array().



```
<?php
  // klíč není uveden
  $cisla = array(10, 20, 30, 40);

  // klíčem v poli je číslo, ale čísla netvoří řadu
  $teploty = array(0 => 10.7, 4 => 10.3, 8 => 12.8,
   12 => 16.8);

  // klíčem v poli je řetězec
  $jmena = array("prvni" => "Petr", "druhe" =>
   "Jana", "treti" => "Martin", "ctvrte" => "Pavla");
?>
```

Přístup k prvkům pole je stejný jako v jiných programovacích jazycích, uvedeme název pole a v hranatých závorkách index požadovaného prvku.

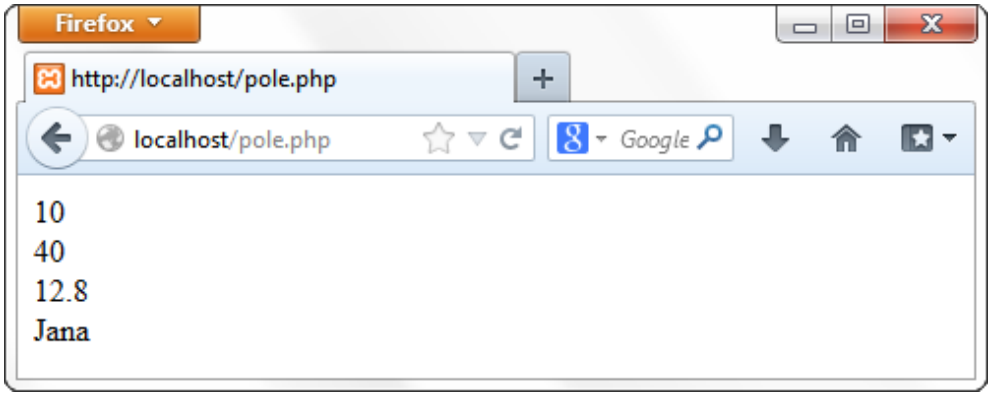
Následující výpis předpokládá, že jsou existují pole vytvořená v předchozím výpisu.

Výpis 4.13 Ukázka výpisu prvků pole.

```
<?php
  echo $cisla[0] . "<br>";
  echo $cisla[3] . "<br>";
  echo $teploty[8] . "<br>";
  echo $jmena["druhe"] . "<br>";
?>
```



Výstup zobrazený v prohlížeči je na následujícím obrázku.



Obrázek 4.6 Zobrazení vypsaných prvků pole v prohlížeči.

Ke zjištění počtu prvků v poli slouží funkce `count()`, která má jako parametr název pole, u kterého chceme počet prvků zjistit. K procházení polí je možné použít cykly, stejně jako v případě jiných programovacích jazyků. Procházení pole, jehož prvky jsou indexovány spojitě, je velmi jednoduché a příliš se neliší procházení pole v Javě. V případě nespojitě indexovaného pole je postup trošku odlišný. Oba postupy si uvedeme v části věnované cyklům.

Jazyk PHP umožňuje používat i vícerozměrná pole. Kromě klasického jednorozměrného pole bývá také občas používáno dvojrozměrné pole, které vznikne tak, že prvky pole nejsou jednoduchého typu, ale jsou to také pole. Tímto způsobem je možné vytvářet i vícerozměrná pole, ale jejich praktické využití není příliš velké. Dvojrozměrné pole bývá používáno například pro reprezentaci matic. Pro procházení dvojrozměrným polem je potřeba použít dva vnořené cykly, kdy vnější cyklus prochází prvky vnějšího pole a v každém kroku se pak pomocí vnitřního cyklu projdou prvky vnitřního pole.

Objekt

Datový typ objekt souvisí s objektově orientovaným programováním, které bude podrobněji probráno v následujících kapitolách.

4.2.3 Přetypování proměnných

Datový typ proměnných v PHP lze změnit, a to pomocí automatického nebo explicitního přetypování. Jak již bylo uvedeno, datový typ proměnné se určí přiřazením hodnoty do proměnné. Automatické přetypování se provede v případě, kdy proměnné přiřadíme hodnotu jiného typu.

Výpis 4.14 Automatické přetypování proměnné.



```
<?php
  $a = 10;
  $a = 2.5;
  $a = "ahoj";
?>
```

Prvním příkazem do proměnné `$a` přiřadíme hodnotu `10` a typ proměnné se nastaví na celé číslo (integer). Hodnota proměnné `$a` je `10`. Druhým příkazem změníme hodnotu proměnné `$a` na `2.5`, ale současně se automaticky změní i datový typ této proměnné – novým datovým typem je desetinné číslo (double). Poslední příkaz opět změní hodnotu proměnné `$a` na řetězec „`ahoj`“ a současně se změní i datový typ na řetězec (string).

Automatická konverze funguje i v případě převodu řetězce na číslo. Pokud řetězec začíná číslicemi, za nimiž následují jiné znaky, převedou se číslice ze začátku řetězce na číslo a zbytek řetězce je ignorován. Výsledkem převodu je datový typ integer. Například řetězec „`10 lidí`“ bude převeden na číslo `10`. Pokud je celý řetězec tvořen znaky odpovídajícími desetinnému číslu, je tento řetězec převeden na typ double. Řetězec „`3.5`“ bude převeden na číslo `3.5`, řetězec „`1.5231e2`“ na číslo `152.31`. Pokud řetězec obsahuje jiné znaky, které nemohou být převedeny na desetinné číslo, je převedena pouze část před desetinnou čárkou a výsledkem je celé číslo. Řetězec „`12.5 stupně Celsia`“ bude převeden na číslo `12`. Pokud řetězec začíná znakem − (mínus), je výsledkem převodu záporné číslo (platí v případě celých i desetinných čísel.

Další automatickou konverzí je přetypování mezi dvěma numerickými typy. Pokud je provedena numerická operace mezi celým a desetinným číslem, výsledkem je desetinné číslo.

Explicitní přetypování provedeme uvedením nového datového typu před názvem proměnné nebo před hodnotou, kterou chceme přetypovat. Pro převod na celočíselný typ se používá klíčové slovo `integer` nebo `int`. Převodem desetinného čísla na celé číslo se převede pouze celá část původní hodnoty, desetinná část je ignorována. Převodem desetinného čísla `10.9` na celé číslo vznikne celé číslo `10`. Pro převod na desetinné číslo se používají klíčová slova `double`, `float` nebo `real`. Převodem celého čísla na desetinné se k původní hodnotě přidá desetinná část. Výsledkem převodu celého čísla `5` na desetinné číslo bude desetinné číslo `5.0`.

4.2.4 Funkce pro práci s proměnnými



Jazyk PHP obsahuje několik vestavěných funkcí pro práci s proměnnými. První funkcí je `gettype()`, která vrací datový typ proměnné. Parametrem funkce je název proměnné, u které chceme datový typ zjistit. Vrácená hodnota představuje název datového typu. Hodnotami, které může funkce vrátit, jsou „`integer`" (celé číslo), „`double`" (desetinné číslo), „`string`" (řetězec), „`array`" (pole), „`object`" (objekt), „`class`" (třída) a „`unknown type`" (neznámý typ).

Druhou funkcí je funkce `settype()`, která převede proměnnou na požadovaný datový typ. Funkce má dva parametry. Prvním parametrem jen název proměnné a druhým datový typ, který chceme proměnné nastavit. Převod lze provést na datové typy `array`, `double`, `integer`, `object` a `string`. Pokud nelze požadovaný typ nastavit, vrátí funkce logickou konstantu `false`.

Funkce `isset()` se používá pro zjištění, zda byla proměnné přiřazena hodnota. Parametrem funkce je název proměnné. Funkce vrací logickou konstantu `true` (hodnota byla proměnné přiřazena) nebo `false` (hodnota nebyla proměnné přiřazena). Funkce `unset()` se používá k odstranění dané proměnné a uvolnění paměti, kterou proměnná zabírá. Parametrem funkce je opět název proměnné.

Funkce `empty()` je opakem funkce `isset()`. Pokud proměnné nebyla přiřazena hodnota, má nulovou hodnotu nebo hodnotu prázdného řetězce, funkce vrací `true`. V opačném případě vrací `false`.

Pro otestování, zda je proměnná konkrétního typu slouží funkce `is_typ()`. Místo slova `typ` je v názvu funkce uveden konkrétní datový typ. Pro

otestování, zda proměnná představuje celé číslo slouží funkce `is_int()`, `is_integer()` nebo `is_long()`. Pro otestování, zda se jedná o desetinné číslo jsou to funkce `is_double()`, `is_float()` a `is_real()`. Pro zbylé datové typy jsou to funkce `is_string()` (řetězce), `is_array()` (pole), `is_object()` (objekt). Všechny funkce mají jako parametr název proměnné a vracejí `true`, pokud je proměnná daného typu nebo `false`, pokud je jiného typu.

Výpis 4.15 Ukázka použití funkcí pro práci s proměnnými.



```
<?php
  $a = 20;
  $b = -2.5;
  $c = array("a", "b", "c");
  $d = "auto";
  echo gettype($a) . "<br>"; // vypíše integer
  settype($a, "double");
  echo gettype($a) . "<br>"; // vypíše double
  echo gettype($b) . "<br>"; // vypíše double
  echo gettype($c) . "<br>"; // vypíše array
  echo gettype($d) . "<br>"; // vypíše string
  echo empty($e) . "<br>"; // vypíše 1
  echo is_double($b) . "<br>"; // vypíše 1
  unset($b);
  echo isset($b) . "<br>"; // vrací false, nevypíše
   nic
?>
```

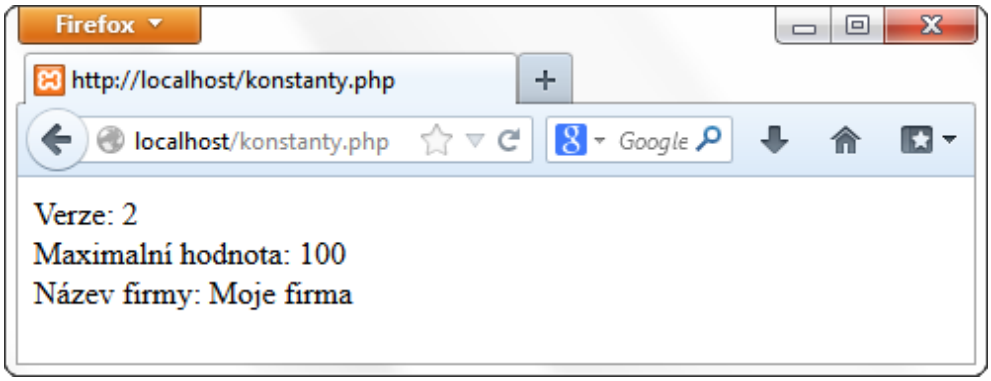
4.3 Konstanty



Na rozdíl od proměnných, jejichž hodnota se může během vykonávání skriptu měnit, konstanty mají po prvním přiřazení stále stejnou hodnotu. Názvy (identifikátory) konstant nezačínají znakem dolaru `$`. Konvencí pro názvy konstant je, že se píši pouze velkými písmeny, případně se používá znak podtržítka `_`. Pro vytvoření konstanty se v PHP používá funkce `define()`, která má dva parametry. Prvním parametrem je název konstanty, který musí být uveden v uvozovkách. Druhým parametrem je hodnota. Číselné hodnoty nemusí být uvedeny v uvozovkách. Pro otestování, zda nějaká konstanta existuje, slouží funkce `defined()`, která má jako parametr název konstanty. Pokud konstanta existuje, funkce vrátí `1`, pokud neexistuje, vrátí `0`. Pokud chceme v kódu pracovat s hodnotou konstanty, stačí uvést její název.

Výpis 4.16 Ukázka definování a vypsání konstant.

```
<?php
  define("VERZE", 2);
  define("MAX", 100);
  define("FIRMA", "Moje firma");
  echo "Verze: " . VERZE . "<br>";
  echo "Maximální hodnota: " . MAX . "<br>";
  echo "Název firmy: " . FIRMA . "<br>";
?>
```



Obrázek 4.7 Výpis konstant v prohlížeči.

4.4 Operátory a výrazy

V jazyce PHP je možné použít unární nebo binární operátory a nebo ternární operátor.

4.4.1 Unární operátory

Prvním unárním operátorem je znaménko mínus `-`, pomocí které je možné vytvořit opačné číslo k nějakému číslu (tj. číslo, které má opačné znaménko než původní hodnota. Dalšími dvěma unárními operátory jsou operátory inkrementace `++` a dekrementace `--`, které slouží ke zvýšení (inkrementace) nebo snížení (dekrementace) hodnoty proměnné o `1`. Oba operátory mohou být umístěny jak před, tak za proměnnou, ke které se vztahují. Na hodnotu této proměnné nemá umístění operátoru vliv. Hodnota této proměnné bude v obou případech zvýšena nebo snížena o `1`. Rozdíl nastane, pokud je inkrementovaná nebo dekrementovaná proměnná součástí jiného výrazu. Pokud je operátor umístěn před proměnnou, je nejprve provedena inkrementace nebo dekrementace a poté se vyhodnotí zbytek výrazu. Pokud je operátor umístěn za proměnnou, je nejprve vyhodnocen výraz s původní hodnotou proměnné a poté je provedena inkrementace nebo dekrementace.

Výpis 4.17 Ukázka použití unárních operátorů.



```
<?php
  $a = 10;
  $b = -$a; // $b = -10

  $c = -5;
  $d = -$c; // $d = 5

  $e = 10;
  $f = ++$e; // $f = 11, $e = 11
  $e = 10;
  $g = $e++; // $g = 10, $e = 11
  $e = 10;
  $h = 2 * ++$e; // $h = 22, $e = 11
  $e = 10;
  $i = 2 * $e++; // $i = 20, $e = 11
?>
```

4.4.2 Binární operátory

Jako operátor přiřazení je v PHP používán znak = (rovná se). Pomocí tohoto operátoru můžeme přiřadit hodnoty proměnným. S tímto operátorem jsme se již setkali v předchozích kapitolách.

Binární operátory jsou operátory, které jsou spojené se dvěma operandy. V PHP existují tři skupiny binárních operátorů. Jedná se o aritmetické operátory, operátory porovnání a logické operátory.

Tabulka 4.1 Aritmetické operátory.

Operátor	Operace	Popis
+	sčítání	Součet dvou číselných hodnot
-	odečítání	Rozdíl dvou číselných hodnot
*	násobení	Součin dvou číselných hodnot
/	dělení	Podíl dvou číselných hodnot
%	zbytek po dělení (modulo)	Zbytek po celočíselném dělení

Aritmetické operátory jsou k dispozici prakticky v každém programovacím jazyce a mají vždy stejný význam jako v matematice. Ani v PHP tomu není jinak. Priorita operátorů je také stejná jako v matematice. Násobení, dělení a zbytek po dělení mají vyšší prioritu než sčítání a odečítání. K úpravě priority operátorů je možné použít kulaté závorky.

Výpis 4.18 Ukázka použití aritmetických operátorů.

```
<?php
  $a = 10;
  $b = 5;
  $c = 2;
  $d = $a + 2; // $d = 12
  $e = $b % 2; // $e = 1
  $f = $a + $b * $c; // $f = 20
  $g = ($a + $b) * $c; // $g = 30
?>
```



Operátory porovnání slouží ke srovnání dvou hodnot (nejčastěji dvou proměnných nebo proměnné a nějaké konstanty).

Tabulka 4.2 Porovnávací operátory.

Operátor	Operace	Výsledek
==	rovnost	`true`, pokud jsou porovnávané hodnoty stejné
!=	nerovnost	`true`, pokud jsou porovnávané hodnoty různé
<>	nerovnost	`true`, pokud jsou porovnávané hodnoty různé
>	větší	`true`, pokud je první hodnota větší než druhá
<	menší	`true`, pokud je první hodnota menší než druhá
>=	větší než	`true`, pokud je první hodnota větší nebo rovna než druhá
<=	menší než	`true`, pokud je první hodnota menší nebo rovna než druhá
===	shodnost	`true`, pokud jsou porovnávané hodnoty stejného typu a jsou si rovny
!==	neshodnost	`true`, pokud jsou porovnávané hodnoty různé a jsou různého typu

Při použití porovnávacích operátorů mohou být porovnávány i výrazy (v následujícím výpisu je to přiřazení do proměnné `$f`.

Výpis 4.19 Ukázka použití porovnávacích operátorů.

```
<?php
  $a = 10;
  $b = 5;
  $c = 2;
  $d = $a > 9; // $d = 1 (true)
  $e = $b == $c; // $e = 0 (false)
  $f = ($a - $b) > ($c + 2); // $f = 1 (true)
```



```
  $g = $b >= $c; // $g = 1 (true)
?>
```

Další skupinou operátorů jsou logické operátory, které slouží ke kombinaci podmínek. Jedna podmínka je vytvořena použitím jednoho porovnávacího operátoru. Pokud potřebujeme testovat více věcí najednou, spojíme jednotlivé podmínky pomocí logických operátorů. V PHP se používají tři logické operátory. Prvním logickým operátorem je negace. Tento operátor je unární a výsledkem je opačná logická hodnota než byla vstupní hodnota. Dalším logickým operátorem je spojka „a“ (případně „a zároveň“), která představuje logický součin. Jedná se o binární operátor, který vrací `true`, pokud jsou všechny dílčí podmínky `true`. Pokud je více podmínek spojenou pouze spojkou „a“, stačí aby jedna z nich byla `false` a výsledkem celého výrazu je `false`. Druhým binárním logickým operátorem je spojka „nebo“, která vrací `true`, pokud alespoň jedna z dílčích podmínek byla `true`. Tento operátor představuje logický součet.

Tabulka 4.3 Logické operátory.

Operátor	Název	Popis
`!`	negace	z `true` udělá `false` a naopak
`&&`	a	`true`, pokud obě porovnávané části jsou `true`
`||`	nebo	`true`, pokud alespoň jedna porovnávaná část je `true`
`and`	a	`true`, pokud obě porovnávané části jsou `true`
`or`	nebo	`true`, pokud alespoň jedna porovnávaná část je `true`

Jak je vidět v tabulce, v PHP existují dva logické operátory pro logický součin a dva operátory pro logický součet. Pokud použijeme operátor `&&` nebo operátor `and`, výsledek bude stejný. Totéž platí pro operátory `||` a `or`. Rozdíl je ale v prioritě operátorů. Nejvyšší prioritu z logických operátorů má negace `!`. Následují operátory `&&`, `||`, `and` a `or`. Operátor `||` má tedy vyšší prioritu než `and`, i když operátor `and` představuje logický součin a operátor `||` logický součet.

Výpis 4.20 Ukázka použití logických operátorů.



```
<?php
  $a = 10;
  $b = 5;
  $c = 2;
  $d = ($a > 0) && ($b > 0) && ($c > 0); // $d = 1
   (true)
```

```
  $e = ($a > $b) && ($c > $b); // $e = 0 (false)
  $f = !$d; // $f = 0 (false)
  $g = ($b < 10) || ($c > 10); // $g = 1 (true)
  $h = ($b < $a) and ($b > 8) || ($c > 0); // $h = 1
   (true)
?>
```

Dalším operátorem v PHP je operátor pro spojování řetězců, který je reprezentován znakem `.` (tečka). S tímto operátorem jsme se již v předchozích kapitolách setkali.

Výpis 4.21 Ukázka použití operátoru pro spojení řetězců.

```
<?php
  $jmeno = "Rostislav";
  $prijmeni = "Miarka";
  $cele_jmeno = $jmeno . " " . $prijmeni;
  echo $cele_jmeno; // vypíše "Rostislav Miarka"
?>
```



4.4.3 Ternární operátor

Posledním operátorem, který zde uvedeme, je ternární operátor. Unární operátory pracují s jedním operandem, binární pracují se dvěma operandy a ternární operátor pracuje se třemi operandy. Ternární operátor je v PHP pouze jeden a označuje se pomocí znaků `?` `:` (otazník, dvojtečka). Pomocí tohoto operátoru vytvoříme konstrukci, která se skládá ze tří částí. První částí je logická podmínka, za kterou následuje znak `?` (otazník). Za otazníkem následuje hodnota, která je vrácena, pokud je podmínka vyhodnocena jako `true`. Na tomto místě může být uveden i příkaz. Další v pořadí je uvedena `:` (dvojtečka) a za ní následuje hodnota, která je vrácena, pokud byla podmínka vyhodnocena jako `false`. Ve své podstatě nahrazuje ternární operátor příkaz `if … else`, se kterým se seznámíme v následující kapitole.

Výpis 4.22 Ukázka použití ternárního operátoru.

```
<?php
  $a = 10;
  $a > 0 ? $b = "ano" : $b = "ne";
  $c = $a < 0 ? "ano" : "ne";
  echo $b; // vypíše "ano"
  echo $c; // vypíše "ne"
?>
```



4.4.4 Výrazy

Pojem výraz představuje použití několika různých operátorů stejného typu najednou. Použitím několika aritmetických operátorů současně vytvoříme aritmetický výraz. Pomocí logických operátorů vytváříme logické výrazy. Jejich součástí mohou být i aritmetické výrazy, ale naopak to neplatí.

Výpis 4.23 Ukázka výrazů.



```
<?php
  // aritmeticke vyrazy
  $a = 10;
  $b = 5;
  $c = -2;
  $a1 = 3 + $a * 2; // $a1 = 23
  $a2 = $a / $b - 20 + $c; // $a2 = -20
  $a3 = ($c * $c) + ($a * $b); // $a3 = 54

  // logicke vyrazy
  $b1 = ($a > $b) && ($a > $c); // $b1 = 1 (true)
  $b2 = $a + 5 > $b * 4 || $a / 2 < $b * 2; // $b2 =
   1 (true)
?>
```



4.4.5 Zkrácené přiřazení

Jazyk PHP umožňuje pro vybrané operátory používat zkrácené přiřazení, které je možné použít například v jazyce Java. V následující tabulce jsou uvedeny možností zkráceného přiřazení pro různé operátory.

Tabulka 4.4 Zkrácené přiřazení.

Zkrácené přiřazení	Nezkrácený zápis
`$a += $b;`	`$a = $a + $b;`
`$a -= $b;`	`$a = $a - $b;`
`$a *= $b;`	`$a = $a * $b;`
`$a /= $b;`	`$a = $a / $b;`
`$a %= $b;`	`$a = $a % $b;`
`$a .= $b;`	`$a = $a . $b;`

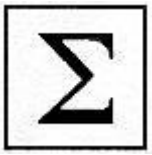


Kontrolní otázky:

1. K čemu slouží příkaz `echo`?
2. Jaké jednoduché datové typy můžeme použít v PHP?

3. Jaká pravidla musí splňovat identifikátor proměnné?
4. Jaké složitější datové typy můžeme použít v PHP?
5. Jaké operátory můžeme použít v PHP?

Shrnutí obsahu kapitoly



V této kapitole jste se seznámili s příkazem „echo“, který umožňuje vypsat text do okna prohlížeče. Dále jste se naučili práci s proměnnými, která je velmi důležitá pro tvorbu aplikací v jakémkoliv jazyce. Jazyk PHP umožňuje používat čtyři jednoduché datové typy – logickou hodnotu, celé číslo, desetinné číslo a řetězec znaků. Kromě toho umožňuje použití složitějších datových typů – polí a objektů. Pro tvorbu aplikací je možné někdy využít i tvorbu konstant. Jazyk PHP umožňuje používat unární a binární operátory a jeden ternární operátor.

5 Příkazy

V této kapitole se dozvíte:

- Jak zapsat větvení v PHP.
- Jaké příkazy cyklu je možné použít v PHP.
- Jak vložit do PHP souboru obsah jiného souboru.
- Jak opustit stránku v PHP.

Po jejím prostudování byste měli být schopni:

- Umět vytvořit podmínku v PHP pomocí příkazu if…else.
- Znát jednotlivé druhy cyklu, které jsou dostupné v PHP.
- Umět pracovat s příkazy require a include.
- Umět opustit stránku v PHP.

Klíčová slova této kapitoly:

if…else, while, do…while, for, break, continue, require, include, exit.

Doba potřebná ke studiu: 6 hodin



Průvodce studiem

Tato kapitola seznamuje studenty se základními řídícími strukturami jazyka PHP. Jedná se o podmínky, cykly, příkazy pro vkládání obsahu jiných souborů a příkazy pro opuštění stránky. Pokud máte základní znalosti algoritmizace, nebude pro Vás studium této kapitoly náročné. Pokud tyto znalosti nemáte, musíte sáhnout po jiné literatuře, protože základy algoritmizace nejsou předmětem této studijní opory.

Příkazy tvoří kostru aplikace. Se základními příkazy jsme se již seznámili v předchozích kapitolách. Jedná se příkaz pro výpis textu do výstup, přiřazení hodnoty do proměnné, definování konstanty nebo provedení výrazu. Kromě těchto příkazů, které se provádějí sekvenčně ve stejném pořadí v jakém jsou uvedeny ve zdrojovém kódu, můžeme v PHP používat příkazy pro větvení, opakování části kódu nebo pro vkládání obsahu jiných souborů do zdrojového kódu. S těmito příkazy se seznámíme v této kapitole.

5.1 Větvení programu

Pomocí větvení programu můžeme vytvořit části (bloky) kódu, které se provedou pouze za určitých podmínek. K větvení jsou v PHP k dispozici dva příkazy – první je příkaz `if…else` a druhým příkaz `switch`.



5.1.1 Příkaz if

Základním příkazem pro větvení programu v PHP je příkaz `if`. Tento příkaz je dostupný prakticky v každém programovacím jazyce. Syntaxe příkazu `if`

v PHP je stejná jako v Javě a je možné pomocí něho vytvořit neúplné, úplné nebo vnořené větvení.

Výpis 5.1 Syntaxe příkazu if.

```
<?php
  // neúplné větvení
  if (podmínka) {
    // příkazy
  }

  // úplné větvení
  if (podmínka) {
    // příkazy
  } else {
    // příkazy
  }

  // vnořené větvení
  if (podmínka1) {
    // příkazy
  } elseif (podmínka2) {
    // příkazy
  } else {
    // příkazy
  }
?>
```

Složené závorky nemusí být uvedeny, pokud má být v dané části proveden pouze jeden příkaz. V případě, že má být provedeno více příkazů, je nutné složené závorky uvést. Podmínka uvedená v části `if` nebo `elseif` musí být logickým výrazem, výsledkem tohoto výrazu musí být hodnota `true` nebo `false`.

Na počátku provádění příkazu `if` se vždy nejprve vyhodnotí podmínka uvedená v závorkách u příkazu `if`. Pokud je použito neúplné větvení a podmínka je vyhodnocena jako `true`, provede se příkaz (příkazy) následující za příkazem `if`. Pokud je podmínka vyhodnocena jako `false`, nestane se nic. V případě úplného větvení je začátek provádění příkazu `if` stejný. Pokud je výsledkem vyhodnocení podmínky `true`, provedou se příkazy v části `if`, pokud je výsledkem `false`, provedou se příkazy v části `else`. U vnořeného větvení se nejprve vyhodnotí podmínku uvedená v závorkách u příkazu `if`, tj. podmínka1. Pokud je výsledkem `true`, provedou se příkazy následující za ní.

Pokud je výsledkem `false`, vyhodnotí se podmínka v části `elseif`, tj. podmínka2. Pokud je podmínka2 vyhodnocena jako `true`, jsou provedeny příkazy následující za ní. Část `elseif` se může vyskytovat vícekrát, pokud je podmínka uvedená u příslušné části `elseif` vyhodnocena jako `false`, pokračuje se v následující části `elseif`. Pokud je podmínka u poslední části `elseif` vyhodnocena jako `false`, provedou se příkazy uvedené v části `else`.

Výpis 5.2 Příklad neúplného větvení.



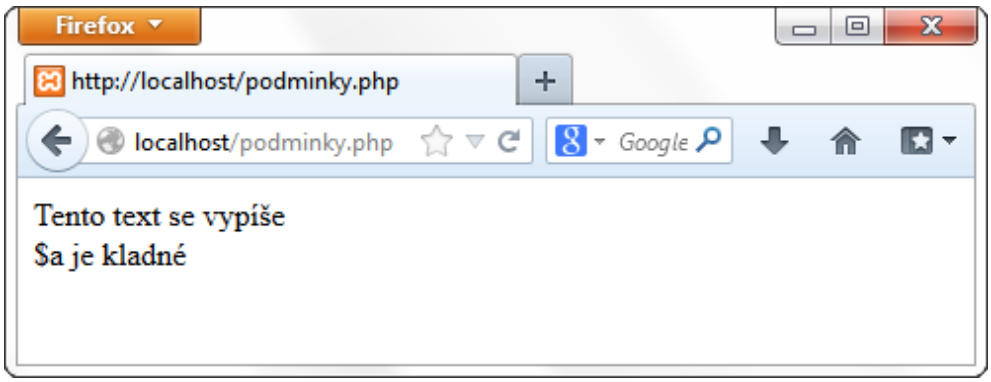
```
<?php
  if (false) echo "Tento text se nevypíše<br>";

  if (true) echo "Tento text se vypíše<br>";

  $a = 10;
  if ($a > 0) echo "\$a je kladné<br>";

  if ($a < 0) echo "\$a je záporné<br>";
?>
```

Výsledkem vykonání uvedených příkazů bude výstup zobrazený na následujícím obrázku.



Obrázek 5.1 Neúplné větvení.

Úplné větvení obsahuje kromě části `if` také část `else`.

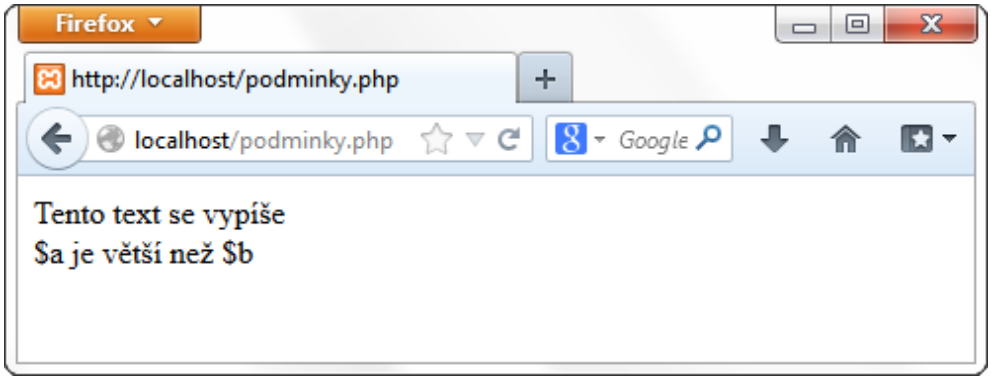
Výpis 5.3 Příklad úplného větvení.



```
<?php
  if (true) {
    echo "Tento text se vypíše<br>";
  } else {
    echo "Tento text se nevypíše<br>";
  }
```

```
  $a = 10;
  $b = 5;
  if ($a > $b) {
    echo "\$a je větší než \$b<br>";
  } else {
    echo "\$a je menší než \$b<br>";
  }
?>
```

Výsledný výstup zobrazený v prohlížeči je zobrazen na následujícím obrázku.



Obrázek 5.2 Úplné větvení.

Vnořené větvení obsahuje kromě částí `if` a `else` navíc jednu nebo více části `elseif`. Následující výpis obsahuje ukázku dvou vnořených větvení a obrázek 5.3 výstup zobrazený v prohlížeči.

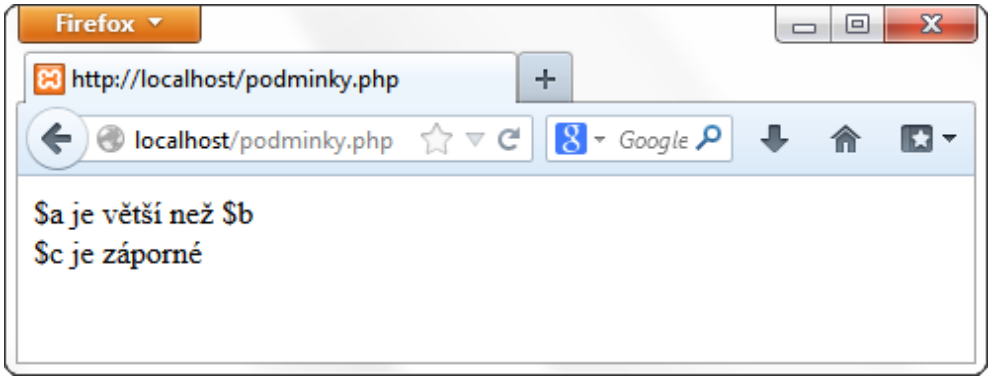
Výpis 5.4 Ukázka vnořeného větvení.



```
<?php
  $a = 10;
  $b = 5;
  if ($a > $b) {
    echo "\$a je větší než \$b<br>";
  } elseif ($a == $b) {
    echo "\$a je rovno \$b<br>";
  } else {
    echo "\$a je menší než \$b<br>";
  }


  $c = -3;
  if ($c > 0) {
    echo "\$c je kladné<br>";
  } elseif ($c == 0) {
```

```
    echo "\$c je rovno 0<br>";
  } else {
    echo "\$c je záporné<br>";
  }
?>
```



Obrázek 5.3 Výstup zobrazený v prohlížeči.

Jazyk PHP umožňuje pro vnořené větvení alternativní syntaxi. Za podmínkami jsou uvedeny dvojtečky a není potřeba uvádět složené závorky pro blok příkazů. Příkaz if je ukončen příkazem endif;. Tento způsob syntaxe je užitečný v případě, kdy potřebujeme vložit do výstupu HTML kód.

Výpis 5.5 Alternativní syntaxe vnořeného větvení.

```
<?php
  if (podmínka1):
    // příkazy
  elseif (podmínka2):
    // příkazy
  elseif (podmínka3):
    // příkazy
  elseif (podmínka4):
    // příkazy
  endif;
?>
```

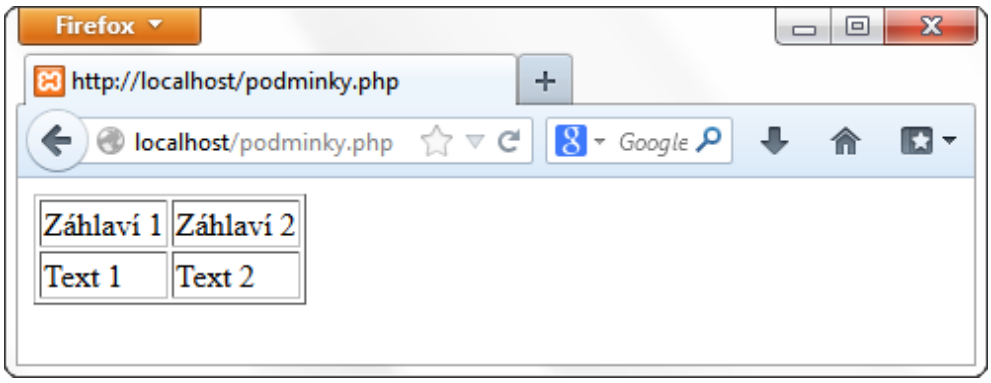
Konkrétní příklad použití alternativní syntaxe pro vnořené větvení je uveden v následujícím výpisu, výstup zobrazený v prohlížeči je zobrazen na obrázku pod ním.

Výpis 5.6 Konkrétní příklad použití alternativní syntaxe.



```
<?php
  $text = "tabulka";
  if ($text == "tabulka"):
?>
```

```
  <table border="1">
    <tr>
      <td>Záhlaví 1</td><td>Záhlaví 2</td>
    </tr>
    <tr>
      <td>Text 1</td><td>Text 2</td>
    </tr>
  </table>
<?php elseif ($text == "obrazek"): ?>
  <img src="obrazek" atl="">
<?php elseif ($text == "odstavec"): ?>
  <div>Odstavec</div>
<?php elseif ($text == "nadpis"): ?>
  <h1>Nadpis</h1>
<?php endif; ?>
```



Obrázek 5.4 Výstup použití alternativní syntaxe zobrazený v prohlížeči.

5.1.2 Příkaz switch



Druhým příkazem, pomocí něhož můžeme v PHP realizovat větvení programu, je příkaz `switch`. Jeho syntaxe je opět velmi podobná syntaxi příkazu `switch` v Javě. Následující výpis představuje obecnou syntaxi příkazu `switch`.

Výpis 5.7 Obecná syntaxe příkazu switch.

```
<?php
  switch(proměnná) {
    case návěští1:
      // příkazy1
    break;
    case návěští2:
      // příkazy2
    break;
    case návěští3:
      // příkazy4
```

```
    break;
    default:
      // příkazy5
  }
?>
```

V závorkách za příkazem `switch` následuje identifikátor proměnné, pomocí jejíž hodnoty chceme provést větvení. Pokud hodnota této proměnné odpovídá některému z návěští uvedených v částech `case`, provedou se příkazy, které následují za touto částí. Příkazy se provádí až do příkazu `break`. Pokud neodpovídá hodnota proměnné žádnému z návěští u částí `case`, provedou se příkazy v části `default`. Tato část je nepovinná a pokud je umístěna ve výčtu možností až jako poslední, není nutné za ní uvádět příkaz `break`, protože je ukončena složenou závorkou, která ukončuje celý příkaz `switch`. V některých případech je možné použít i syntaxi, která je uvedena v následujícím výpisu.

Výpis 5.8 Jiná možnost použití příkazu switch.

```
<?php
  switch(proměnná) {
    case návěští1: // zde chybí break;
    case návěští2:
      // příkazy1
    break;
    case návěští3: // zde chybí break;
    case návěští4:
      // příkazy2
    break;
    default:
      // příkazy3
  }
?>
```

Jak vidíme ve výpisu, u některých návěští chybí příkaz `break`. Použití tohoto kódu nahrazuje použití logické spojky `or` v příkazu `if`. Pokud bude hodnota proměnné rovna `návěští1` nebo `návěští2`, provedou se `příkazy1`. Pokud bude hodnota proměnné rovna `návěští3` nebo `návěští4`, provedou se `příkazy2`. V případě, že hodnota proměnné nebude rovna žádnému z návěští, provedou se `příkazy3`.

Proměnná, pomocí jejíž hodnoty se provádí větvení, může být jakéhokoliv jednoduchého datového typu, tj. celé číslo, desetinné číslo nebo řetězec.

Výpis 5.9 Ukázka použití příkazu switch.

```
<?php
  $jazyk = "cz";
  switch($jazyk) {
    case "en":
      echo "Angličtina";
    break;
    case "cz":
      echo "Čeština"; // toto se vypíše
    break;
    case "sk":
      echo "Slovenština";
    break;
    case "fr":
      echo "Francouzština";
    break;
    default:
      echo "Jiný jazyk";
  }
?>
```



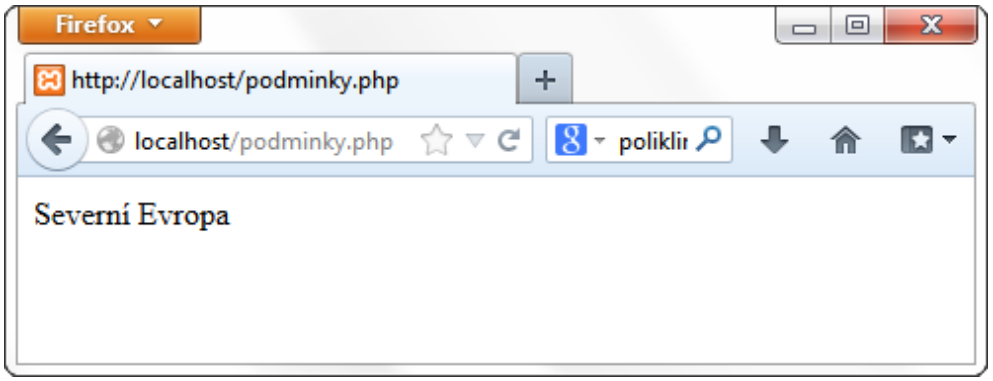
Následující výpis představuje použití varianty příkazu `switch`, kdy příkaz `break` není uveden u všech návěští.

Výpis 5.10 Ukázka použití příkazu switch nahrazujícího spojku "nebo".

```
<?php
  $zeme = "Švédsko";
  switch($zeme) {
    case "Velká Británie":
    case "Německo":
    case "Francie":
      echo "Západní Evropa";
    break;
    case "Česká republika":
    case "Slovenská republika":
      echo "Střední Evropa";
    break;
    case "Finsko":
    case "Švédsko":
    case "Norsko":
      echo "Severní Evropa";
    break;
    default:
      echo "Jiná část světa";
  }
```



```
?>
```



Obrázek 5.5 Výstup zobrazený v prohlížeči.

5.2 Cykly

Pro opakování stejné činností vícekrát po sobě slouží v programování cykly. I v tomto případě jsou cykly v jazyce PHP velmi podobné jako v jazyce Java. Jazyk PHP umožňuje použít čtyři různé cykly: `while`, `do…while`, `for` a `foreach`.

5.2.1 Cyklus while



Cyklus `while` představuje cyklus s podmínkou na začátku. Nejprve je otestována podmínka, která musí být ve tvaru logického výrazu. Pokud je podmínka vyhodnocena jako `true`, provede se příkaz nebo příkazy v těle cyklu a znovu se testuje podmínka. Pokud je podmínka stále `true`, vykonávají se opakovaně příkazy uvedené v těle cyklu. Jakmile je podmínka vyhodnocena jako `false`, provádění cyklu je ukončeno a program pokračuje příkazy následujícími za cyklem `while`. U tohoto cyklu se může stát, že se neprovede ani jednou.

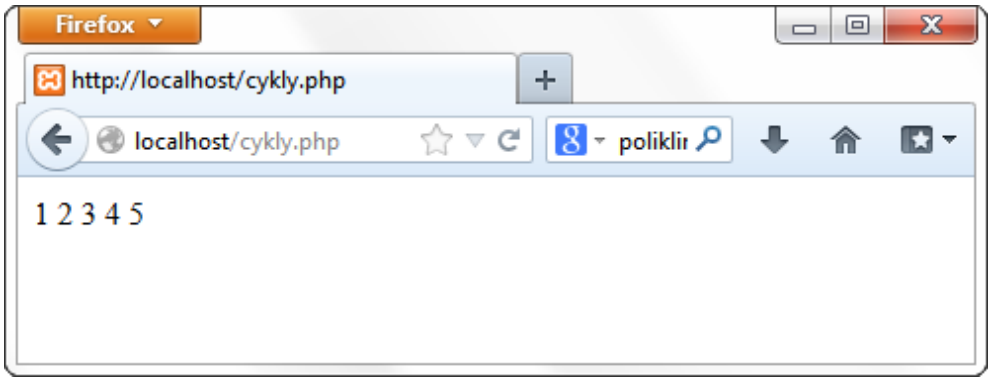
Výpis 5.11 Obecná syntaxe cyklu while.

```
<?php
  while (podmínka) {
    // příkazy
  }
?>
```

V následujícím výpisu je uvedena příklad použití cyklu `while`. Cyklus vypíše do výstupu čísla `1` až `5`.

Výpis 5.12 Ukázka použití cyklu while.

```
<?php
  $cislo = 1;
  while ($cislo <= 5) {
    echo $cislo . " ";
    $cislo++;
  }
?>
```



Obrázek 5.6 Výsledek použití cyklu while zobrazený v prohlížeči.

U příkazu `while` je možné použít alternativní syntaxi, podobně jako u příkazu `if`. Pro ukončení cyklu se používá příkaz `endwhile`. Následující výpisy představují obecnou alternativní syntaxi a konkrétní příklad použití této syntaxe – příklad vypíše čísla `1` až `5`.

Výpis 5.13 Obecná alternativní syntaxe cyklu while.

```
<?php
  while (podmínka):
    // příkazy
  endwhile;
?>
```

Výpis 5.14 Příklad použití alternativní syntaxe cyklu while.

```
<?php
  $cislo = 1;
  while ($cislo <= 5):
    echo $cislo . " ";
    $cislo++;
  endwhile;
?>
```



5.2.2 Cyklus do…while

Druhým cyklem, který si představíme, je cyklus `do…while`. Jedná se o cyklus s podmínkou na konci. Nejprve jsou provedeny příkazy v těle cyklu a poté je vyhodnocena podmínka. Pokud je jejím výsledkem `true`, znovu se provedou příkazy v těle cyklu a opět je vyhodnocena podmínka. Provádění cyklu pokračuje tak dlouho, dokud není podmínka není vyhodnocena jako `false`. Tento cyklus se vždy provede alespoň jednou.

Výpis 5.15 Obecná syntaxe cyklu do...while.

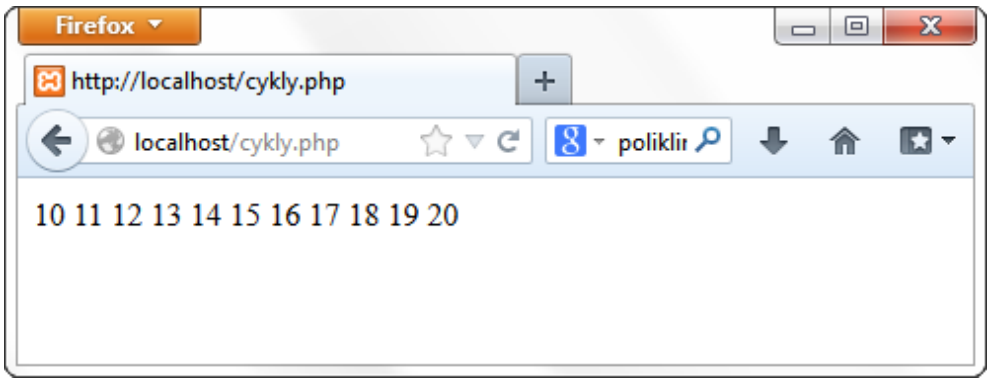
```
<?php
  do {
    // příkazy
  } while (podmínka);
?>
```

Následující výpis obsahuje příklad použití cyklu `do…while`. Tento příklad vypíše čísla od `10` do `20`.

Výpis 5.16 Příklad použití cyklu do...while.



```
<?php
  $cislo = 10;
  do {
    echo $cislo . " ";
    $cislo++;
  } while ($cislo < 21);
?>
```



Obrázek 5.7 Výsledek použití cyklu do...while.

5.2.3 Cyklus for



Dalším cyklem, který můžeme v PHP použít, je cyklus `for`. Tento cyklus bývá nazýván také jako cyklus s řídící proměnnou. Zatímco cykly `while` a `do…while` je možné použít i případě, kdy předem neznáme počet opakování,

která je potřeba provést, cyklus `for` je vhodný hlavně v případech, kdy počet opakování předem známe.

Výpis 5.17 Obecná syntaxe cyklu for.

```
<?php
  for (inicializace; podmínka; krok) {
    // příkazy
  }
?>
```



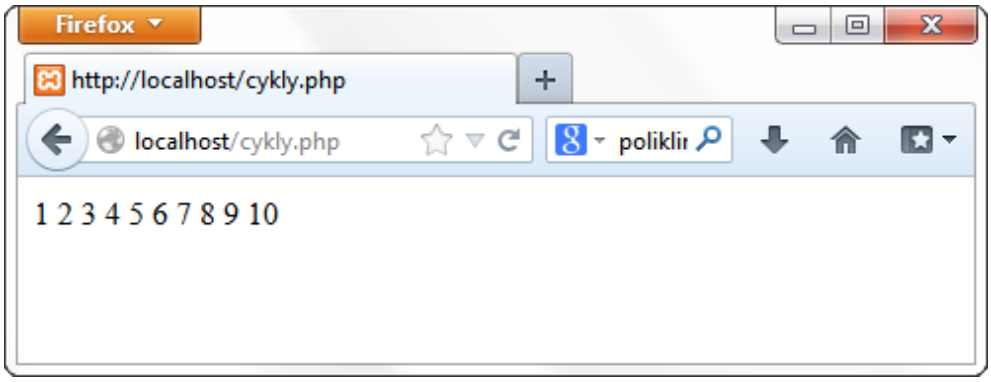
V závorce u příkazu jsou tři části oddělené navzájem znakem `;` (středník). První část zpravidla obsahuje inicializaci řídící proměnná, která určuje počet opakování cyklu. Tato část se provede pouze jednou před začátkem vykonávání cyklu. Druhou částí je `podmínka`, která určuje, kdy bude cyklus ukončen. Třetí část obsahuje krok, nejčastěji zde bývá příkaz pro inkrementaci nebo dekrementaci řídící proměnné. Po inicializaci se otestuje podmínka, pokud je vyhodnocena jako `true`, provedou se příkazy v těle cyklu. Poté se provedou příkazy ve třetí části a znovu se otestuje podmínka. Pokud je stále `true`, provedou se příkazy v těle cyklu a třetí část cyklu.

Následující výpis obsahuje příklad použití cyklu `for`, který vypíše do výstupu čísla do `1` do `10`.

Výpis 5.18 Konkrétní příklad použití cyklu for.

```
<?php
  for ($i = 1; $i <= 10; $i++) {
    echo $i . " ";
  }
?>
```

Na následujícím obrázku je zobrazen výsledek provedení tohoto cyklu zobrazený v prohlížeči.



Obrázek 5.8 Výsledek použití cyklu for.

Podobně jako příkazy `if` a `while`, i cyklus `for` umožňuje použití alternativní syntaxe. Tato syntaxe je uvedena v následujícím výpisu, k ukončení cyklu se používá příkaz `endfor`.

Výpis 5.19 Alternativní syntaxe příkazu for.

```
<?php
  for (inicializace; podmínka; krok):
    // příkazy
  endfor;
?>
```

Konkrétní příklad použití alternativní syntaxe je zobrazen v následujícím výpisu. Výstup tohoto výpisu je stejný jako ve výpisu 5.18.

Výpis 5.20 Příklad použití alternativní syntaxe cyklu for.



```
<?php
  for ($i = 1; $i <= 10; $i++):
    echo $i . " ";
  endfor;
?>
```

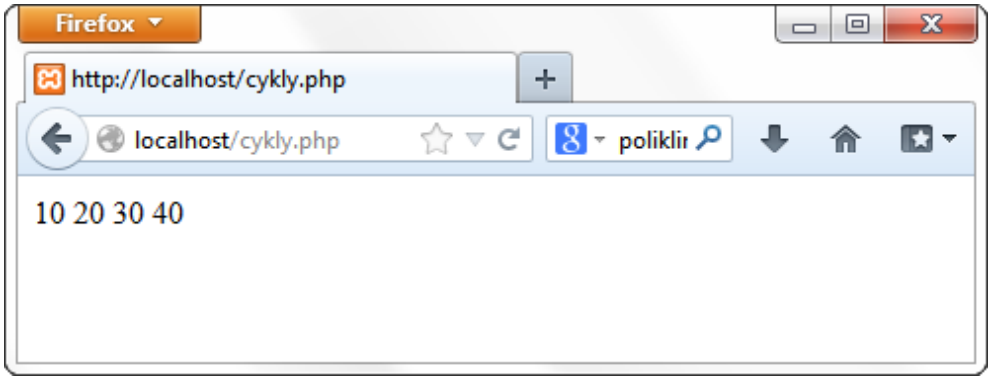
Typickým příkladem použití cyklu `for` je procházení spojitě indexovaného pole. Počet prvků pole je možné zjistit pomocí funkce `count()`, takže známe počet opakování, který je nutné provést, abychom prošli všechny prvky pole.

Výpis 5.21 Výpis spojitě indexovaného pole.



```
<?php
  $cisla = array(10, 20, 30, 40);
  for ($i = 0; $i < count($cisla); $i++) {
    echo $cisla[$i] . " ";
  }
?>
```

Výsledkem provedení tohoto cyklu je zobrazen na následujícím obrázku.



Obrázek 5.9 Obsah pole vypsaný v prohlížeči.

5.2.4 Cyklus foreach



Posledním cyklem, který zde uvedeme, je cyklus `foreach`. Tento cyklus se používá zejména při procházení nespojitě indexovaných polí nebo asociativních polí. Obecná syntaxe cyklu `foreach` může mít dvě podoby.

Výpis 5.22 Obecná syntaxe cyklu foreach.

```
<?php
  foreach (pole as $hodnota) {
    // příkazy
  }
  foreach (pole as $index => $hodnota) {
    // příkazy
  }
?>
```

V závorce za příkazem `foreach` je nejprve uveden výraz označující procházené pole. První varianta postupně prochází polem a každém kroku uloží hodnotu aktuálního prvku do proměnné `$hodnota`. Druhá varianta cyklu `foreach` navíc uloží v každém kroku index aktuálního prvku do proměnné `$index`. Poté se interní ukazatel pole posune na následující prvek. Pro procházení pole jsou navíc k dispozici funkce `reset()`, `next()`, `prev()`, `key()` a `current()`. Všechny tyto funkce mají jako parametr pole, se kterým pracujeme. Funkce `reset()` vrátí interní ukazatel v poli na začátek pole, tj. interní ukazatel se nastaví na první prvek pole. Funkce `next()` a `prev()` slouží k posunu interního ukazatele. Funkce `next()` posune interní ukazatel na následující prvek, tj. o jednu pozici doprava, funkce `prev()` jej posune na předchozí prvek, tj. o jednu pozici doleva. Obě funkce navíc vrátí hodnotu prvku, na který se posunuly. Pokud se pomocí této funkce posune

interní ukazatel mimo pole, funkce vrátí hodnotu `false`. Funkce `key()` vrací index aktuálního prvku v poli, na který ukazuje interní ukazatel. Funkce `current()` pak vrací hodnotu aktuálního prvku v poli.

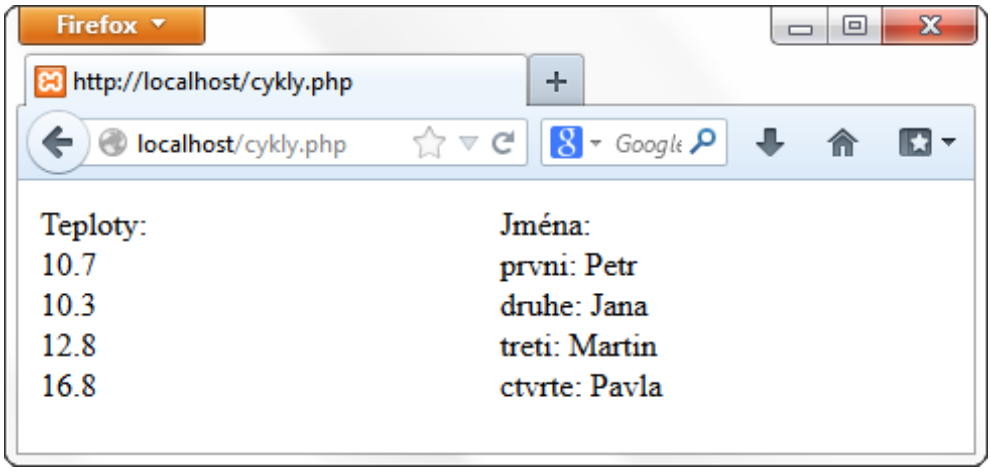
Následující kód ukazuje použití obou variant cyklu `foreach` a na obrázku dále je zobrazen výstup zobrazený v prohlížeči. K vypsání polí vedle sebe je použita tabulka bez ohraničení.

Výpis 5.23 Ukázka použití obou variant cyklu foreach.

```
<?php
  echo '<table border="0" width="100%"><tr><td
   width="50%">';
  // nespojitě indexované pole
  $teploty = array(0 => 10.7, 4 => 10.3, 8 => 12.8,
   12 => 16.8);
  echo "Teploty:<br>";
  foreach ($teploty as $hodnota) {
    echo $hodnota . "<br>";
  }

  echo '</td><td>';

  // asociativní pole
  $jmena = array("prvni" => "Petr", "druhe" =>
   "Jana", "treti" => "Martin", "ctvrte" => "Pavla");
  echo "Jména:<br>";
  foreach ($jmena as $index => $hodnota) {
    echo $index . ": " . $hodnota . "<br>";
  }
  echo '</td></tr></table>';
?>
```



Obrázek 5.10 Použití cyklu foreach.

INVESTICE DO ROZVOJE VZDĚLÁVÁNÍ

5.2.5 Příkazy break a continue



Při používání cyklů je možné použít příkazy `break` a `continue`. Příkaz `break` ukončí provádění celého cyklu a vykonávání kódu pokračuje za cyklem. Příkaz `continue` ukončí aktuální iteraci cyklu a vrátí se na část vyhodnocení podmínky.

5.3 Vkládání souborů do stránek PHP



Jazyk PHP obsahuje dva příkazy, které načítají a provádějí kód ze zadaného souboru. Těmito příkazy jsou `require` a `include`. Použitím těchto příkazů je možné rozdělit aplikaci do více souborů, ve kterých budou umístěny společné konstanty nebo funkce. Vložením obsahu takového souboru pak můžeme použít konstantu nebo funkci na různých místech vytvářené aplikace. Celá aplikace je tím pádem přehlednější a lépe se s ní pracuje.

Příkaz `require` vloží obsah daného souboru na místo, kde je zavolán. Kód vkládaného souboru je tedy součástí kódu souboru, kde byl příkaz `require` zavolán. Protože příkaz `require` vkládá obsah daného souboru na místo, kde byl zavolán, není možné ho použít v cyklu pro načtení více různých souborů. K tomu je vhodnější použít příkaz `include`, který také zpřístupní kód z externího souboru, ale kód externího souboru se provádí při každém výskytu příkazu.

Pokud uvedeme u příkazu `require` nebo `include` pouze název souboru (např. `soubor.php`), hledá se daný soubor ve stejném adresáři, jako soubor, kde jsme tyto příkazy použili. Název souboru je tedy určen relativně. Samozřejmě je možné použít i absolutní cestu k souboru.

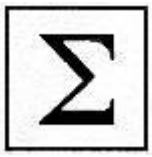
5.4 Opuštění stránky PHP

Posledním příkazem, který si v této kapitole uvedeme, je příkaz `exit`. Tento příkaz slouží k opuštění stránky PHP a k zastavení jejího dalšího provádění. Funguje podobně jako příkaz `break`, s tím rozdílem, že funguje na dokument jako na celek a ne pouze na danou část (například na cyklus). Příkaz `exit` je vhodné použít například při práci s databází, pokud dojde k chybě v komunikaci s databázovým serverem.



Kontrolní otázky:

1. K čemu slouží v PHP příkaz `if`?
2. Jaké příkazy cyklu můžeme použít v PHP?
3. K čemu slouží příkazy `break` a `continue`?
4. Jaký je rozdíl mezi příkazy `require` a `include`?
5. K čemu slouží příkaz `exit`?



Shrnutí obsahu kapitoly

V této kapitole jste se v úvodu seznámili s příkazem pro vytvoření větvení v PHP. Dále jste se seznámili s jednotlivými typy příkazů cyklu v PHP a s rozdíly mezi nimi. Jazyk PHP umožňuje použít cyklus s podmínkou na začátku, cyklus s podmínkou na konci i cyklus s řídící proměnnou. Příkazy `require` a `include` umožňují vložit do souboru obsah jiného souboru. Příkaz `exit` umožňuje opustit právě prováděný PHP skript.

6 Zpracování vstupu od uživatele

V této kapitole se dozvíte:

- Jakým způsobem může být zadán vstup od uživatele.
- Jak zpracovávat jednotlivé druhy uživatelského vstupu.

Po jejím prostudování byste měli být schopni:

- Umět zpracovávat vstup od uživatele odeslaný metodou GET i metodou POST.

Klíčová slova této kapitoly:

GET, POST, superglobální pole.

Doba potřebná ke studiu: 4 hodiny

Průvodce studiem



V této kapitole se seznámíte se zpracováním vstupních dat od uživatelů. Požadavky mohou být na server odeslány dvěma metodami – metodou GET nebo metodou POST. Princip zpracování parametrů je u obou metod stejný, rozdíl je zejména v tom, kde jsou odesílané parametry umístěny. V závěru kapitoly se seznámíte se superglobálními proměnnými v PHP.

Jednou z nejdůležitějších činností při fungování jakékoliv aplikace je komunikace s uživatelem. Na jedné straně je potřeba přijímat a zpracovávat vstupy od uživatele a na druhé straně je potřeba na ně reagovat a generovat odpovídající výstupy. V případě webové aplikace uživatel zadává vstupy nejčastěji pomocí webového prohlížeče, kterým odesílá požadavky na server. Vstup od uživatele může mít dvojí formu. První formou vstupu je zadání parametrů, které jsou součástí URL adresy. Takové parametry se na server odesílají metodou GET. Druhou formou jsou formuláře umístěné na WWW stránce. Formulář může být odeslán jak metodou GET, tak metodou POST.

6.1 Parametry v rámci URL adresy



Parametry uvedené v URL adrese odkazující na daný PHP soubor jsou ve tvaru

```
jmeno_parametru=hodnota_parametru
```

Tyto parametry jsou součástí URL adresy a jsou uvedeny v části dotaz, která začíná znakem `?` (otazník). Pokud je parametrů více, jsou spojeny pomocí znaku `&` (ampersand). U názvu nebo hodnot parametrů se neuvádějí uvozovky ani jiné speciální znaky. Názvy i hodnoty parametrů by měly být bez diakritiky a bez mezer. Pokud použijeme mezeru, webový prohlížeč ji nahradí její reprezentací v hexadecimální soustavě.

Příklady URL adres, které obsahují parametry:

1. `http://www.osu.cz/index.php?kategorie=27`



Adresa obsahuje jeden parametr s názvem `kategorie`, hodnota parametru je `27`.

2. `http://www.osu.cz/index.php?kategorie=888&id=11243`

Adresa obsahuje dva parametry. První parametr se jmenuje `kategorie` a má hodnotu `888`. Druhý parametr se jmenuje `id` a má hodnotu `11243`.

3. `http://www.osu.cz/index.php?id=3313&search=miarka&seaktg=zam`

Adresa obsahuje tři parametry. První parametr má název `id` a jeho hodnota je `3313`. Druhý parametr se jmenuje `search` a má hodnotu `miarka`. Třetí parametr se jmenuje `seaktg` a má hodnotu `zam`.

Vstup od uživatele zadávaný prostřednictvím parametrů v URL adrese zpravidla nebývá zadáván přímo uživatelem. Uživatel by musel znát parametry, které může použít a také hodnoty, které vedou k nějakému výsledku. Nejčastěji odesílání parametrů pomocí URL adresy funguje tak, že uživatel pouze kliká na odkazy umístěné na stránce. Odkaz adresy je pak připraven tak, aby se zaslaly správné parametry a správné hodnoty.



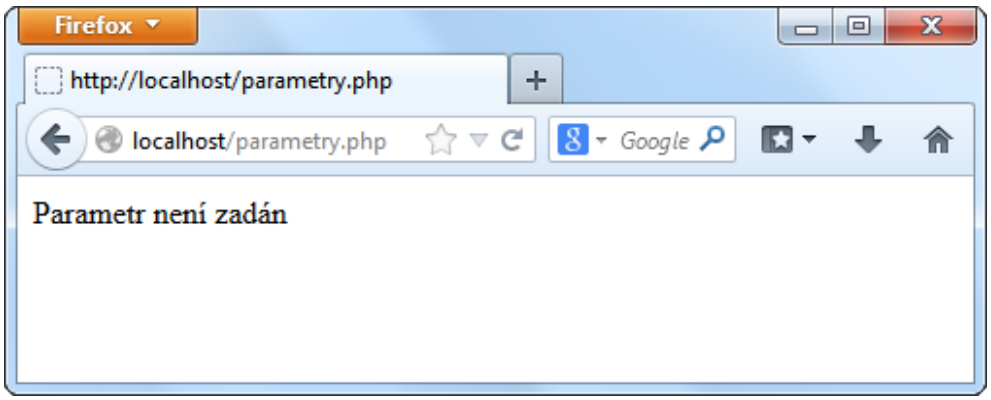
Ke zpracování parametrů zaslaných metodou GET slouží superglobální pole `$_GET`. Přívlastek superglobální znamená, že toto pole je přístupné v kterémkoliv souboru. Pole `$_GET` je asociativní, tzn. indexy pole jsou tvořeny textovými řetězci a představují názvy jednotlivých parametrů odeslaných metodou GET. Pokud bude součástí URL adresy například parametr s názvem parametr, jeho hodnotu získáme pomocí `$_GET['parametr']`. Pokud potřebujeme otestovat, zda byl nějaký konkrétní parametr zadán, můžeme k tomuto účelu použít funkci `isset()`, viz. následující výpis.

Výpis 6.1 Získání parametru zaslaného metodou GET

```
<?php
  if (isset($_GET['parametr'])) {
    echo $_GET['parametr'];
  } else {
    echo "Parametr není zadán";
  }
?>
```

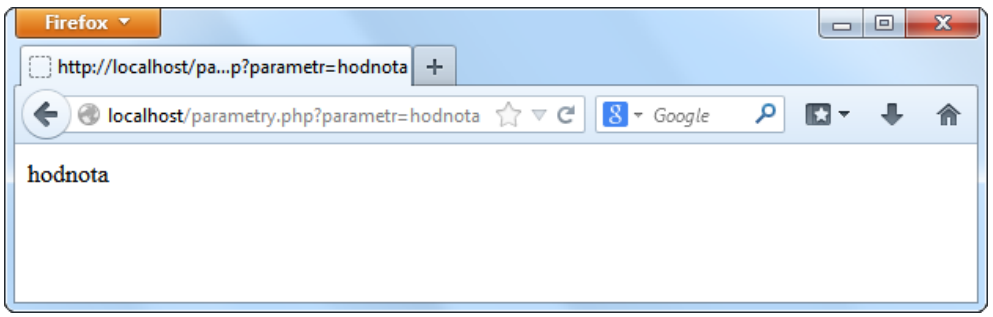


Uvažujme nyní, že kód uvedený ve výpisu 6.1 je uložen v souboru parametry.php, který je umístěn v kořenovém adresáři lokálního serveru (tj. serveru `localhost`). Pokud do prohlížeče zadáme adresu `http://localhost/parametry.php`, vypíše se text „Parametr není zadán“, protože tato adresa žádný parametr neobsahuje. Výsledek je zobrazen na následujícím obrázku.



Obrázek 6.1 Parametr nezadán.

Pokud zadáme adresu `http://localhost/parametry.php?parametr=hodnota`, vypíše se v prohlížeči text „hodnota“. Výstup v prohlížeči je zobrazen na následujícím obrázku.



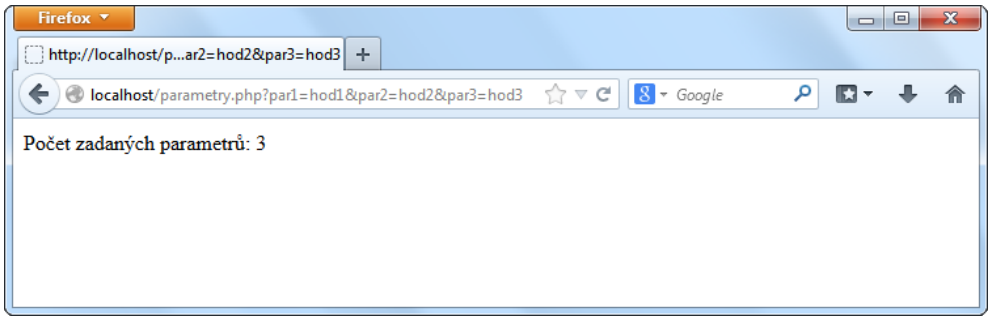
Obrázek 6.2 Vypsání hodnoty parametru.

Protože proměnná $_GET je pole, můžeme s ní pracovat jako s jakýmkoliv jiným polem. Můžeme například zjistit počet prvků v poli, tzn. zjistit počet

odeslaných parametrů. Uvažujme případ, kdy bude v adresním řádku prohlížeče zadána adresa `http://localhost/parametry.php?par1=hod1&par2=hod2&par3=hod3`. V adrese jsou uvedeny 3 parametry. Následující výpis zobrazuje výpis počtu parametrů a obrázek pod ním výsledek zobrazený v prohlížeči.

Výpis 6.2 Počet parametrů odeslaných metodou GET

```
<?php
  echo "Počet zadaných parametrů: " . count($_GET);
?>
```

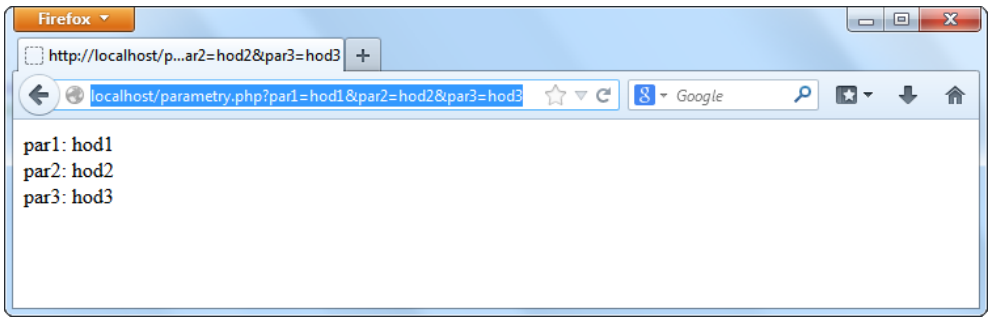


Obrázek 6.3 Vypsání počtu zadaných parametrů.

Pokud zadáme stejnou adresu jako v předchozím příkladu, můžeme k výpisu všech zadaných parametrů a jejich hodnot použít cyklus `foreach`. Výpis 6.3 ukazuje použití tohoto cyklu, obrázek 6.4 pak zobrazuje výsledek zobrazený v prohlížeči.

Výpis 6.3 Vypsání všech zadaných parametrů a jejich hodnot.

```
<?php
  foreach ($_GET as $nazev => $hodnota) {
    echo $nazev . ": " . $hodnota . "<br>";
  }
?>
```



Obrázek 6.4 Vypsání všech zadaných parametrů.

6.2 Formuláře

Druhou možností, jak získat vstupní parametry od uživatele, je použití formulářů. Formuláře v jazyce HTML umožňují umístit na www stránku ovládací prvky (formulářové prvky), pomocí nichž může uživatel zaslat parametry na webový server. S formuláři jste se seznámili v kapitole 2. U formuláře si můžeme vybrat, jakou metodou bude odeslán na server. Pokud žádnou metodu neuvedeme, použije se metoda GET. Kromě toho můžeme použít metodu POST. Při odeslání formuláře metodou GET je zpracování parametrů stejné jako v případě přímého zadání parametru do URL adresy. názvy formulářových prvků a jejich hodnoty jsou totiž při odeslání formuláře přidány do URL adresy.

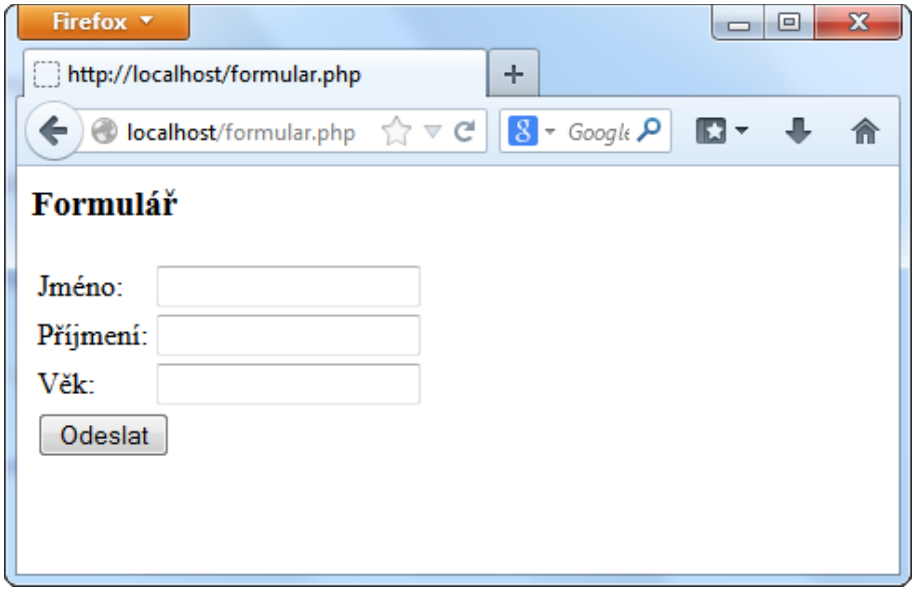
Následující výpis představuje HTML kód pro výpis jednoduchého formuláře. Formulář obsahuje 3 textová pole a jedno tlačítko. Obrázek pod výpisem pak zobrazuje tento formulář ve webovém prohlížeči.

Výpis 6.4 Jednoduchý formulář

```
<html>
  <head>
    <title>Formulář</title>
  </head>
  <body>
    <h3>Formulář</h3>
    <form action="formular.php" method="get">
    <table border="0" cellspacing="0">
      <tr>
        <td>Jméno:</td>
        <td><input type="text" name="jmeno"></td>
      </tr>
      <tr>
        <td>Příjmení:</td>
        <td><input type="text" name="prijmeni"></td>
      </tr>
      <tr>
        <td>Věk:</td><td><input type="text"
         name="vek"></td>
      </tr>
      <tr>
        <td colspan="2"><input type="submit"
         value="Odeslat" name="odeslat"></td>
      </tr>
    </table>
    </form>
```



```
  </body>
</html>
```



Obrázek 6.5 Formulář



Metoda odeslání je u formuláře nastavena na GET. Pokud klikneme na tlačítko *Odeslat*, formulář se odešle na server a názvy ovládacích prvků jsou přidány jako parametry do URL adresy. Pokud formulář odešleme na server aniž bychom vyplnili nějaké údaje, jsou i přesto všechny ovládací prvky přidány do parametrů. URL adresa tedy bude ve tvaru `http://localhost/formular.php?jmeno=&prijmeni=&vek=&odeslat=Odeslat`. Pokud do některého ovládacího prvku nevyplníme hodnotu, parametr je i přesto v URL adrese vytvořen. Jeho hodnotou je ale prázdný řetězec znaků. Testování metodou `isset()` tedy v tomto případě nepomůže, protože všechny parametry existují. Ale neobsahují žádnou hodnotu. Následující výpis řeší vypsání hodnot u jednotlivých parametrů. Pokud některý z parametrů nebyl zadán, vypíše se text „nezadáno“. Ve výpisu je použit ternární operátor.

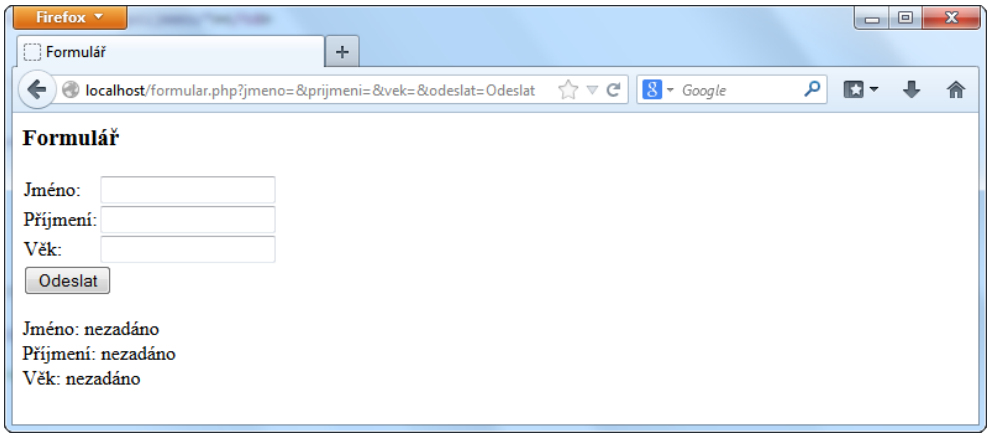
Výpis 6.5 Ošetření nezadaných parametrů – metoda GET



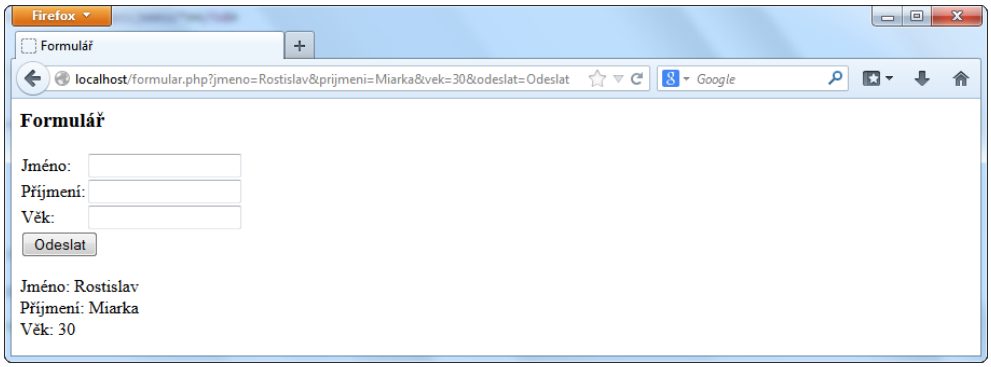
```
<?php
  echo "Jméno: ";
  echo (strlen($_GET['jmeno']) == 0) ? "nezadáno<br>"
   : $_GET['jmeno'] . "<br>";
  echo "Příjmení: ";
  echo (strlen($_GET['prijmeni']) == 0) ?
   "nezadáno<br>" : $_GET['prijmeni'] . "<br>";
  echo "Věk: ";
  echo (strlen($_GET['vek']) == 0) ? "nezadáno<br>" :
   $_GET['vek'] . "<br>";
```

```
?>
```

Pokud nevyplníme žádné hodnoty a odešleme formulář, bude výsledek stejný jako na následujícím obrázku – parametry a jejich hodnoty jsou zobrazeny pod formulářem. Pokud některé z polí vyplníme, bude výsledek stejný jako na obrázku 6.7.



Obrázek 6.6 Výsledek odeslání formuláře – metoda GET – prázdný formulář



Obrázek 6.7 Výsledek odeslání formuláře – metoda GET – vyplněný formulář



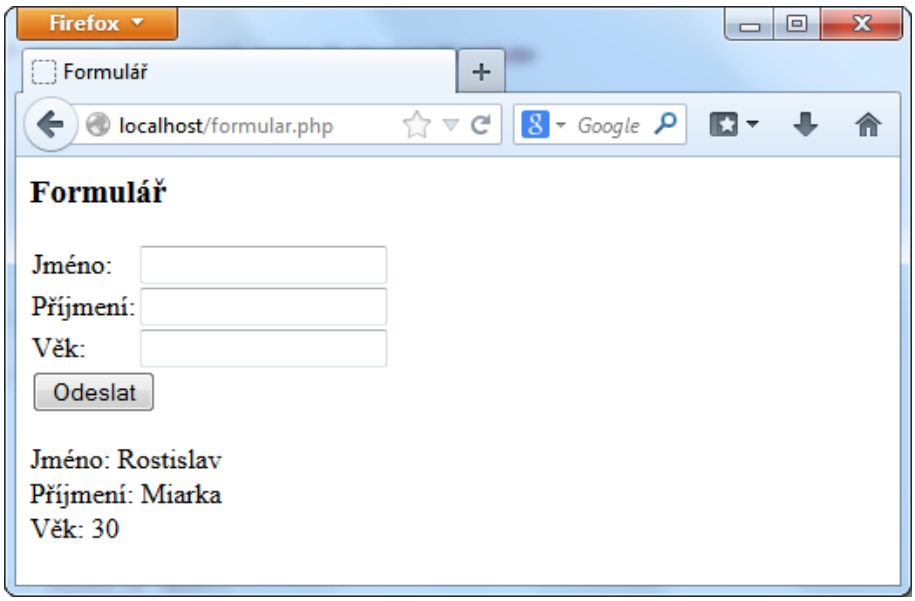
Pokud změníme metodu odeslání formuláře na POST, po odeslání formuláře bude URL adresa ve stejné podobě jako před odesláním. Parametry a jejich hodnoty nejsou viditelné v URL adrese, ale odesílají se v těle požadavku. Ke zpracování parametrů odeslaných metodou POST je v PHP k dispozici superglobální pole `$_POST`. Práce s tímto polem je stejná jako v případě pole `$_GET`, rozdíl je pouze v názvu pole. I při odeslání metodou POST se z nevyplněných ovládacích prvků vytvoří parametry, jejichž hodnotou je prázdný řetězec.

Následující výpis řeší stejnou úlohu jako výpis 6.5, rozdíl je pouze v tom, že k odeslání formuláře je použita metoda POST.

Výpis 6.6 Ošetření nezadaných parametrů – metoda POST

```
<?php
  echo "Jméno: ";
  echo (strlen($_POST['jmeno']) == 0) ?
   "nezadáno<br>" : $_POST['jmeno'] . "<br>";
  echo "Příjmení: ";
  echo (strlen($_POST['prijmeni']) == 0) ?
   "nezadáno<br>" : $_POST['prijmeni'] . "<br>";
  echo "Věk: ";
  echo (strlen($_POST['vek']) == 0) ? "nezadáno<br>"
   : $_POST['vek'] . "<br>";
?>
```

Výsledek po odeslání vyplněného formuláře je zobrazen na následujícím obrázku. Rozdíl mezi obrázky 6.7 a 6.8 je pouze v URL adrese, která je zobrazena ve webovém prohlížeči.



Obrázek 6.8 Výsledek odeslání formuláře – metoda POST – vyplněný formulář

6.3 Superglobální proměnné

Superglobální proměnné jsou proměnné, které jsou přístupné v jakékoliv části kteréhokoliv PHP skriptu (souboru). Proměnné `$_GET` a `$_POST` patří mezi superglobální proměnné – obě proměnné představují asociativní pole, ve kterém jsou uloženy hodnoty parametrů odeslaných metodou GET nebo POST. Indexy v poli tvoří názvy parametrů. Kromě těchto dvou polí obsahuje PHP i další superglobální proměnné. Nejdůležitější z nich jsou:

- `$_SERVER` – pole, které obsahuje informace jako jsou hlavičky, cesty a umístění skriptů. Záznamy v tomto poli vytváří webový server.

- `$_FILES` – asociativní pole položek, které byly uploadovány na server pomocí metody POST.

- `$_COOKIE` – asociativní pole proměnných, které byly zaslány skriptu přes Cookies.

- `$_SESSION` – asociativní pole obsahující proměnné v relacích, které jsou dostupné aktuálnímu skriptu. Relace budou podrobněji popsány v následujících kapitolách.

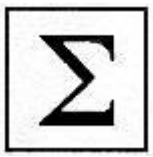
- `$_REQUEST` – asociativní pole, které obsahuje prvky z polí `$_GET`, `$_POST` a `$_COOKIE`. Z hlediska bezpečnosti ale není příliš vhodné toto pole používat. Bezpečnější je používat vždy pole, které odpovídá odeslanému požadavku.

Kontrolní otázky:



1. Jakým způsobem získáme hodnotu parametru odeslaného metodou GET?
2. Jakým způsobem získáme hodnotu parametru odeslaného metodou POST?
3. Co je to superglobální proměnná?

Shrnutí obsahu kapitoly



V této kapitole jste se seznámili se dvěma způsoby odeslání parametrů na webový server. První způsob představuje odeslání požadavku metodou GET. Parametry a jejich hodnoty jsou v tomto případě součástí URL adresy. K získání hodnot parametrů slouží superglobální pole `$_GET`. Druhým způsobem je odeslání požadavku metodou POST, při kterém jsou parametry a jejich hodnoty umístěny v těle požadavku. K získání hodnot parametrů slouží superglobální pole `$_POST`. Kromě těchto dvou polí obsahuje PHP i další superglobální pole.



Korespondenční úkol č. 1

Vytvořte PHP skript, který po otevření zobrazí formulář pro vyhledávání hardwaru podle parametrů. Typ hardwaru si určete sami. Formulář musí obsahovat minimálně 5 parametrů. Ve formuláři použijte alespoň 4 různé druhy formulářových prvků (textové pole, zaškrtávací pole, rozbalovací seznam...).

Formulář se bude odesílat na stejný PHP skript. Po odeslání formuláře se vypíší parametry, které uživatel zadal do formuláře. Parametry budou vypsány jako tabulka - v prvním sloupci bude název parametru, ve druhém sloupci zadaná hodnota.

Příklady parametrů pro různý hardware:

- monitor - uhlopříčka, pozorovací úhly, odezva...
- CPU - taktovací frekvence, počet jader, výrobce..

Příklad výstupu po zpracování formuláře pro monitor:

Parametr	Hodnota
úhlopříčka	24"
odezva	5 ms
...	...

7 Funkce

V této kapitole se dozvíte:

- Jak vytvořit uživatelsky definovanou funkci v PHP.
- Jakým způsobem se předávají parametry v PHP.
- Jaké jsou nejdůležitější předdefinované funkce v PHP.

Po jejím prostudování byste měli být schopni:

- Vytvořit vlastní uživatelsky definovanou funkci.
- Umět použít ve funkci předávání parametrů hodnotou.
- Umět použít ve funkci předávání parametrů odkazem.

Klíčová slova této kapitoly:

funkce, function, return, předávání parametrů.

Doba potřebná ke studiu: 8 hodiny

Průvodce studiem



Studium této kapitoly předpokládá základní znalosti algoritmizace, konkrétně tvorbu uživatelských funkcí (metod, procedur). Při volání funkce mohou být parametry předány dvěma způsoby – hodnotou nebo odkazem. Ve druhé části se seznámíte s nejdůležitějšími předdefinovanými funkcemi jazyka PHP, které Vám mohou usnadnit tvorbu Vašich aplikací.

Funkce v jazyce PHP představují určitý blok kódu, který je pouze jednou nadefinován a může být pak v jiných částech programu vyvolán. V jiných programovacích jazycích bývají někdy funkce také nazývány jako metody. Funkce většinou mají jako vstup argument nebo argumenty, které bývají označovány také jako parametry. Funkce provádí předdefinovanou sadu operací a vrací výslednou hodnotu. Pomocí funkcí můžeme vytvářet modulární aplikace. Kód, který by se v aplikaci opakoval na více místech, je nadefinován jako funkce a na potřebných místech je pouze zavolán.

Používání funkcí šetří čas a snižuje chybovost kódu. Jazyk PHP obsahuje velké množství předdefinovaných funkcí, s některými z nich jsme se již setkali. Mezi předdefinované funkce jazyka PHP patří například `gettype()`, `count()`, `isset()` nebo `defined()`. S dalšími předdefinovanými funkcemi se seznámíte na konci této kapitoly. Kromě předdefinovaných funkcí je možné v PHP vytvářet uživatelsky definované funkce. S uživatelsky definovanými funkcemi se seznámíte v následujících odstavcích.

7.1 Deklarace, definice a volání funkce

Deklarace funkce se provádí příkazem `function`, za kterým následuje název funkce (identifikátor funkce). Pro názvy funkcí platí stejná pravidla jako pro

názvy proměnných, tj. je možné používat písmena anglické abecedy, číslice a znak podtržítka. Za názvem funkce následuje seznam parametrů (argumentů) funkce, který je uveden v kulatých závorkách. Jednotlivé parametry jsou odděleny čárkami. Za kulatými závorkami následují složené závorky, které obsahují tělo funkce. Tělo funkce tvoří kód, který má funkce provádět, a označuje se jako definice funkce. Funkce nemusí obsahovat žádné parametry, ale za jejím názvem musí být uvedeny prázdné kulaté závorky. Funkce může vracet nějakou hodnotu. Pokud má funkce vracet hodnotu, musí tělo funkce obsahovat příkaz `return`, který bývá zpravidla uveden na konci těla funkce.

Volání funkce provedeme uvedením jejího názvu, za kterým následují parametry uvedené v kulatých závorkách. Pokud funkce vrací nějakou hodnotu, měla by být tato hodnota uložena do nějaké proměnné nebo jinak zpracována. Při volání funkce je samozřejmě potřeba dodržet počet parametrů, které funkce obsahuje.

Následující výpis obsahuje obecnou deklaraci, definici a volání funkce.

Výpis 7.1 Obecná definice a volání funkce.



```
<?php
  // deklarace a definice funkce
  function nazev_funkce(parametry) {
    ...tělo funkce...
  }

  // volání funkce
  nazev_funkce(parametry);
?>
```

Nyní si uveďme konkrétní příklady funkce. Nejprve to bude funkce, která nevrací žádnou hodnotu. Funkce má jako parametr text, který představuje titulek stránky v HTML. Text je vložen mezi značky `<title>` a `</title>` a je vypsán do výstupu v prohlížeči. Druhá funkce má dva parametry a vrací jejich součet. Uvedení závorek u součtu v příkazu `return` není nutné a je uvedeno pouze pro přehlednost.

Výpis 7.2 Příklad konkrétních funkcí.

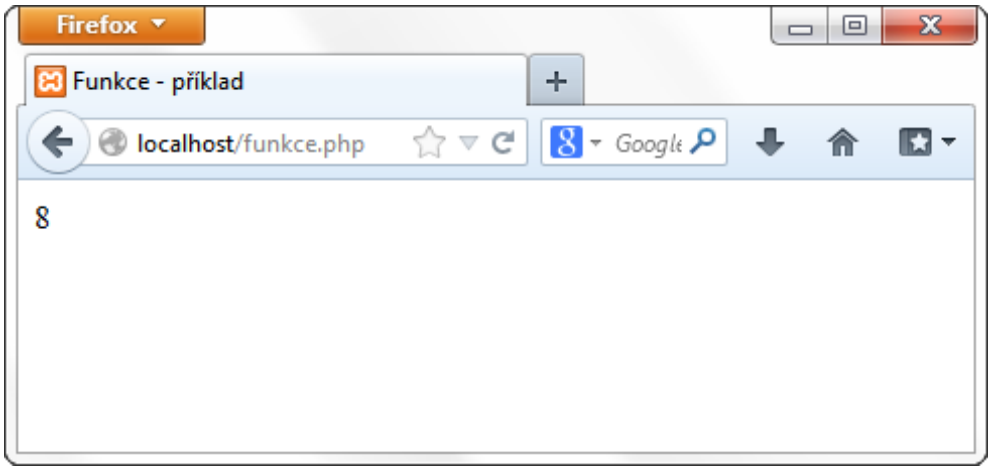


```
<?php
  function titulek($text) {
    echo "<title>" . $text . "</title>";
  }
```

```
  titulek("Funkce - příklad");

  function soucet($a, $b) {
    return ($a + $b);
  }

  echo soucet(5, 3);
?>
```



Obrázek 7.1 Ukázka použití funkce.

7.2 Předávání parametrů

Předávání parametrů (argumentů) v PHP funguje úplně stejně jako ve většině jiných programovacích jazyků. V deklaraci funkce jsou v závorce uvedeny formální parametry, jejichž identifikátory (názvy) v podstatě pouze představují symboly používané v těle funkce. Při volání funkce jsou formální parametry nahrazeny skutečnými parametry, tj. skutečnými hodnotami, které funkcí „posíláme“. Při předávání parametrů funkci rozlišujeme předávání parametrů hodnotou a předávání odkazem.



Jazyk PHP používá implicitně předávání parametrů hodnotou. Při volání funkce je v rámci vykonávání těla funkce vytvořena lokální kopie parametru, se kterou se pracuje. Pokud tuto hodnotu uvnitř funkce změníme, nemá to na předávanou hodnotu žádný vliv. Následující kód představuje ukázku použití předávání parametru hodnotou a jeho efekt. Obrázek pak zobrazuje výsledek v prohlížeči.

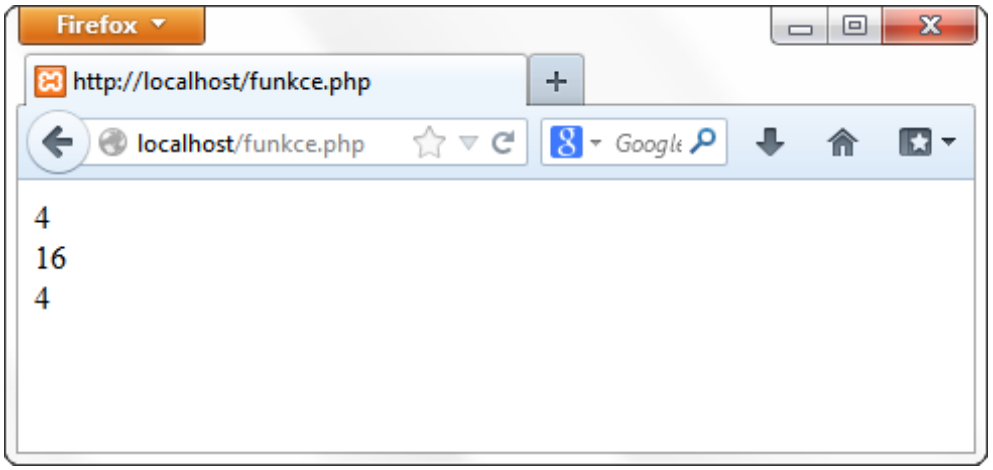
Výpis 7.3 Předávání parametrů hodnotou.

```
<?php
  function mocnina($x) {
    $x = $x * $x;
```



```
    echo $x . "<br>";
  }

  $a = 4;
  echo $a . "<br>";
  mocnina($a);
  echo $a . "<br>";
?>
```



Obrázek 7.2 Předávání parametrů hodnotou.

Vidíme, že nejprve je vypsána původní hodnota proměnné `$a`. Následně je volána funkce `mocnina()`, ve které je předaný parametr umocněn a vytištěn na obrazovku. Hodnota parametru je `4`, takže se vytiskne číslo `16`. Nakonec je opět vytištěna hodnota proměnné `$a`. Je vytištěna původní hodnota, protože umocnění se provedlo s lokální kopií předaného parametru. Tato kopie zanikla při ukončení provádění těla funkce.



Druhou možností, jak předávat v PHP parametry funkcím, je předávání odkazem. Funkci není předávána hodnota parametru, ale odkaz na původní předávanou hodnotu. Efekt tohoto způsobu předávání parametrů je, že změny parametru uvnitř těla funkce se projeví i mimo funkci. Pokud chceme parametr předat odkazem, je potřeba před jeho identifikátor umístit znak `&` (ampersand). Následující výpis ukazuje použití parametru předaného odkazem, na obrázku je pak zobrazen výsledek zobrazený v prohlížeči.

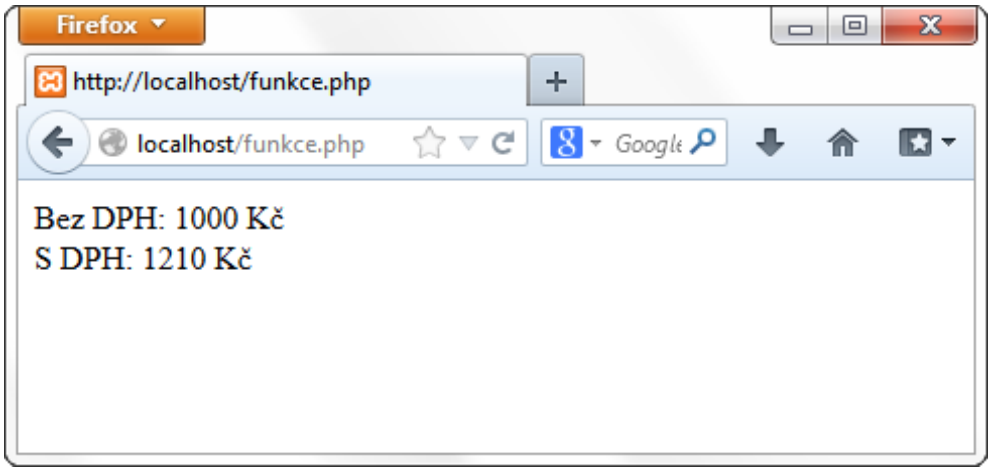
Výpis 7.4 Předávání parametru odkazem.



```
<?php
  function pricti_dph(&$x) {
    $x = $x + $x * 0.21;
  }
```

```
  $cena = 1000;
  echo "Bez DPH: " . $cena . " Kč<br>";
  pricti_dph($cena);
  echo "S DPH: " . $cena . " Kč<br>";
?>
```

Na následujícím obrázku je zobrazen výstup v prohlížeči. Vidíme, že hodnota proměnné `$cena` se po zavolání funkce `pricti_dph()` změnila a druhý výpis již vypíše původní hodnotu zvýšenou o DPH, tj. o 21 %.

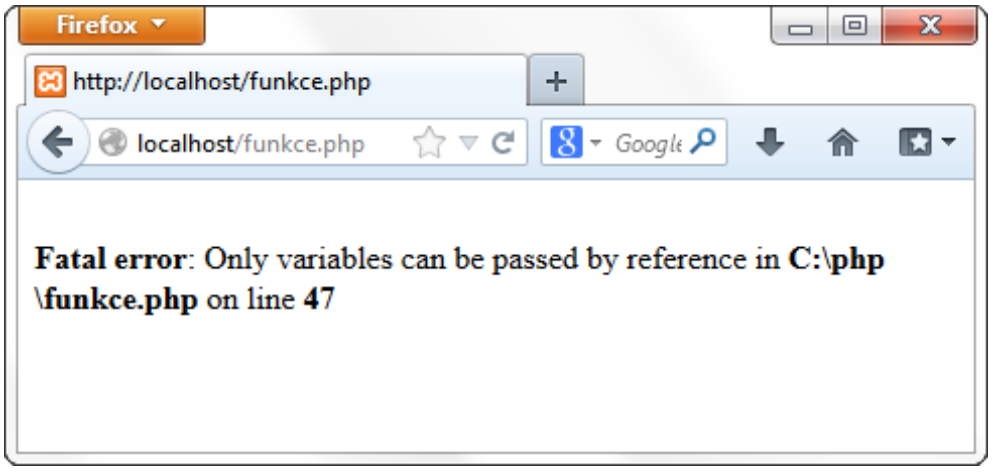


Obrázek 7.3 Předávání parametrů odkazem.

Při předávání parametrů hodnotou mohou být předány i konstanty nebo literály. Ve výpisu 7.2 je při volání funkce `soucet()` použito předání konstant, v tomto případě se jedná o hodnoty `5` a `3`. Při předávání parametrů odkazem ale musí být předána hodnota proměnné, předání konstanty způsobí chybu. Pokud bychom funkci `pricti_dph()`, která byla definována ve výpisu 7.4, zavolali následujícím způsobem, výsledkem bude chybová zpráva zobrazená na obrázku.

Výpis 7.5 Volání funkce, které způsobí chybu.

```
<?php
  pricti_dph(100);
?>
```



Obrázek 7.4 Chybová zpráva.

Jazyk PHP umožňuje u některých parametrů stanovit výchozí hodnoty. Přiřazení hodnoty se provádí v deklaraci funkce. Parametr, který má nastavenu výchozí hodnotu, je nepovinný při volání funkce. Pokud funkce obsahuje povinné i nepovinné parametry, je potřeba v deklaraci funkce nejprve uvést povinné parametry a teprve poté parametry nepovinné. Následující výpis ukazuje modifikaci funkce `pricti_dph()`, která obsahuje parametr s nastavenou výchozí hodnotou, tj. nepovinný parametr.

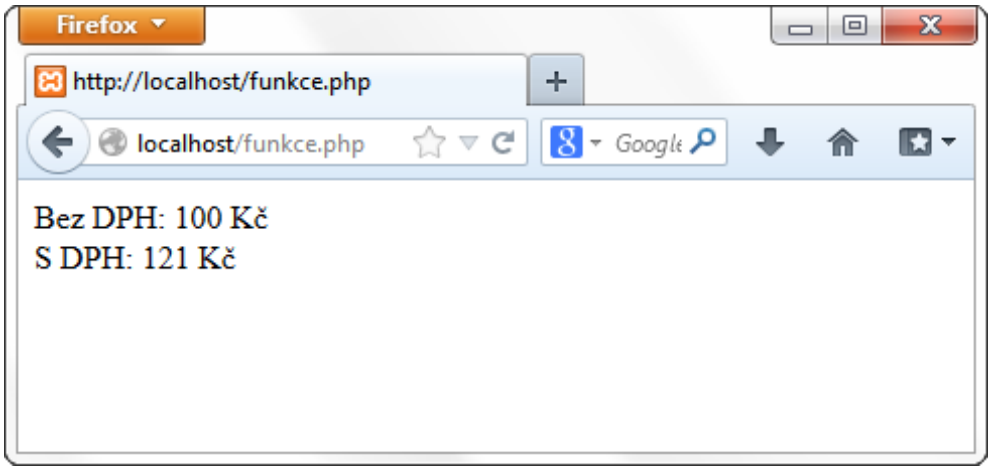
Výpis 7.6 Funkce s nepovinným parametrem.



```
<?php
  function pricti_dph(&$x, $sazba = 0.21) {
    $x = $x + $x * $sazba;
  }

  $cena = 100;
  echo "Bez DPH: " . $cena . " Kč<br>";
  pricti_dph($cena);
  echo "S DPH: " . $cena . " Kč<br>";
?>
```

Jak vidíme, druhý parametr funkce `pricti_dph()` má nastavenou výchozí hodnotu, touto hodnotou je `0.21`. Při volání funkce tedy stačí uvést pouze jeden parametr, v tomto případě je parametrem hodnota proměnné `$cena`. Výsledek uvedeného kódu zobrazený v prohlížeči zobrazuje následující obrázek.



Obrázek 7.5 Funkce s nepovinným parametrem.

7.3 Rozsah platnosti proměnných



S problematikou tvorby uživatelsky definovaných funkcí souvisí i rozsah platnosti proměnných, který určuje, ve kterých částech programu je proměnná přístupná. Proměnné vytvořené uvnitř těla funkce, jsou lokální pouze v rámci této funkce. Pokud ve funkce vytvoříme proměnnou se stejným identifikátorem, jako má některá proměnná mimo funkci, jedná se o novou proměnnou.

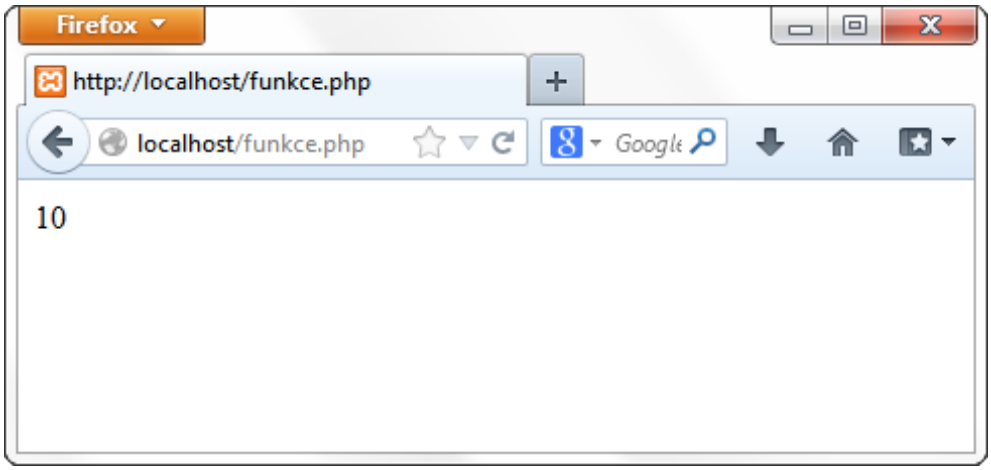
Výpis 7.7 Ukázka lokální proměnné v rámci funkce.



```
<?php
  function zmen_hodnotu() {
    $cislo = 20;
  }

  $cislo = 10;
  zmen_hodnotu();
  echo $cislo;
?>
```

Funkce `zmen_hodnotu()` přiřazuje do proměnné `$cislo` hodnotu `20`. Mimo funkci vytvoříme globální proměnnou, která se jmenuje také `$cislo` a přiřadíme jí do ní hodnotu `10`. Po zavolání funkce `zmen_hodnotu()` se ale hodnota proměnné `$cislo` nezmění, protože při volání funkce je vytvořená nová lokální proměnná. I když se obě proměnné jmenují stejně, jedná se o dvě různé proměnné. Výsledkem vykonání uvedeného kódu tedy bude vypsání čísla `10`, jak ukazuje následující obrázek.



Obrázek 7.6 Ukázka lokální proměnné.

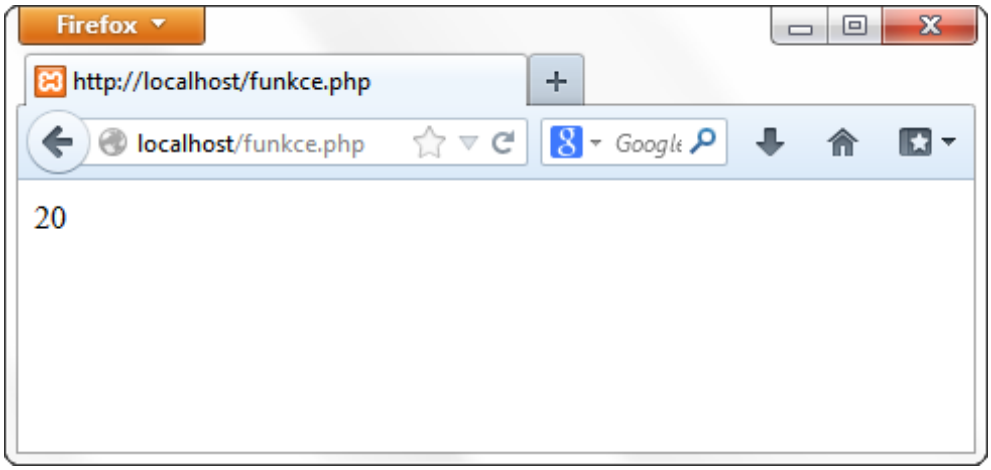
Pokud chceme v těle funkci přistupovat ke globální proměnné definované mimo funkci, je potřeba použít příkaz `global`. Tímto příkazem říkáme, že v rámci funkce nechceme vytvořit novou proměnnou, ale že chceme použít proměnnou, která je dostupná mimo tělo funkce. Následující výpis je velmi podobný výpisu 7.7, pouze je v těle funkce `zmen_hodnotu()` použitý příkaz `global`. Výsledkem provedení tohoto kódu je vypsání čísla `20`, jak opět ukazuje následující obrázek.

Výpis 7.8 Použití příkazu global.



```
<?php
  function zmen_hodnotu() {
    global $cislo;
    $cislo = 20;
  }

  $cislo = 10;
  zmen_hodnotu();
  echo $cislo;
?>
```



Obrázek 7.7 Použití příkazu global.

Alternativou použití příkazu `global` je použití superglobálního pole `$GLOBALS`, které obsahuje všechny globální proměnné programu (skriptu). V následujícím výpisu je uvedena ukázka použití tohoto pole ve funkci `zmen_hodnotu()`. U názvu proměnné v hranatých závorkách již není nutné používat znak `$` před názvem proměnné. V případě, že zbylý kód bude stejný jako v předchozím výpisu, vypíše se v prohlížeči číslo `20`.

Výpis 7.9 Ukázka použití superglobálního pole.

```
<?php
  function zmen_hodnotu() {
    $GLOBALS["cislo"] = 20;
  }
?>
```

Jazyk PHP umožňuje definovat v rámci funkcí statické proměnné, které si uchovávají svou předchozí hodnotu mezi jednotlivými voláními funkce. Statickou proměnnou vytvoříme pomocí klíčového slova `static`, které uvedeme v deklaraci proměnné.

7.4 Vnořené funkce a rekurze

Jazyk PHP umožňuje používat vnořené funkce, tzn. že v rámci jedné funkce vytvoříme další funkci. Všechny funkce, včetně funkcí vnořených do jiných funkcí, jsou v PHP globální, tzn. je možné je volat i mimo funkci, ve které jsou definovány. Používáním vnořených funkcí se ovšem snižuje přehlednost kódu a je pak složitější se v něm vyznat.

Pokud funkce volá sebe sama, označujeme ji jako rekurzivní funkci. Řešení problému je pak rozděleno na menší úlohy, které jsou poté několikrát

opakovány. U rekurze je potřeba dávat pozor, aby volání funkce skončilo a nedošlo k „zacyklení“, tzn. že provádění funkce se nikdy nezastaví. Příkladem problému, kdy je možné použít rekurzi, je výpočet n-té mocniny čísla nebo výpočet faktoriálu.

7.5 Předdefinované funkce



Jak již bylo uvedeno, jazyk PHP obsahuje velké množství předdefinovaných funkcí. V této studijní opoře jste se již seznámili s některými z nich. Nyní si uvedeme nejdůležitější skupiny funkcí, které jsou v PHP k dispozici. Kompletní seznam funkci je možné nalézt v dokumentaci k jazyku PHP.

Pro snadnější tvorbu aplikací v jazyce PHP jsou k dispozici předdefinované funkce, které je možné rozdělit do skupin podle toho, k čemu slouží. Mezi nejdůležitější skupiny operací při tvorbě webových aplikací patří práce s polem, s řetězci, se soubory, s kalendářním datem a časem, s databází, s obrázky, matematické operace a další.

Pro práci s polem jsou k dispozici například funkce pro procházení polí, vyhledávání hodnot v poli, řazení polí, získání indexů pole (pole v PHP může být asociativní, tj. indexy pole netvoří celá čísla, ale řetězce). Funkce pro práci s textovými řetězci umožňují například zjistit délku řetězce, rozdělit řetězec, najít v řetězci znak nebo znaky, nahradit v řetězci znak nebo znaky. Funkce pro práci se soubory umožňují vytvořit soubor (na serveru), otevřít soubor, číst ze souboru a zapisovat do něj. V případě kalendářního data můžeme zjistit aktuální datum a čas a dále pak zjistit jednotlivé části zadaného data a času – můžeme například zjistit, jaký je měsíc, den, rok apod. V případě práce s aktuálním datem a časem je potřeba myslet na to, že se jedná o datum a čas na serveru. Pokud je server umístěn v jiném časovém pásmu, aktuální datum a čas se samozřejmě liší od data a času v počítači. Pro práci s databází jsou k dispozici funkce pro připojení k databázi, provedení dotazu nad databází, procházení výsledků dotazu, vložení záznamu do tabulky apod. V jazyce PHP je možné vytvářet obrázky a vkládat je do webových stránek. Mezi základní operace patří vytvoření nebo smazání obrázku, změna velikosti obrázku, kreslení základních geometrických tvarů, práce s písmy nebo práce s barvami. V oblasti matematických funkcí jsou k dispozici například goniometrické funkce, absolutní hodnota, logaritmus, mocnina, odmocnina apod.

S některými předdefinovanými funkcemi se seznámíte dále v této studijní opoře.

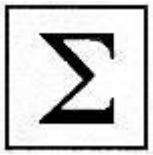
INVESTICE DO ROZVOJE VZDĚLÁVÁNÍ

Kontrolní otázky:



1. K čemu slouží funkce?
2. Jak vytvoříte funkci v jazyce PHP?
3. Jakým způsobem je možné předávat parametry funkcím v PHP?
4. Co je to vnořená funkce?
5. Co je to rekurzivní funkce?

Shrnutí obsahu kapitoly



V této kapitole jste s předdefinovanými funkcemi v PHP i s uživatelsky definovanými funkcemi. Uživatelsky definované funkce umožňují vytvářet modulární aplikace a snižují chybovost kódu. Definice funkce začíná klíčovým slovem `function`. Předávání parametrů v jazyce PHP je možné hodnotou a nebo odkazem. Jazyk PHP umožňuje také definici vnořených nebo rekurzivních funkcí. Pro usnadnění tvorby aplikací je v jazyce PHP k dispozici velké množství předdefinovaných funkcí. Nejdůležitějšími skupinami jsou funkce pro práci s polem, se soubory, s řetězci, s databází nebo matematické funkce.

8 Práce se soubory

V této kapitole se dozvíte:

- Jak pracovat s textovými soubory v PHP.
- Jak pracovat s adresáři v PHP.

Po jejím prostudování byste měli být schopni:

- Umět otevřít a zavřít soubor.
- Načíst data ze souboru, zapsat data do souboru, pohybovat se s kurzorem v rámci souboru.
- Umět pracovat s adresáři v PHP.

Klíčová slova této kapitoly:

soubor, čtení, zápis, pohyb kurzoru, manipulace se soubory, procházení adresáře.

Doba potřebná ke studiu: 4 hodiny



Průvodce studiem

Studium této kapitole není náročné, kapitola seznamuje čtenáře s předdefinovanými funkcemi pro práci se soubory a s adresáři. V kapitole se naučíte otevřít a zavřít textový soubor, načítat obsah souboru, zapisovat do souboru, manipulovat se souborem jako s celkem a nakonec se naučíte pracovat s adresáři.

Při vytváření webových aplikací je někdy potřeba ukládat si data získaná od uživatelů. V některých případech je zbytečné použít pro uložení dat databázi. Místo databáze můžeme k uložení dat použít textové soubory. Jazyk PHP umožňuje pracovat s textovými soubory – vytvářet nebo odstraňovat soubory, měnit jejich obsah, číst ze souborů. Kromě toho je v PHP možné pracovat s adresářovou strukturou – procházet obsah složky, vytvořit složku nebo ji přejmenovat.

8.1 Otevření a zavření souboru



Základní operací při práci se soubory je otevření souboru. Abychom mohli ze souboru číst nebo do něj zapisovat je potřeba ho nejprve otevřít. Po ukončení práce se souborem je samozřejmě potřeba soubor zavřít. K otevření souboru je v PHP k dispozici funkce `fopen()`, jejíž základní hlavička je ve tvaru:

```
resource fopen(string název_souboru, string režim)
```

Prvním parametrem je název souboru, který chceme otevřít. Může se jednat o lokální souboru v rámci adresářové struktury serveru, ale také o soubor otevřený v rámci internetu pomocí protokolu HTTP nebo FTP. Druhým

parametrem určujeme, v jakém režimu se má soubor otevřít – může se jednat o čtení ze souboru, zápis do souboru nebo přidávání do souboru. K určení požadovaného režimu slouží hodnoty uvedené v následující tabulce. Kromě dvou povinných parametrů je možné ve funkci `fopen()` použít ještě dva volitelné parametry. První volitelný parametr může být nastaven na hodnotu `true` nebo `false` a určuje, zda se zadaný soubor bude hledat i v adresáři zadaných v `include_path`. Druhý volitelný parametr určuje kontext, který souvisí s používáním proudů při práci se soubory.

Hodnota	Popis
`r`	Otevře soubor pouze pro čtení. Data budou zapsána na začátek souboru.
`r+`	Otevře soubor pro čtení a zápis. Data budou zapsána na začátek souboru.
`w`	Otevře soubor pouze pro zápis. Stávající obsah souboru bude vymazán. Pokud soubor neexistuje, pokusí se jej PHP vytvořit.
`w+`	Otevře soubor pro čtení a zápis. Stávající obsah souboru bude vymazán. Pokud soubor neexistuje, pokusí se jej PHP vytvořit.
`a`	Otevře soubor pouze pro přidávání. Data budou přidána na konec souboru. Pokud zadaný soubor neexistuje, pokusí se jej PHP vytvořit.
`a+`	Otevře soubor pro přidávání a čtení. Data budou přidána na konec souboru. Pokud zadaný soubor neexistuje, pokusí se jej PHP vytvořit.
`x`	Vytvoří a otevře soubor pouze pro zápis. Data budou zapsána na začátek souboru. Pokud zadaný soubor existuje, volání funkce `fopen()` selže a vrátí se hodnota `false`.
`x+`	Vytvoří a otevře soubor pro čtení a zápis. Data budou zapsána na začátek souboru. Pokud zadaný soubor existuje, volání funkce `fopen()` selže a vrátí se hodnota `false`.
`c`	Otevře soubor pouze pro zápis. Pokud zadaný soubor neexistuje, bude vytvořen. Pokud soubor existuje není jeho obsah smazán, ani nedojde k chybě při volání funkce `fopen()`. Data budou zapsána na začátek souboru.
`c+`	Otevře soubor pro čtení a zápis. Pokud zadaný soubor neexistuje, bude vytvořen. Pokud soubor existuje není jeho obsah smazán, ani nedojde k chybě při volání funkce `fopen()`. Data budou zapsána na začátek souboru.

V případě, že funkci `fopen()` použijeme k otevření jiného souboru v rámci Internetu (pomocí protokolu HTTP), můžeme tento soubor otevřít pouze pro čtení. K hodnotám určujícím režim otevření souboru můžeme přidat příznak `b`, pomocí kterého označíme otevíraný soubor jako binární (například se může jednat o obrázek).

Pokud otevření zadaného souboru proběhne v pořádku, vrátí funkce `fopen()` identifikátor souboru (ukazatel na soubor), pomocí něho provádíme další práci se souborem. V případě, že pokus o otevření souboru nebyl úspěšný, vrátí funkce `fopen()` hodnotu `0` (`false`).

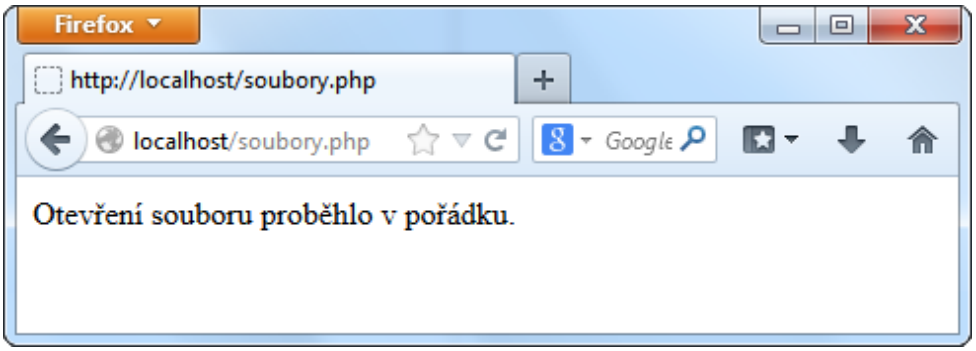
Následující výpis obsahuje ukázku použití funkce `fopen()`. Pokud soubor s názvem `aaa.txt` neexistuje, bude vytvořen a otevřen pro čtení i zápis.

Výpis 8.1 Otevření souboru.



```
<?php
  if ($soubor = fopen("aaa.txt", "w+")) {
    // soubor otevren, muzeme provest cteni nebo
     zapis
    echo "Otevření souboru proběhlo v pořádku.";
  } else {
    // chyba
    echo "Chyba při otevírání souboru.";
  }
?>
```

Název souboru je uveden relativně, takže adresářem pro hledání souboru bude adresář, ve kterém se nachází soubor `soubory.php` (v tomto případě je to kořenový adresář webového serveru). Pokud soubor neexistuje a má být vytvořen, je potřeba, aby byla kořenovému adresáři serveru nastavena přístupová práva pro vytváření souborů. Na obrázku, který následuju je zobrazeno okno prohlížeče pro provedení příkazů uvedených ve výpisu 8.1.



Obrázek 8.1 Otevření souboru.

INVESTICE DO ROZVOJE VZDĚLÁVÁNÍ

Po ukončení práce se souborem je potřeba soubor zavřít. K zavření souboru slouží funkce `fclose()`, jejíž hlavička vypadá následovně:



```
bool fclose(resource soubor)
```

Parametrem funkce je ukazatel na soubor, který chceme zavřít. Pokud uzavření souboru proběhne úspěšně, vrátí funkce `true`, jinak vrátí hodnotu `false`.

Následující výpis ukazuje použití funkce `fclose()`, předpokládá se, že v proměnné `soubor` je uložen platný odkaz na existující soubor.

Výpis 8.2 Zavření souboru.



```
<?php
  if (fclose($soubor)) {
    echo "Zavření souboru proběhlo v pořádku.";
  } else {
    echo "Chyba při zavírání souboru.";
  }
?>
```

V některých případech může být užitečné nejprve otestovat, zda požadovaný soubor existuje. K testování existence souboru slouží funkce `file_exists()`, jejímž parametrem je název souboru. Pokud soubor existuje, funkce vrátí `true`, jinak vrátí `false`.

8.2 Práce s obsahem souboru

Po otevření souboru ke čtení je kurzor v souboru nastaven na začátek souboru. Ke zjištění, zda je kurzor na konci souboru slouží funkce `feof()`, která má jeden parametr – platný ukazatel na otevřený soubor. Funkce vrací `true`, pokud je kurzor na konci souboru, jinak vrací `false`. Ke zjištění velikosti souboru slouží funkce `filesize()`, jejímž parametrem je název souboru. Funkce vrací velikost souboru v bytech.

8.2.1 Čtení ze souboru



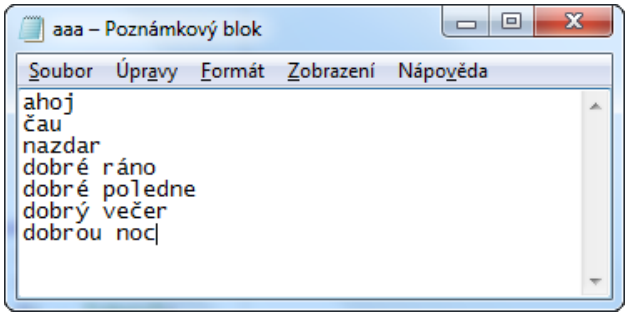
Čtení ze souborů můžeme provádět několika způsoby. Obsah souboru můžeme načíst celý najednou a dále s ním pracovat a nebo můžeme obsah souboru číst postupně – můžeme načítat řetězce o zadané délce, můžeme načíst jeden znak nebo jeden řádek.

K načtení a vytištění celého obsahu souboru slouží funkce `fpassthru()`. Parametrem funkce je platný ukazatel na otevřený soubor. Funkce načte obsah

souboru a odešle jej na výstupní proud (např. do okna prohlížeče). Pokud je čtení úspěšné, vrátí funkce počet načtených znaků. Pokud dojde při čtení souboru k chybě, funkce vrátí hodnotu `false`.

Druhou možností načtení celého obsahu souboru najednou je použití funkce `file()`, která má jako parametr název souboru. Funkce `file()` načte celý obsah souboru a uloží jej do pole. Prvky pole tvoří jednotlivé řádky souboru, počet prvků pole je tedy roven počtu řádků souboru. Kromě názvu souboru má funkce ještě dva nepovinné parametry, kterými se ale nebudeme v tomto textu zabývat.

Ve zbývající části kapitoly budeme pracovat se souborem `aaa.txt`. Soubor otevřený v aplikaci Poznámkový blok je zobrazený na následujícím obrázku.



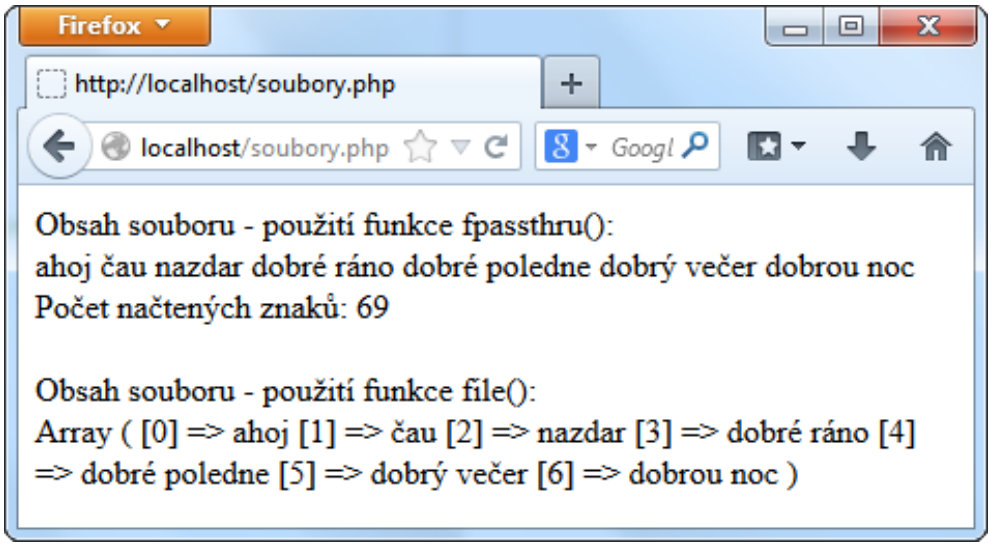
Obrázek 8.2 Obsah souboru aaa.txt.

Následující výpis ukazuje použití funkcí `fpassthru()` a `file()`. Obrázek pod výpisem pak zobrazuje výsledek zobrazený v prohlížeči.

Výpis 8.3 Použití funkcí fpassthru() a file().



```
<?php
  if ($soubor = fopen("aaa.txt", "r")) {
    echo "Obsah souboru - použití funkce
     fpassthru():<br>";
    $znaky = fpassthru($soubor);
    echo "<br>Počet načtených znaků: $znaky<br><br>";
    fclose($soubor);
  } else {
    echo "Chyba při otevírání souboru.";
  }
  $pole = file("aaa.txt");
  echo "Obsah souboru - použití funkce file():<br>";
  print_r($pole);
?>
```



Obrázek 8.3 Použití funkcí fpassthru() a file().

Při zpracování obsahu souboru po částech můžeme načíst jeden znak nebo zadaný počet znaků. Pro načtení jednoho znaku ze souboru slouží funkce `fgetc()`, jejímž parametrem je platný ukazatel na otevřený soubor. Funkce vrací řetězec o délce jeden znak.

Jako alternativu můžeme použít funkci `fgets()`, která ze souboru načte jeden řádek textu. Funkce má jeden povinný a jeden volitelný parametr. Povinným parametrem je platný ukazatel na otevřený soubor, volitelným parametrem je celé číslo určující počet načtených znaků. Hlavička funkce je ve tvaru

```
string fgets(resource soubor, int délka)
```

Pokud nebyl zadán parametr `délka`, načte se text do konce aktuálního řádku (případně do konce souboru). Pokud byl zadán parametr `délka`, je ze souboru načteno (`délka - 1`) znaků. V případě, že je před načtením určeného počtu znaků dosaženo konce řádku nebo konce souboru, vrátí funkce menší počet znaků. Další velmi podobnou funkcí je funkce `fgetss()`, která má stejné parametry jako funkce `fgets()`. Rozdíl mezi těmito funkcemi je v tom, že funkce `fgetss()` odstraní z načteného řetězce HTML značky. Pokud zadáme parametr `délka`, jsou tyto značky započítány do délky načteného řetězce.

Poslední funkcí pro čtení obsahu souboru, kterou si zde uvedeme, je funkce `fread()`. Tato funkce má stejné parametry jako funkce `fgets()`, tj. ukazatel na otevřený soubor a počet znaků, které se mají načíst. Na rozdíl od funkce `fgets()` jsou oba parametry funkce `fread()` povinné. Dalším rozdílem je, že ze souboru bude načten počet znaků, který je roven parametru

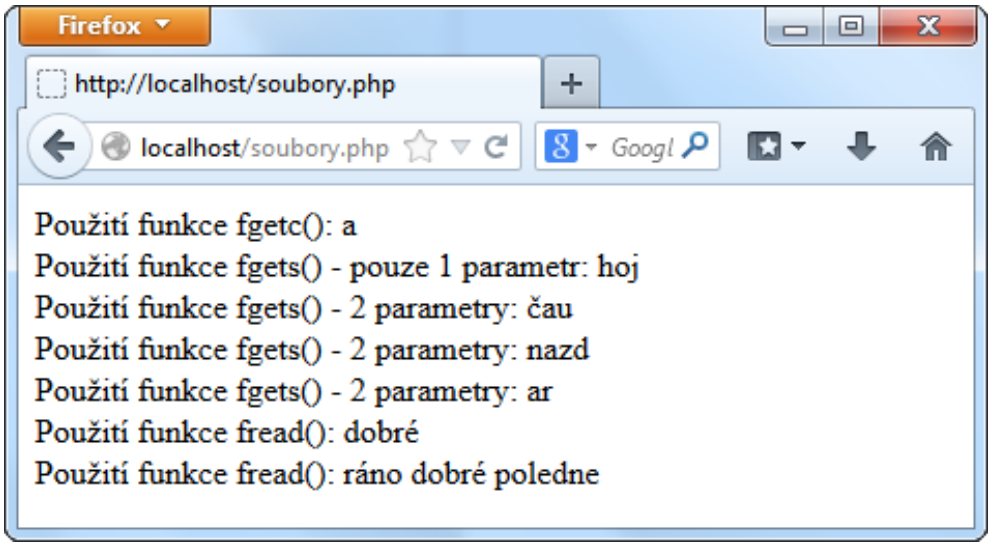
délka a že čtení znaků neskončí při dosažení konce řádku. Samozřejmě ale skončí po dosažení konce souboru.

Následující výpis zobrazuje ukázku použití uvedených funkcí, obrázek pod výpisem pak výsledek zobrazený v okně prohlížeče.

Výpis 8.4 Použití funkcí fgetc(), fgets() a fread().



```
<?php
  if ($soubor = fopen("aaa.txt", "r")) {
    echo "Použití funkce fgetc(): " . fgetc($soubor)
     . "<br>";
    echo "Použití funkce fgets() - pouze 1 parametr:
     " . fgets($soubor) . "<br>";
    echo "Použití funkce fgets() - 2 parametry: " .
     fgets($soubor, 10) . "<br>";
    echo "Použití funkce fgets() - 2 parametry: " .
     fgets($soubor, 5) . "<br>";
    echo "Použití funkce fgets() - 2 parametry: " .
     fgets($soubor, 5) . "<br>";
    echo "Použití funkce fread(): " . fread($soubor,
     5) . "<br>";
    echo "Použití funkce fread(): " . fread($soubor,
     20) . "<br>";
    fclose($soubor);
  } else {
    echo "Chyba při otevírání souboru.";
  }
?>
```



Obrázek 8.4 Použití funkcí fgetc(), fgets() a fread().

INVESTICE DO ROZVOJE VZDĚLÁVÁNÍ

8.2.2 Zápis do souborů



Pro zápis textu do souboru slouží funkce `fwrite()`, jejíž hlavička má následující tvar:

```
int fwrite(resource soubor, string text, int délka)
```

Prvním parametrem funkce je platný ukazatel na otevřený soubor. Druhým parametrem je řetězec, který chceme do souboru zapsat. Poslední parametr `délka` je volitelný a udává počet znaků, které se mají zapsat do souboru. Pokud není tento parametr zadán, zapíše se celý řetězec uvedený v parametru `text`. V případě, že zápis do souboru proběhne úspěšně, vrátí funkce `fwrite()` počet zapsaných bytů. V případě neúspěchu vrátí `false`.

Při zápisu do souboru je potřeba mít brát zřetel na to, v jakém režimu byl otevřen cílový soubor. Na použitém režimu závisí to, kam se data do souboru zapíší.

Druhou funkcí, kterou můžeme použít pro zápis do souboru, je funkce `fputs()`. Tato funkce je ale pouze aliasem funkce `fwrite()`, tj. má stejné parametry i návratovou hodnotu a liší se pouze v názvu.

Následující výpis zobrazuje ukázku použití funkce `fwrite()`.

Výpis 8.5 Ukázka použití funkcí fwrite() a fputs().



```
<?php
  if ($soubor = fopen("bbb.txt", "w")) {
    $pocet = fwrite($soubor, "První text");
    echo "Počet zapsaných znaků: $pocet<br>";
    $pocet = fputs($soubor, "Druhý text");
    echo "Počet zapsaných znaků: $pocet<br>";
    $pocet = fwrite($soubor, "Třetí text", 5);
    echo "Počet zapsaných znaků: $pocet<br>";
    fclose($soubor);
  } else {
    echo "Chyba při otevírání souboru.";
  }
?>
```

8.2.3 Pohyb uvnitř souborů



Při práci s obsahem souboru je někdy potřeba přesunout pozici kurzoru v souboru na jiné místo, na kterém má být provedeno čtení nebo zápis. Pro přesun na začátek souboru je určena funkce `rewind()`, jejímž jediným

parametrem je platný ukazatel na otevřený soubor. Pokud přesun na začátek souboru proběhne úspěšně, funkce vrátí hodnotu `true`, jinak vrátí `false`. V případě, že soubor byl otevřen v režimu přidávání (tj. režim byl „`a`“ nebo „`a+`“), při zápisu se vždy data zapíší na konec souboru, a to bez ohledu na to, kam jsme přesunuli kurzor v rámci souboru.

Pro zjištění aktuální pozice kurzoru v souboru je určena funkce `ftell()`, která má jako parametr platný ukazatel na otevřený soubor. Pokud při zjišťování pozice kurzoru nastane chyba, vrátí funkce hodnotu `false`. V opačném případě vrátí celé číslo udávající pozici kurzoru v rámci souboru.

Funkce `fseek()` slouží pro umístění kurzoru v rámci souboru na konkrétní pozici. Pokud umístění kurzoru proběhne úspěšně, vrací funkce hodnotu `0`, jinak vrací hodnotu `-1`. Funkce `fseek()` má tři parametry. Prvním parametrem je platný ukazatel na otevřený soubor, druhým parametrem je pozice (počet znaků od začátku souboru) a třetím parametrem je nastavení, jak se bude počítat výsledná pozice. První dva parametry jsou povinné, třetí parametr je nepovinný.

Poslední funkcí, která souvisí s pohybem kurzoru v rámci souboru, je funkce `foef()`, která již byla zmíněna v úvodu této kapitoly. Funkce zjišťuje, zda se kurzor nachází na konci souboru.

8.3 Manipulace se soubory



Při práci se soubory nemusíme pracovat pouze s obsahem souboru, ale můžeme pracovat i se souborem jako s celkem. Jazyk PHP umožňuje vytvořit kopii souboru, přejmenovat soubor a odstranit soubor.

Pro zkopírování souboru je určena funkce `copy()`, která má dva parametry. Prvním parametrem je název zdrojového souboru, resp. cesta k tomuto souboru. Druhým parametrem je název cílového souboru. Pokud vytvoření kopie souboru proběhne úspěšně, funkce vrátí hodnotu `true`. V opačném případě vrátí hodnotu `false`.

Přejmenování souborů je možné provést pomocí funkce `rename()`. Parametry této funkce jsou starý název souboru a nový název souboru. Pokud jako nový název zadáme název již existujícího souboru, bude tento soubor přepsán. Stejně jako předchozí funkce, i funkce `rename()` vrací v případě úspěchu hodnotu `true`, jinak vrací `false`.

Pokud chceme soubor odstranit, použijeme k tomuto účelu funkci `unlink()`. Parametrem funkce je název souboru, který chceme odstranit. Pokud odstranění souboru proběhne úspěšně, funkce vrací `true`, jinak vrací `false`.

Kromě manipulace se soubory je někdy potřeba zjišťovat vlastnosti souborů. K otestování, zda zadaný soubor nebo adresář vůbec existuje slouží funkce `file_exists()`. Ke zjištění velikosti souboru slouží funkce `filesize()`. Obě funkce již byly zmíněny v této kapitole. Další vlastností souboru, kterou můžeme zjistit, je typ souboru. K určení typu souboru je určena funkce `filetype()`, která má jako parametr název souboru a vrací textový řetězec označující typ souboru. V případě úspěchu vrací funkce jednu z osmi předdefinovaných hodnot, v případě neúspěchu vrací `false`. Návratové hodnoty označují o jaký typ souboru se jedná. Hodnota `file` určuje, že se jedná o normální soubor, hodnota `dir` určuje, že se jedná o adresář, hodnota `unknown` určuje, že typ souboru se nepodařilo zjistit. Zbylé hodnoty určují speciální typy souborů.

Ke zjištění, zda je zadaný soubor určitého typu pak slouží sada logických funkcí, které vrací `true` v případě, že zadaný soubor je určitého typu, nebo `false`, pokud je jiného typu. Všechny funkce mají jeden parametr, a tím je název souboru. Funkce `is_dir()` zjišťuje, zda se jedná o adresář. Funkce `is_file()` zjišťuje, zda se jedná o normální soubor. Funkce `is_executable()` testuje, zda se jedná o spustitelný soubor. Kromě toho je možné zjistit, zda lze zadaný soubor nebo adresář číst a nebo do něj zapisovat. K tomuto účelu slouží funkce `is_readable()` – čtení a `is_writable()` – zápis.

Výpis 8.6 Ukázka manipulace se soubory.

```
<?php
  // vytvoreni kopie souboru ve stejnem adresari
  copy("./aaa.txt", "./aaa-kopie.txt");
  // vytvoreni kopie souboru v jinem adresari –
   adresar musí existovat
  copy("./aaa.txt", "./pokus/aaa-kopie.txt");
  // prejmenovani souboru
  rename("./pokus/aaa-kopie.txt", "./pokus/aaa-
   nove.txt");
  // odstraneni souboru
  unlink("aaa-kopie.txt");
  // zjisteni typu souboru
  echo filetype("aaa.txt");
```



```
  echo is_dir("./pokus");
  echo is_file("aaa.txt");
  echo is_readable("aaa.txt");
  echo is_writable("aaa.txt");
?>
```

8.4 Práce se adresáři



Jazyk PHP umožňuje pracovat také s adresáři. Můžeme nastavit aktuální adresář, procházet obsah adresáře, vytvořit nebo odstranit adresář.

Při tvorbě aplikací v jazyce PHP je vždy nastaven aktuální pracovní adresář. ke zjištění názvu aktuálního adresáře slouží funkce `getcwd()`. Tato funkce nemá žádné parametry a vrací textový řetězec představující aktuální pracovní adresář. V případě chyby funkce vrací `false`. K nastavení aktuálního adresáře slouží funkce `chdir()`. Parametrem funkce řetězec určující cestu k adresáři. V případě úspěchu funkce vrací hodnotu `true`, jinak vrací `false`. Název nového aktuálního adresáře může být uveden absolutně (celá cesta k adresáři) nebo relativně k právě aktuálnímu adresáři.

Procházení obsahu adresáře je možné dvěma způsoby. Prvním způsobem je použití funkcí pro procházení adresáře, druhým způsobem je použití třídy `Directory`. Oba způsoby jsou velmi podobné.

Při použití funkcí pro procházení adresářů musíme nejprve otevřít požadovaný adresář. K otevření adresáře je určena funkce `opendir()`, jejímž parametrem je textový řetězec představující cestu k adresáři. Cesta může být zadána absolutně nebo relativně. Funkce vrací ukazatel na otevřený adresář. K procházení položek otevřeného adresáře slouží funkce `readdir()`, která vrací název následující položky v adresáři. Parametrem funkce je platný ukazatel na otevřený adresář. Funkce `readdir()` se pohybuje v adresáři od začátku do konce. Pokud se potřebujeme vrátit na začátek adresáře, použijeme funkci `rewinddir()`. Parametrem funkce je platný ukazatel na otevřený adresář. Funkce nevrací žádnou hodnotu (resp. vrací `void`). Po ukončení práce s adresářem je potřeba zavřít otevřený adresář, a to pomocí funkce `closedir()`. Parametrem je opět platný ukazatel na otevřený adresář a návratovým typem funkce je `void`.

Druhým způsobem procházení adresáře je použití třídy `Directory`. Funkce `dir()`, jejímž parametrem je textový řetězec představující cestu k adresáři, vrací instanci třídy `Directory`. Třída `Directory` obsahuje tři metody,

které slouží k pohybu v adresáři. Metoda `read()` má stejnou funkčnost jako funkce `readdir()`, metoda `rewind()` je ekvivalentem funkce `rewinddir()` a metoda `close()` je ekvivalentní k metodě `closedir()`. Kromě uvedených metod obsahuje třída `Directory` ještě dvě členské proměnné – proměnná `$path` obsahuje řetězec představující otevřený adresář, proměnná `$handle` obsahuje ukazatel na otevřený adresář.

Vytvoření nového adresáře je možné pomocí metody `mkdir()`. Parametrem je textový řetězec představující nový adresář. Odstranění adresáře je možné pomocí metody `rmdir()`, jejímž parametrem je opět textový řetězec představující adresář. Obě metody vrací `true`, pokud je operace úspěšná, nebo `false` v případě neúspěchu.

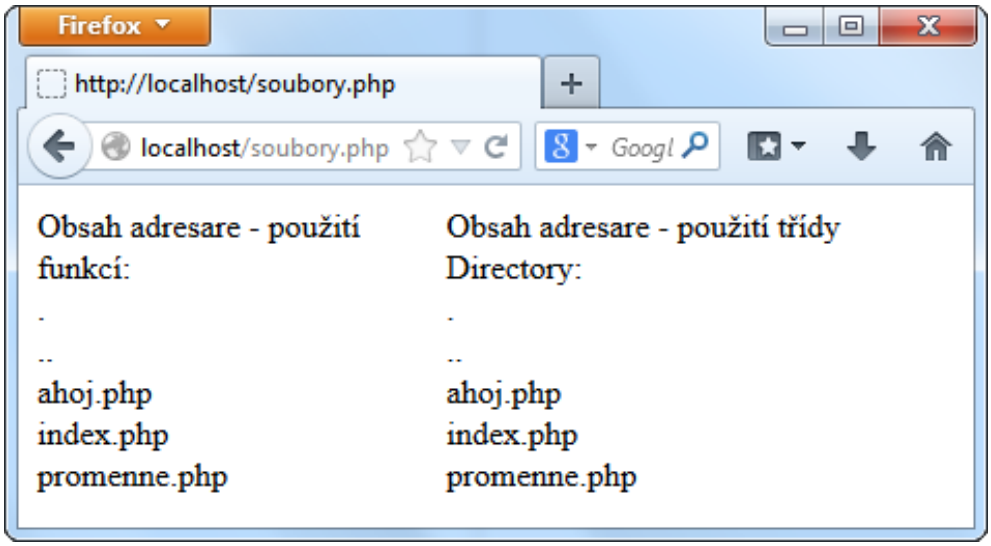
Následující výpis zobrazuje ukázku práce s adresáři, obrázek pod výpisem pak výstup v prohlížeči.

Výpis 8.7 Ukázka práce s adresáři.

```
<?php
  echo '<table border="0" cellspacing="0"><tr><td>';
  $adresar = opendir("pokus");
  echo "Obsah adresare - použití funkcí:<br>";
  while (false !== ($zaznam = readdir($adresar))) {
    echo "$zaznam<br>";
  }
  closedir($adresar);

  echo '</td><td>';
  // pouziti tridy Directory
  $adresar = dir("pokus");
  echo "Obsah adresare - použití třídy
   Directory:<br>";
  while (false !== ($zaznam = $adresar->read())) {
    echo "$zaznam<br>";
  }
  $adresar->close();
  echo '</td></tr></table>';
?>
```



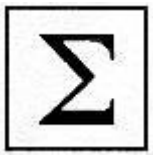


Obrázek 8.5 Výpis obsahu adresáře.



Kontrolní otázky:

1. Jaké parametry má funkce `fopen()` a co je jejím návratovým typem?
2. Jaký je rozdíl mezi funkcemi `fgetc()`, `fgets()` a `fread()`?
3. Jaký je rozdíl mezi funkcemi `fgets()` a `fgetss()`?
4. Jakými způsoby je možné vypsat obsah konkrétního adresáře?



Shrnutí obsahu kapitoly

V této kapitole jste s předdefinovanými funkcemi v PHP, které slouží pro práci se soubory. K otevření souboru slouží funkce `fopen()`, k jeho zavření pak `fclose()`. Čtení ze souboru je možné provést pomocí funkcí `fgetc()`, `fgets()`, `fgetss()` nebo `fread()`. Zápis do souboru je možný pomocí funkce `fwrite()`. Funkce `copy()`, `rename()` a `unlink()` slouží ke zkopírování, přejmenování a odstranění souboru. Výpis obsahu adresáře je možný dvěma způsoby – pomocí funkcí nebo pomocí třídy `Directory`.

9 Práce s řetězci

V této kapitole se dozvíte:

- Jakým způsobem je možné vytisknout řetězec v PHP.
- Jak vyhledávat v řetězci.
- Jak upravovat řetězce.
- Jak spojovat a rozdělovat řetězce.

Po jejím prostudování byste měli být schopni:

- Umět vytisknout řetězec a vytvořit formátovaný výstup.
- Umět vyhledávat v řetězci.
- Umět upravovat řetězce pomocí předdefinovaných funkcí.
- Umět rozdělit řetězec na pole a spojit pole do řetězce.

Klíčová slova této kapitoly:

řetězce, echo, print, printf, strlen, strpos, htmlspecialchars, htmlentities, implode, explode, str_split.

Doba potřebná ke studiu: 4 hodiny

Průvodce studiem



Studium této kapitoly není náročné. Jedná se o přehledovou kapitolu, ve které jsou představeny vybrané funkce pro práci s řetězci. Jejich výčet není úplný, jsou představeny pouze některé z nich. V kapitole se postupně seznámíte se způsoby vypisování řetězců, s vyhledáváním v řetězci, s úpravou řetězců a s rozdělováním a spojováním řetězců.

Prakticky každá webová aplikace pracuje s řetězci. Při práci s aplikací je potřeba zpracovávat vstupy od uživatele nebo generovat výstupy. V některých případech se jedná o číselné údaje, ale častěji se jedná o řetězce znaků. Jazyk PHP obsahuje mnoho funkcí pro práci s řetězci, které usnadňují tvorbu aplikací. V této kapitole se seznámíme s nejdůležitějšími z nich.

9.1 Vypsání řetězce



Základní operací s jakýmkoliv řetězcem je jeho vypsání na obrazovku, resp. do okna webového prohlížeče. Doposud jsme se setkali s funkcí `echo`, která vypisuje řetězec. Ve skutečnosti není `echo` funkcí, ale jedná se o konstrukt jazyka PHP. Proto při volání nemusí být použity závorky. Konstrukt `echo` se nechová jako funkce, takže nemůže vždy být použit jako funkce.

Konstrukt `echo` vypisuje jak konstanty (řetězcové nebo číselné), tak obsah proměnných. Pro výpis řetězcových konstant je možné použít jak klasické, tak jednoduché uvozovky. Spojování řetězců nebo obsahu proměnných do jednoho výpisu je možné pomocí operátoru **.** (tečka) nebo **,** (čárka). Pro výpis

speciálních znaků (uvozovky, obrácené lomítko atd.) je potřeba použít escape sekvenci – přidáním obráceného lomítka před speciální znak.

Kromě konstruktu `echo` je možné k výpisu řetězce použít konstrukt `print`. Opět se nejedná o klasickou funkci, ale o jazykový konstrukt, takže při volání nemusí být uvedeny závorky. Hlavní rozdíl mezi konstrukty `echo` a `print` je v tom, že `print` vždy vrací hodnotu `1`. Další rozdíl je v tom, že pokud chceme ve výpisu pomocí konstruktu `print` spojit více řetězců, můžeme použít pouze operátor `.` (tečka).

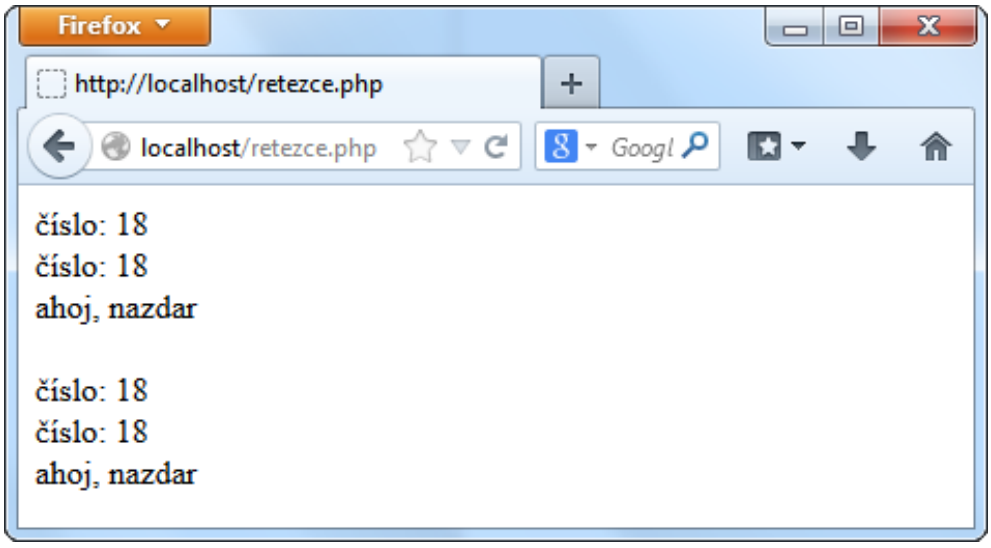
Následující výpis ukazuje použití konstruktů `echo` a `print`. Obrázek pod výpisem pak zobrazuje výstup v prohlížeči.

Výpis 9.1 Ukázka použití konstruktů echo a print.



```
<?php
  $cislo = 18;
  echo "číslo: $cislo<br>";
  echo "číslo: " . $cislo . "<br>";
  echo "ahoj" , ", " , "nazdar<br><br>";

  print "číslo: $cislo<br>";
  print "číslo: " . $cislo . "<br>";
  print "ahoj" . ", " . "nazdar";
?>
```



Obrázek 9.1 Použití konstruktů echo a print.

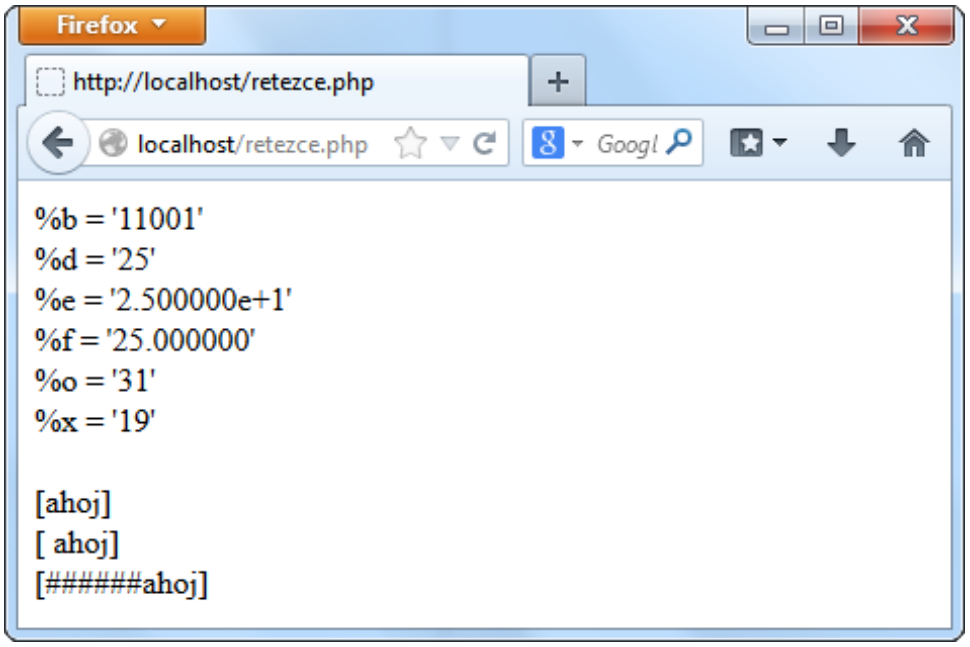
Další funkcí, kterou je možné použít k výpisu řetězců do okna prohlížeče, je funkce `printf()`. Tato funkce slouží k vytvoření formátovaného výstupu a vrací celé číslo – délku vypsaného řetězce. Počet parametrů této funkce není

pevně dán, liší podle toho, jak vypadá vstupní řetězec. Prvním parametrem funkce je vstupní řetězec, který se má vypsat. Tento řetězec může obsahovat formátovací řetězce, které určují, jak se vypíše obsah proměnných. Formátovací řetězce vždy začínají znakem `%` (procento), za kterým následují znaky určující způsob výpisu. Můžeme například určit délku vypsaného řetězce, počet vypisovaných desetinných míst apod. Počet použitých formátovacích řetězců určuje počet zbývajících parametrů funkce `printf()`. Jako další parametry funkce je potřeba uvést názvy proměnných nebo konstanty, na které se mají aplikovat formátovací pravidla. Kompletní popis funkce `printf()` je možné nalézt v dokumentaci k jazyku PHP, zde si ovšem uvedeme několik příkladů.

```
<?php
  $a = 25;
  // binarni reprezentace
  printf("%%b = '%b'<br>", $a);
  // klasicka celociselna reprezentace
  printf("%%d = '%d'<br>", $a);
  // vedecka reprezentace
  printf("%%e = '%e'<br>", $a);
  // reprezentace jako desetinne cislo
  printf("%%f = '%f'<br>", $a);
  // osmickova soustava
  printf("%%o = '%o'<br>", $a);
  // hexadecimalni soustava
  printf("%%x = '%x'<br><br>", $a);

  $b = "ahoj";
  // klasicky vypis
  printf("[%s]<br>", $b);
  // vypis o delce 10 znaku, zarovnani vpravo
  printf("[%10s]<br>", $b);
  // vypis o delce 10 znaku, zarovnani vpravo,
   doplneni znaku # na volna mista
  printf("[%'#10s]<br>", $b);
?>
```





Obrázek 9.2 Ukázka použití funkce printf().

9.2 Procházení řetězců, vyhledávání v řetězcích



Při tvorbě webových aplikací nepotřebujeme řetězce pouze vypisovat, ale potřebujeme s nimi také dále pracovat. V případě, že chceme přistupovat k jednotlivým znakům řetězce, můžeme použít dva způsoby. Oba jsou velmi podobné. S řetězci v jazyce PHP můžeme pracovat jako s poli. Použitím hranatých závorek `[ ]` a uvedením indexu požadovaného znaku získáme jeden znak z řetězce. První znak v řetězci má index `0`, druhý index `1` atd. Druhou možností přístupu k jednotlivým znakům řetězce je použití složených závorek `{ }`. I v tomto případě je v závorkách uveden index požadovaného znaku.

Funkce `strlen()` zjišťuje délku řetězce. Parametrem funkce je řetězec, u kterého chceme zjistit délku (počet znaků). Funkce vrací celé číslo udávají počet znaků řetězce. Pokud je řetězec prázdný, vrátí funkce hodnotu `0`.

Pro zjištění, zda se nějaký řetězec nebo znak nachází v jiném řetězci, slouží funkce `strpos()`. Funkce `strpos()` vrací pozici prvního výskytu hledaného řetězce v jiném řetězci. Pokud není hledaný řetězec nalezen, funkce vrátí hodnotu `false`. Funkce `strpos()` má 2 povinné parametry. Prvním parametrem je řetězec, ve kterém chceme hledat. Druhým parametrem je hledaný řetězec. Třetím, volitelným, parametrem je index, na kterém chceme zahájit hledání. Jako třetí parametr musí být zadáno nezáporné číslo. Pokud není třetí parametr zadán, hledá se od začátku řetězce (od indexu 0). Funkce `strpos()` je case senzitivní, tj. při hledání řetězce se rozlišují velká a malá

písmena. Funkce `stripos()` má stejné parametry i návratové hodnoty jako funkce `strpos()`, ale rozdíl je v tom, že funkce `stripos()` není case senzitivní, tj. při hledání se nerozlišují velká a malá písmena.

Dalšími velmi podobnými funkcemi jsou `strrpos()` a `strripos()`. Mají stejné parametry i návratové hodnoty jako funkce `strpos()`, ale při hledání zjišťují pozici posledního výskytu hledaného řetězce. Funkce `strrpos()` je case senzitivní, funkce `strripos()` je case insenzitivní. Třetí parametr těchto funkcí může být záporné číslo. V případě, že zadáme záporné číslo, odečte se od konce řetězce zadaný počet znaků a hledání probíhá směrem k začátku řetězce. Pokud tedy jako třetí parametr zadáme hodnotu -2, začne hledání na předposledním znaku řetězce.

Následující výpis ukazuje použití výše uvedených funkcí. Obrázek pod výpisem pak zobrazuje výsledek v okně prohlížeče.

Výpis 9.2 Ukázka použití funkcí strlen(), strpos() a strrpos().

```
<?php
  $text = "Dobrý den! Jak se máte?";
  echo "Třetí znak: " . $text[1] . "<br>";
  echo "První znak: " . $text{0} . "<br>";
  echo "Počet znaků: " . strlen($text) . "<br>";

  // case senzitivni hledani, retezec DEN neni
   nalezen
  $pozice = strpos($text, "DEN");
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'DEN': $pozice<br>";

  // case insenzitivni hledani, retezec DEN je
   nalezen
  $pozice = stripos($text, "DEN");
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'DEN': $pozice<br>";

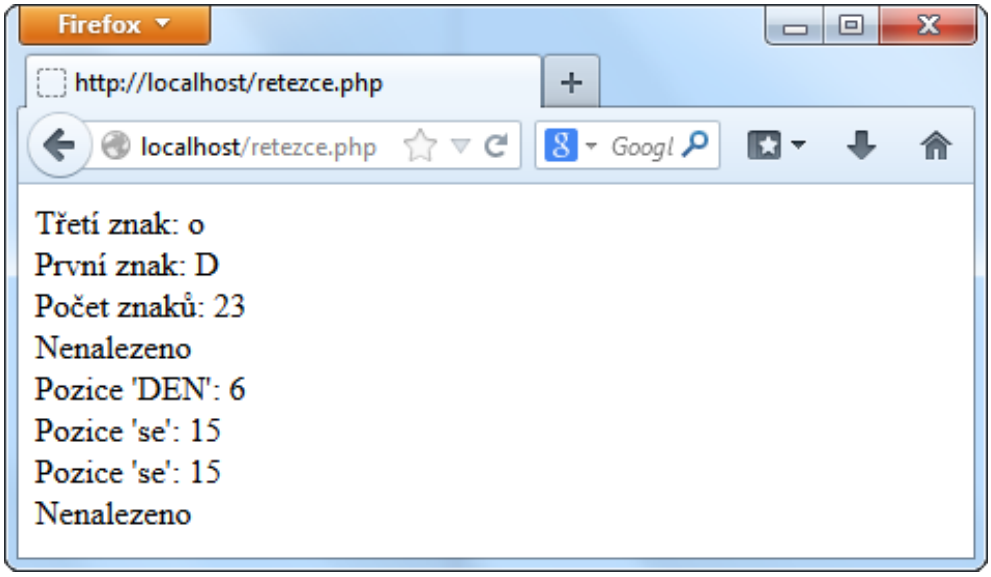
  // hledani od indexu 10 (od 11. znaku)
  $pozice = stripos($text, "se", 10);
  if ($pozice === false)
    echo "Nenalezeno<br>";
```



```
  else
    echo "Pozice 'se': $pozice<br>";

  // hledani posledniho vyskytu
  $pozice = strrpos($text, "se");
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'se': $pozice<br>";

  // hledani od konce retezce smerem k zacatku
   retezce
  $pozice = strrpos($text, "se", -10);
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'se': $pozice<br>";
?>
```



Obrázek 9.3 Použití funkcí strlen(), strpos() a strrpos().

9.3 Úprava řetězců



Další operací, kterou můžeme provádět s řetězci, je jejich úprava. Pod pojmem úprava řetězce si můžeme představit mnoho různých činností. Jako první si uvedeme situaci, kdy chceme některé znaky v řetězci nahradit jinými znaky. K nahrazení částí řetězce jiným řetězcem slouží funkce `str_replace()`. Tato funkce má tři povinné parametry. Prvním parametrem je řetězec, který bude v původním řetězci nahrazen jiným řetězcem. Druhým parametrem je řetězec, který nahradí řetězec uvedený jako první parametr. Třetím parametrem je řetězec, ve kterém se úpravy provádějí. Změny se provedou ve všech

výskytech nahrazovaného řetězce. Funkce vrací nový řetězec po úpravách. Na pozici všech třech povinných parametrů se může objevit také pole. Funkce může pole i vracet. Funkce má ještě volitelný čtvrtý parametr, který se předává odkazem. Pokud je čtvrtý parametr uveden, je do něj uložen počet provedených úprav. Funkce `str_replace()` je case senzitivní. Jazyk PHP obsahuje i case insenzitivní variantu s názvem `str_ireplace()`, která má stejné parametry jako funkce `str_replace()`.

Další dvě funkce sloužící pro úpravu řetězců v PHP souvisí s jazykem HTML, který se používá pro tvorbu webových stránek. Při zpracování vstupů od uživatele je většinou potřeba, aby byla zadaná data ve formě prostého textu. Pokud uživatel vyplňuje nějaká dat např. do prvku typu `textarea`, může samozřejmě zadat i HTML značky. Zadání HTML značek může být použito například k útokům na webovou aplikaci. Funkce `htmlspecialchars()` dokáže nahrazuje znaky jazyka HTML (konkrétně jde o znaky `&`, `"`, `'`, < a >) přepsat HTML entitami. Například znak `&` je nahrazen entitou `&`, znaky < a > jsou nahrazeny entitami `<` a `>`. Funkce má jeden povinný parametr, kterým je vstupní řetězec. Funkce vrací upravený řetězec. Pokud při převodu nastane nějaká chyba, funkce vrátí prázdný řetězec. Funkce má i další volitelné parametry, které souvisí s nastavením znakové sady řetězce apod. Podobnou funkcí je funkce `htmlentities()`, ale na rozdíl od předchozí funkce nahrazuje všechny znaky, pro které existuje HTML entita. Parametry i návratová hodnota funkce jsou stejné jako u funkce `htmlspecialchars()`. Opačnými funkcemi k funkcím `htmlspecialchars()` a `htmlentities()` jsou funkce `htmlspecialchars_decode()` a `html_entity_decode()`, které převádějí HTML entity na odpovídající znaky.

Při zpracování víceřádkového textu (zadaného například do prvku `textarea`) je někdy potřeba zachovat rozložení textu na řádky při zobrazení na HTML stránce. V prvku `textarea` je přechod na nový řádek proveden pomocí znaku pro nový řádek (např. `\n`), v kódu HTML je potřeba použít značku `<br>` (případně `<br/>` pro kód XHTML). K nahrazení znaků nového řádku HTML značkami slouží funkce `nl2br()`, jejímž parametrem je řetězec, ve kterém chceme provést změnu. Kromě toho má funkce druhý, nepovinný, parametr – logickou hodnotu určující, zda se jedná o XHTML nebo o HTML. Hodnota `true` znamená XHTML a je použita jako výchozí hodnota v případě neuvedení druhé parametru.

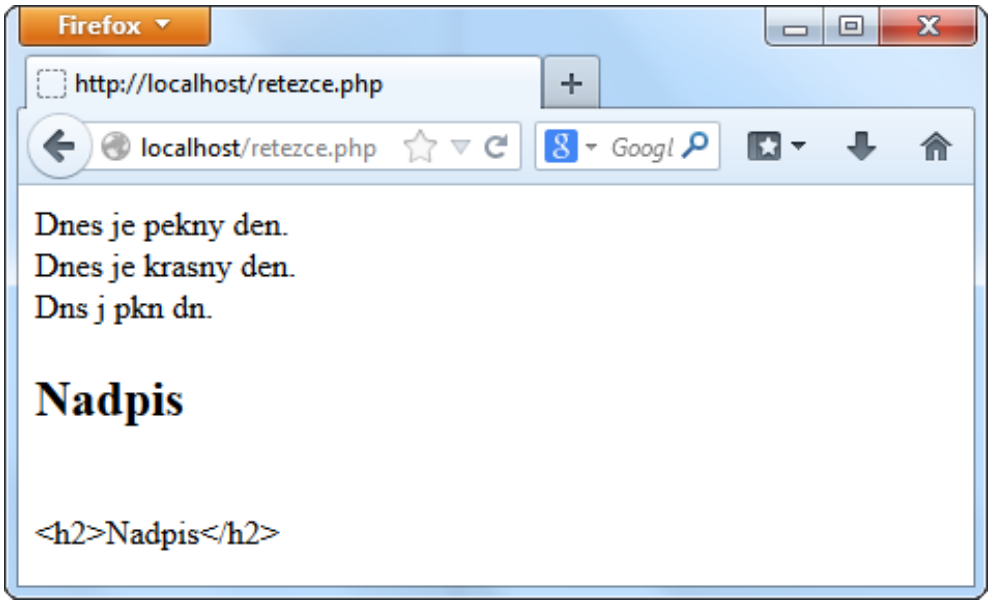
Funkce `wordwrap()` slouží k vložení HTML značky pro konec řádku po zadaném počtu znaků. Ve výchozím nastavení je maximální délka řádku `75` znaků a značka `<br>` se vždy vloží na pozici mezery. Pomocí volitelných parametrů lze nastavit kratší délku řádku i to, že se HTML značka vloží do slova po dosažení zadaného počtu znaků (může se projevit v případě velmi dlouhých slov).

Posledními funkcemi, které zde uvedeme, jsou funkce `stripslashes()` a `addslashes()`. Funkce `addslashes()` přidá před speciální znaky (jednoduché i dvojité uvozovky, obrácené lomítko a nulový bajt) znak obráceného lomítka. Tím pádem už je při zpracování skriptu považován za obyčejný znak. Při zpracování dat zaslaných metodou GET nebo POST nebo pomocí cookie se tato metoda volá automaticky. Předpokladem je, že v konfiguraci PHP je zapnutá volba Magic Quotes. Funkce `stripslashes()` má opačnou funkčnost jako funkce `addslashes()`. Ze vstupního řetězce odstraňuje znak obráceného lomítka.

Výpis 9.3 Ukázka použití funkcí str_replace() a htmlspecialchars().



```
<?php
  $text = "Dnes je pekny den.";
  echo $text . "<br>";
  echo str_replace("pekny", "krasny", $text) .
   "<br>";
  $samohlasky = array("a", "e", "i", "o", "u", "y");
  echo str_ireplace($samohlasky, "", $text) . "<br>";
  $text = "<h2>Nadpis</h2>";
  echo $text . "<br>";
  echo htmlspecialchars($text) . "<br>";
?>
```



Obrázek 9.4 Použití funkcí str_replace() a htmlspecialchars().

9.4 Rozdělování a spojování řetězců



V některých situacích je potřeba spojit prvky pole do jednoho řetězce. K tomuto účelu je v PHP k dispozici funkce `implode()`. Funkce je k dispozici ve dvou variantách. První varianta má dva parametry – prvním parametrem je řetězec, který se při spojování vloží mezi jednotlivé prvky pole. Druhým parametrem je pole, jehož prvky chceme spojit. Druhá varianta funkce má pouze jeden parametr, kterým je pole, jehož prvky chceme spojit. Mezi jednotlivé prvky pak není vložen žádný oddělovací řetězec. Obě varianty funkce `implode()` vrací výsledný řetězec. Alternativou je použití funkce `join()`, která je ale pouze aliasem funkce `implode()` – má tedy stejné parametry i návratovou hodnotu.

Opačnou operací je rozdělení řetězce na části, které se uloží do pole. K tomuto účelu je možné použít funkci `explode()`, která má dva povinné parametry. Prvním parametrem je řetězec představující oddělovač, podle kterého se vstupní řetězec rozdělí. Druhým parametrem je řetězec, který chceme dělit. Kromě toho je možné použít třetí volitelný parametr, kterým je umožňuje nastavit výsledný počet prvků pole. Návratovou hodnotou je pole řetězců.

V případě, že vstupní řetězec nechceme dělit podle nějakého oddělovače, ale chceme ho rozdělit na řetězce stejné délky, můžeme použít funkci `str_split()`, která má jako povinný parametr vstupní řetězec. Maximální délka jednotlivých řetězců se nastavuje pomocí druhého volitelného parametru.

Pokud není druhý parametr uveden, rozdělí se vstupní řetězec po jednotlivých znacích.

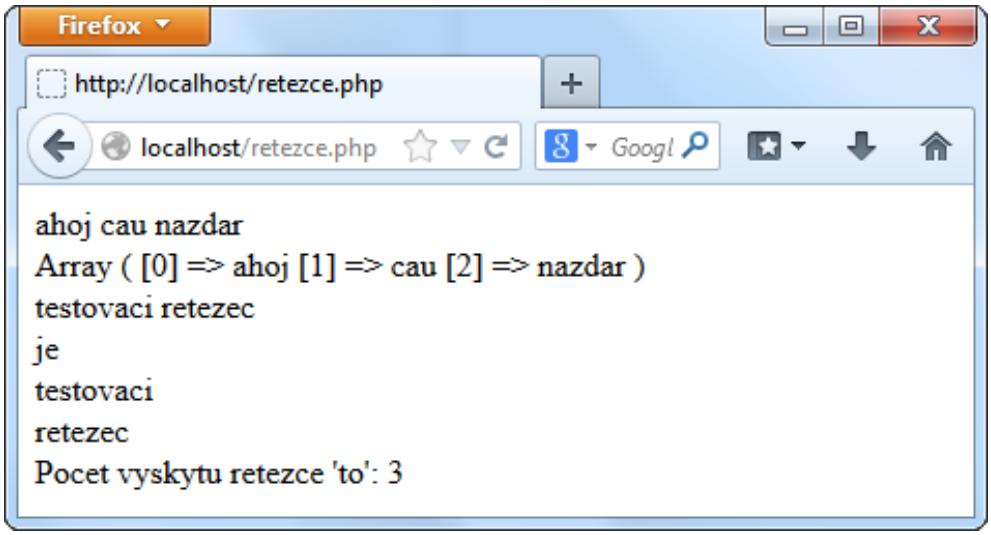
Jazyk PHP obsahuje i funkci `substr()`, která vrací část vstupního řetězce. Návratovou hodnotou je také řetězec. Funkce má dva povinné parametry, kdy prvním parametrem je vstupní řetězec a druhým je index znaku, na kterém má začínat podřetězec. První znak má index `0`. Třetím parametrem je délka podřetězce. Tento parametr je volitelný. Pokud není zadán, vrátí se podřetězec od zadané pozice do konce vstupního řetězce.

Funkce `substr_count()` slouží k určení, kolikrát se ve vstupním řetězci vyskytuje určitá část. Funkce vrací celé číslo. Povinnými parametry jsou vstupní řetězec a řetězec, jehož výskyt se v něm vyhledává. Volitelnými parametry je možné nastavit, kde začne a skončí hledání ve vstupním řetězci.

Výpis 9.4 Použití funkcí implode(), explode(), substr() a substr_count().



```
<?php
  $slova1 = array("ahoj", "cau", "nazdar");
  $text = implode(" ", $slova1);
  echo "$text<br>";
  $slova2 = explode(" ", $text);
  print_r($slova2);
  echo "<br>";
  $text = "toto je testovaci retezec";
  echo substr($text, 8) . "<br>";
  echo substr($text, 5, 2) . "<br>";
  echo substr($text, 8, -8) . "<br>";
  echo substr($text, -7) . "<br>";
  echo "Pocet vyskytu retezce 'to': " .
   substr_count($text, "to");
?>
```



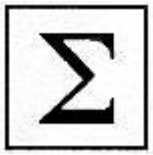
Obrázek 9.5 Použití funkcí implode(), explode(), substr() a substr_count().

Kontrolní otázky:



1. Jakým způsobem je možné vytisknout řetězec?
2. Jakou funkcí je možné vytvořit formátovaný řetězec?
3. Jaký je rozdíl mezi funkcemi `implode()` a `join()`?
4. K čemu slouží funkce `substr_count()`?

Shrnutí obsahu kapitoly



V této kapitole jste s předdefinovanými funkcemi v PHP, které slouží pro práci s textovými řetězci. Tisk řetězce je možné provést příkazy echo a print. Pro vytvoření formátovaného výstupu slouží funkce printf. K řetězci lze přistupovat jako k poli, pro práci s jednotlivými znaky se používají index uvedené v hranatých nebo složených závorkách. Pomocí funkcí lze v PHP zjistit délku řetězce a najít první nebo poslední výskyt řetězce v jiném řetězci. Úpravu řetězce je možné provést nahrazením části řetězce jiným řetězcem, přidáním obráceného lomítka před speciální znaky nebo nahrazením speciálních znaků jazyka HTML odpovídajícími HTML entitami. Řetězce lze také rozdělit podle zadaného oddělovače na pole dílčích řetězců a nebo je možné prvky pole spojit do jednoho řetězce. Jazyk PHP obsahuje mnoho funkcí, jejich popis v této kapitole není úplný, jsou zmíněny jen některé z funkcí.

10 Práce s polem, práce s datem a časem

V této kapitole se dozvíte:

- Jak vytvořit pole a procházet jeho prvky.
- Jak vyhledávat v poli a řadit pole.
- Jak pracovat s datem a časem pomocí funkcí a pomocí třídy DateTime.

Po jejím prostudování byste měli být schopni:

- Umět použít funkce pro vytvoření a procházení polí.
- Umět použít funkce pro vyhledávání v poli a pro řazení polí.
- Umět pracovat s datem a časem.

Klíčová slova této kapitoly:

pole, array, print_r, count, in_array, array_search, array_keys, datum a čas, date, time, mktime, getdate.

Doba potřebná ke studiu: 2 hodiny



Průvodce studiem

Studium této kapitoly není náročné. Jedná se kapitolu, která studenta seznamuje v dalšími předdefinovanými funkcemi v jazyce PHP. Úvod kapitoly se zabývá prací s polem – vytvoření a procházení pole, vyhledávání v poli, řazení polí. Druhá část kapitoly je věnována práci s datem a časem pomocí funkcí a pomocí objektového přístupu.

Další skupinou předdefinovaných funkcí, které jsou dostupné v jazyce PHP, jsou funkce pro práci s pole a funkce pro práci s kalendářním datem a časem. V této kapitole se seznámíme s některými z těchto funkcí. Stejně jako v případě řetězců nebude jejich výčet úplný

10.1 Práce s polem

Datový typ pole patří mezi složené datové typy. Pole v PHP je v podstatě mapou, která spojuje hodnoty (prvky pole) s jejich klíči (indexy). Indexy mohou být tvořeny řadou čísel, kdy první prvek má index `0`, druhý index `1` atd. Tento způsob reprezentace pole je typický pro jazyky jako C/C++ nebo Java. Kromě toho mohou být indexy prvků tvořeny nespojitou řadou čísel a nebo textovými řetězci. Pokud jsou indexy tvořeny textovými řetězci, hovoříme o asociativním poli. Jako prvky pole můžeme použít i pole, tím vytvoříme dvourozměrné pole.



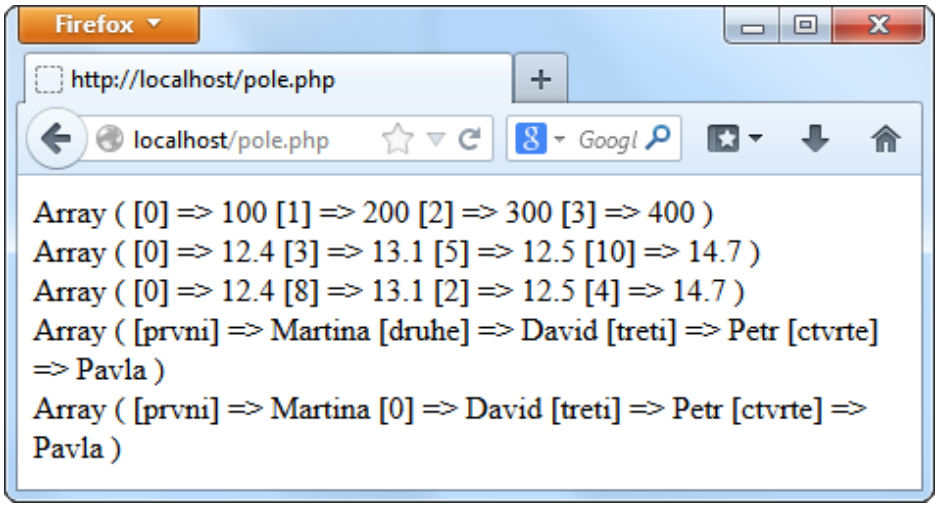
První funkcí, která souvisí s poli, je funkce `array()`, která vytvoří pole a vrátí jeho instanci. Vytvoření pole je možné několika způsoby a jedním z nich je použití této funkce. Jako parametry funkce uvádíme prvky pole. První možností je uvedení prvků pole oddělených čárkami. V takovém případě je

vytvořeno spojitě indexované pole, kdy první prvek pole má index `0`. Druhou možností je uvedení indexů a hodnot, čímž zpravidla vytváříme nespojitě indexované nebo asociativní pole. Jednotlivé prvky jsou opět odděleny čárkami a uvádí se ve tvaru `index => hodnota`. Poslední možností je, že indexy uvedeme pouze u některých prvků. U prvků, u kterých není index uveden, se doplní automaticky. Následující výpis ukazuje různé možnosti použití funkce `array()`. Na obrázku pod výpisem je zobrazen výsledek zobrazený v okně prohlížeče. K vytištění pole je použita funkce `print_r()`, která vytiskne pole v čitelné podobě – vytiskne prvky pole a k nim i jejich indexy.

Výpis 10.1 Ukázka použití funkce array().

```
<?php
  // spojite indexovane pole
  $pole1 = array(100, 200, 300, 400);
  // nespojite indexovane pole
  $pole2 = array(0 => 12.4, 3 => 13.1, 5 => 12.5, 10
   => 14.7);
  // nespojite indexovane pole
  $pole3 = array(0 => 12.4, 8 => 13.1, 2 => 12.5, 4
   => 14.7);
  // asociativni pole
  $pole4 = array("prvni" => "Martina", "druhe" =>
   "David", "treti" => "Petr", "ctvrte" => "Pavla");
  // asociativni pole
  $pole5 = array("prvni" => "Martina", "David",
   "treti" => "Petr", "ctvrte" => "Pavla");
  print_r($pole1);
  echo "<br>";
  print_r($pole2);
  echo "<br>";
  print_r($pole3);
  echo "<br>";
  print_r($pole4);
  echo "<br>";
  print_r($pole5);
?>
```





Obrázek 10.1 Použití funkce array().



Další funkcí, která je velmi často užitečná při práci s polem, je funkce `count()`. Parametrem funkce je pole a návratovou hodnotou je počet prvků tohoto pole. Použití funkce `count()` se hodí zejména v případě, že chceme projít spojitě indexované pole – potom k průchodu polem můžeme použít cyklus `for`. Kromě toho je v PHP k dispozici funkce `sizeof()`, která je aliasem funkce `count()`.

V případě spojitě indexovaných polí můžeme k procházení pole použít cyklus `for`. Pokud ale pracujeme s nespojitě indexovaným nebo asociativním polem, můžeme k procházení pole použít sadu funkcí. Každé pole má interní ukazatel na aktuální prvek pole, který je po vytvoření pole nastaven na první prvek pole.

K získání hodnoty aktuálního prvku slouží funkce `current()`, případně její alias – funkce `pos()`. Parametrem funkcí je pole, ze kterého chceme získat aktuální prvek. Podobnou funkčnost má funkce `each()`, jejímž parametrem je také pole. Funkce vrací nejen hodnotu aktuálního prvku, ale i jeho index a to ve formě pole, které má čtyři prvky. Indexy těchto prvků jsou `0`, `1`, `key` a `value`. Prvky s indexy `0` a `key` obsahují název indexu aktuální prvku. Prvky s indexy `1` a `value` obsahují jeho hodnotu. Kromě toho se funkce posune na následující prvek pole. Pokud se posunem interní ukazatel dostane mimo rozsah pole, vrátí funkce `each()` hodnotu `false`.

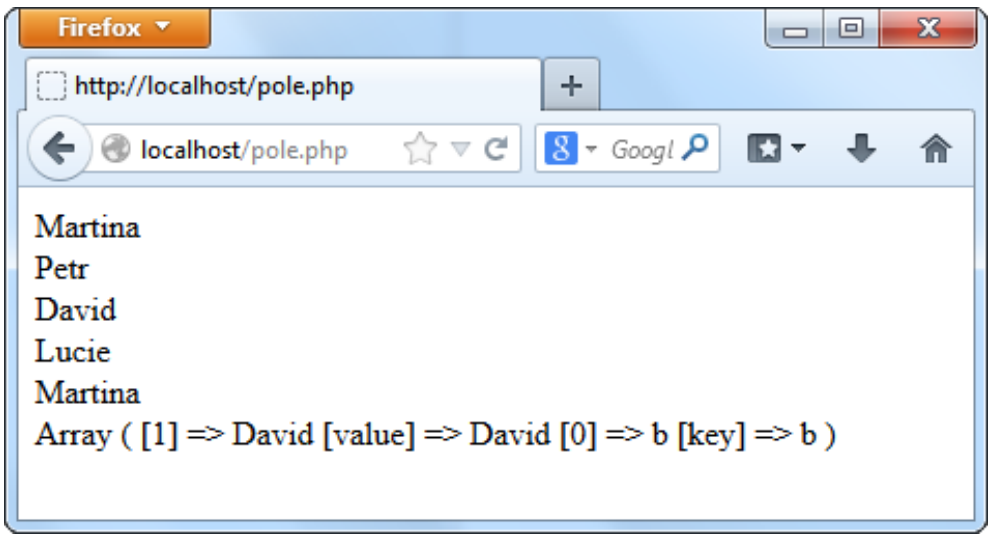
K procházení pole můžeme použít funkce `next()` a `prev()`. Obě funkce mají jako parametr pole. Funkce `next()` se posune na následující prvek pole od aktuální pozice a vrátí jeho hodnotu. Pokud je aktuální pozicí poslední prvek a posunem se dostaneme mimo rozsah pole, vrátí funkce hodnotu

`false`. Funkce `prev()` funguje úplně stejně, ale posouvá se na předchozí prvek od aktuální pozice.

Funkce `reset()` slouží k posunu interního ukazatele pole na první prvek, funkce `end()` slouží k posunu na poslední prvek. Pokud je pole prázdné, vrací obě funkce hodnotu `false`.

Výpis 10.2 Procházení polem promocí funkcí.

```
<?php
  $pole = array("a" => "Martina", "b" => "David", "c"
   => "Petr", "d" => "Pavla", "e" => "Radim", "f" =>
   "Lucie");
  $prvek = current($pole);
  echo $prvek . "<br>";
  $prvek = next($pole);
  $prvek = next($pole);
  echo $prvek . "<br>";
  $prvek = prev($pole);
  echo $prvek . "<br>";
  $prvek = end($pole);
  echo $prvek . "<br>";
  $prvek = reset($pole);
  echo $prvek . "<br>";
  $prvek = next($pole);
  $funkce_each = each($pole);
  print_r($funkce_each);
?>
```



Obrázek 10.2 Ukázka procházení polem pomocí funkcí.



Procházení polem je jednou z nejdůležitějších činností při práci s polem. Kromě toho ale občas potřebujeme i další činnosti – vyhledávat v poli, řadit pole atd. Ke zjištění, zda se v poli vyskytuje určitá hodnota, je určena funkce `in_array()`. Funkce má dva parametry, prvním parametrem je hledaná hodnota, druhým parametrem je pole, ve kterém se hledá. U této funkce je možné použít ještě třetí, volitelný, parametr, kterým se určuje, zda se při hledání v poli kontroluje i shoda datového typu. Funkce `in_array()` vrací `true`, pokud je hledaná hodnota nalezena, jinak vrací `false`. Funkce `in_array()` je case senzitivní. Podobnou funkcí je funkce `array_search()`, která má stejné parametry jako funkce `in_array()`. Funkce nevrací logickou hodnotu, ale index prvku, jehož hodnota odpovídá hledané hodnotě. Pokud není zadaná hodnota nalezena v poli, vrací funkce hodnotu `false`.

Funkce `array_key_exists()` zjišťuje, zda se v poli nachází zadaný index (klíč). Prvním parametrem funkce je hledaný index, druhým parametrem je pole, ve kterém se hledá. Funkce vrací logickou hodnotu `true`, pokud je index nalezen, jinak vrací `false`. Funkce `array_keys()` vrací ve formě pole všechny klíče v zadaném poli. Pole vstupuje do funkce jako parametr. Pomocí dalších volitelných parametrů můžeme určit, že se vrátí pouze některé z indexů. Podobnou funkcí je funkce `array_values()`, která vrací všechny hodnoty zadaného pole. Jediným parametrem funkce je pole, se kterým se pracuje.

Výpis 10.3 Vyhledávání v poli.



```
<?php
  $pole = array("a" => "Martina", "b" => "David", "c"
   => "Petr", "d" => "Pavla", "e" => "Radim", "f" =>
   "Lucie");

  if (in_array("Martina", $pole))
    echo "'Martina' se nachazi v poli<br>";
  else
    echo "'Martina' se nenachazi v poli<br>";

  if (in_array("pavla", $pole))
    echo "'pavla' se nachazi v poli<br>";
  else
    echo "'pavla' se nenachazi v poli<br>";

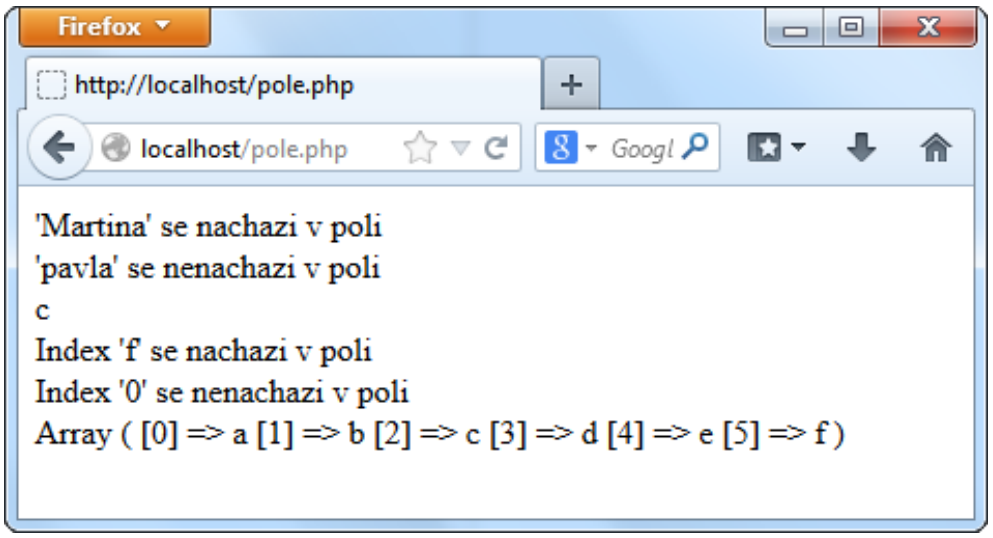
  $index = array_search("Petr", $pole);
```

```
  echo "$index<br>";

  if (array_key_exists("f", $pole))
    echo "Index 'f' se nachazi v poli<br>";
  else
    echo "Index 'f' se nenachazi v poli<br>";

  if (array_key_exists("0", $pole))
    echo "Index '0' se nachazi v poli<br>";
  else
    echo "Index '0' se nenachazi v poli<br>";

  $indexy = array_keys($pole);
  print_r($indexy);
?>
```



Obrázek 10.3 Vyhledávání v poli.

Seřazení pole je možné pomocí funkce `sort()`. Povinným parametrem je pole, které se řadí. Volitelným parametrem je nastavení řazení – porovnání hodnot jako čísel nebo jako řetězců atd. Funkce vrací hodnotu `true`, pokud je řazení úspěšné, jinak vrací hodnotu `false`. Funkce `sort()` řadí prvky od nejmenšího k největšímu. K seřazení od největšího prvku po nejmenší slouží funkce `rsort()`. Parametry jsou stejné jako u funkce `sort()`. Funkce `ksort()` slouží k vzestupnému seřazení pole podle jeho indexů. Sestupné seřazení pole podle indexů je možné pomocí funkce `krsort()`. Obě funkce vrací `true`, pokud je řazení úspěšné, jinak vrací `false`.

Kromě uvedených funkcí, obsahuje jazyk PHP mnoho dalších funkcí pro práci s polem. Pomocí těchto funkcí můžeme například vypočítat součet nebo součin prvků pole (samozřejmě pouze v případě, že prvky pole jsou tvořeny čísly),

spojit více polí dohromady, vytvořit pole obsahující prvky v opačném pořadí, určit jedinečné hodnoty v poli a počet jejich výskytů atd.

10.2 Práce s datem a časem

Při práci s kalendářním datem a časem můžeme použít dva přístupy. První přístup je procedurální, kdy používáme funkce pro datum a čas. Druhý přístup je objektový, k práci s datem a časem používáme třídu `DateTime`.



V procedurálním přístupu jsou základními funkcemi `mktime()`, `date()` a `time()`. Funkce `time()` nemá žádné parametry a vrací celé číslo – aktuální Unixové časové razítko. Časové razítko udává počet sekund od počátku. Jako počátek se považuje datum 1.1.1970 0:0:0 GMT.

Pomocí funkce `mktime()` můžeme pomocí parametrů zadat požadované datum a požadovaný čas. Parametry se zadávají ve formě celých čísel v pořadí hodiny, minuty, sekundy, měsíc, den, rok. Rok se zadává pomocí dvou nebo čtyř číslic. Ostatní parametry se zadávají relativně k nadřazené části data. Nadřazenou částí data je vždy nejbližší větší časový úsek. Roky jsou nadřazeny měsícům, měsíce dnům, dny hodinám atd. Pokud tedy například zadáme v případě měsíce hodnoty `1` až `12`, nastavíme tím příslušný měsíc. Pokud zadáme hodnotu menší než `1`, posuneme datum do předchozího roku než bylo zadáno. Hodnota `0` znamená prosinec předchozího roku, hodnota `-1` listopad atd. Pokud zadáme hodnotu větší než `12`, posouváme datum do následujícího roku. V případě minut jsou nadřazenou částí data hodiny. Hodnoty `0` až `59` nastaví čas na příslušnou minutu. Hodnoty menší než `0` posunou čas do předchozí hodiny, hodnoty větší než `59` do následující hodiny. Funkce `mktime()` vrací celé číslo – Unixové časové razítko odpovídající zadanému datu a času.

Funkce `date()` slouží k naformátování data a času. Povinným parametrem je řetězec, pomocí něhož určujeme formát data a času. Nepovinným parametrem celé číslo udávající Unixové časové razítko. Pokud zadáme pouze první parametr, pro formátování se použije aktuální datum a čas serveru. Zadáním druhého parametru můžeme naformátovat libovolné jiné datum. Formát data a času se určuje pomocí formátovacích znaků, mezi které můžeme vložit oddělovač (mezera, lomítko, tečka atd.). Popis nejdůležitějších formátovacích znaků je uveden v následující tabulce.

INVESTICE DO ROZVOJE VZDĚLÁVÁNÍ

Tabulka 10.1 Formátovací znaky pro datum a čas.



Znak	Význam	Hodnoty
d	Den v měsíci zobrazený pomocí dvou číslic, s nulou na začátku.	01 až 31
D	Třípísmenná textová zkratka dne v týdnu.	Mon (pondělí) až Sun (neděle)
j	Den v měsíci zobrazený bez nuly na začátku.	1 až 31
l	Malé písmeno „L“. Plný název dne v týdnu.	Sunday (neděle) až Saturday (sobota)
N	Číselná reprezentace dne v týdnu dle normy ISO-8601.	1 (pondělí) až 7 (neděle)
w	Číselná reprezentace dne v týdnu dle anglického vyjádření.	0 (neděle) až 6 (sobota)
z	Den v roce, začíná od nuly	0 až 365
W	Číslo týdnu v roce dle normy ISO-8601.	0 až 52
F	Plný název měsíce.	January (leden) až December (prosinec)
m	Číselná reprezentace měsíce zobrazená pomocí dvou číslic.	01 až 12
M	Třípísmenná zkratka měsíce	Jan (leden) až Dec (prosinec)
n	Číselná reprezentace měsíce bez nuly na začátku	1 až 12
t	Počet dnů v zadaném měsíci.	28 až 31
L	Zjištění, zda je přestupný rok.	1 (ano) nebo 0 (ne)
Y	Plné vyjádření roku – čtyři číslice.	např. 1981 nebo 2013
y	Vyjádření roku pomocí dvou číslic (poslední dvě číslice)	např. 81 nebo 13
a	Určení zda se jedná o dopoledne nebo odpoledne, malá písmena.	am nebo pm
A	Určení, zda se jedná o dopoledne nebo odpoledne, velká písmena	AM nebo PM
g	Hodiny, 12-hodinový formát, bez nuly na začátku.	1 až 12
G	Hodiny, 24-hodinový formát, bez nuly na začátku.	0 až 23
h	Hodiny, 12-hodinový formát, s nulou na začátku.	01 až 12

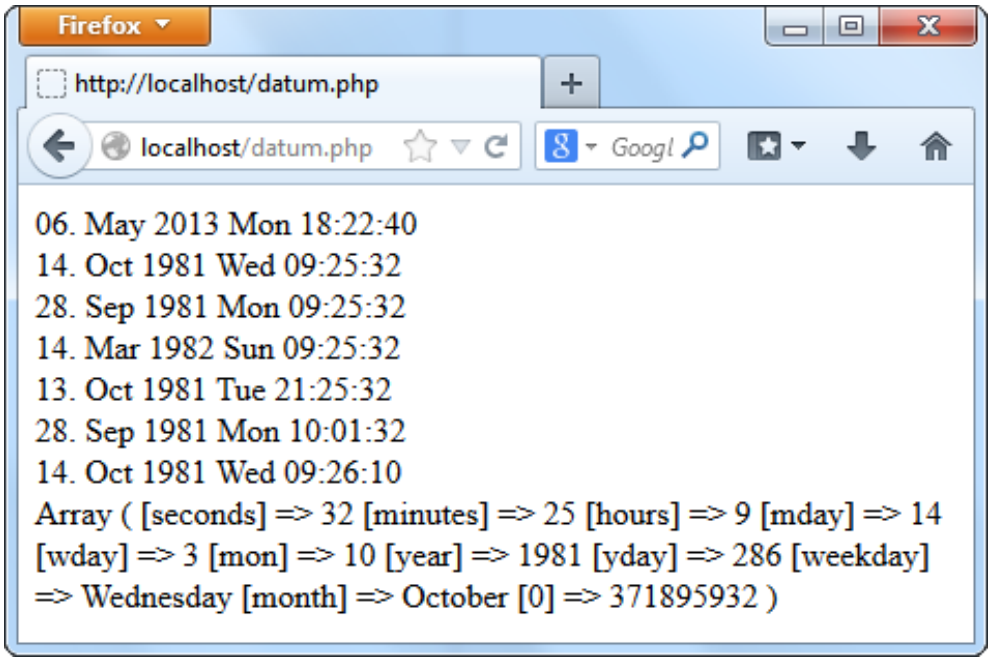
H	Hodiny, 24-hodinový formát, s nulou na začátku.	00 až 23
i	Minuty, s nulou na začátku.	00 až 59
s	Sekundy, s nulou na začátku.	00 až 59

Funkce `getdate()` vrací asociativní pole, které reprezentuje datum. Pokud funkci zavoláme bez parametrů, je použito aktuální datum a aktuální čas serveru. Volitelným parametrem je zadání časového razítka, které reprezentuje jakékoliv jiné datum.

Výpis 10.4 Práce s datem a časem.



```
<?php
  // aktualni datum a cas
  $dnes = date("d. M Y D H:i:s");
  echo "$dnes<br>";
  // 14. rijna 1981 9 hod 25 min 32 sek
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 32,
   10, 14, 1981));
  echo "$dnes<br>";
  // posun dne
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 32,
   10, -2, 1981));
  echo "$dnes<br>";
  // posun mesice
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 32,
   15, 14, 1981));
  echo "$dnes<br>";
  // posun hodin
  $dnes = date("d. M Y D H:i:s", mktime(-3, 25, 32,
   10, 14, 1981));
  echo "$dnes<br>";
  // posun minut
  $dnes = date("d. M Y D H:i:s", mktime(9, 61, 32,
   10, -2, 1981));
  echo "$dnes<br>";
  // posun sekund
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 70,
   10, 14, 1981));
  echo "$dnes<br>";
  // pouziti funkce getdate()
  $dnes = getdate(mktime(9, 25, 32, 10, 14, 1981));
  print_r($dnes);
?>
```



Obrázek 10.4 Práce s datem a časem.



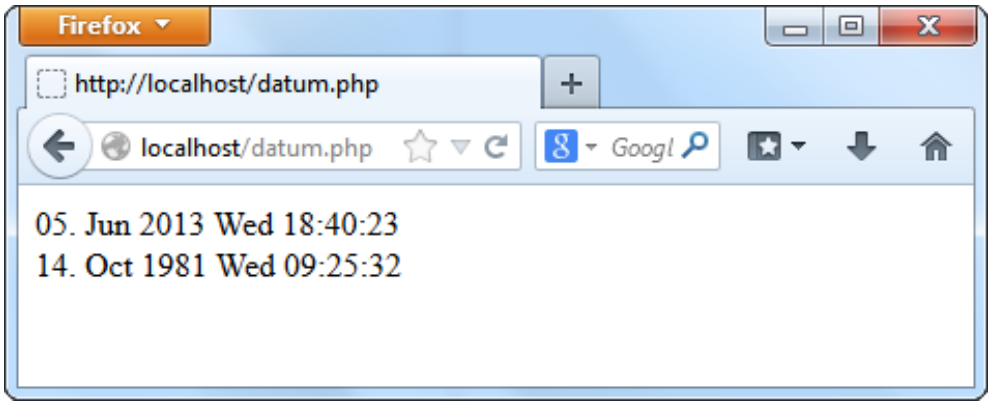
Práce s datem a časem pomocí objektového přístupu je možná pomocí třídy `DateTime`. Vytvořením nové instance této třídy vytvoříme objekt představující datum a čas. Pokud použijeme konstruktor bez parametrů, použije se aktuální datum a čas. Jiné datum a čas můžeme vytvořit jeho zadáním při vytváření objektu. Třída `DateTime` obsahuje metody, které mají stejnou funkčnost jako při použití procedurálního přístupu. K naformátování data slouží metoda `format()`, k nastavení data slouží metoda `setDate()`, k nastavení času metoda `setTime()` a k nastavení časového razítka metoda `setTimestamp()`.

Výpis 10.5 Použití třídy DateTime.



```
<?php
  // aktualni datum a cas
  $dnes = new DateTime();
  echo $dnes->format("d. M Y D H:i:s") . "<br>";

  // zmena data a casu
  $dnes->setDate(1981, 10, 14);
  $dnes->setTime(9, 25, 32);
  echo $dnes->format("d. M Y D H:i:s") . "<br>";
?>
```

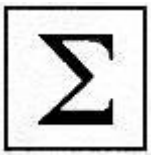


Obrázek 10.5 Použití třídy DateTime.



Kontrolní otázky:

1. Jakým způsobem je možné indexovat prvky pole?
2. Jaké funkce je možné použít pro vyhledávání v poli?
3. K čemu slouží funkce `mktime()`?
4. K čemu slouží funkce `getdate()`?



Shrnutí obsahu kapitoly

V této kapitole jste se seznámili s předdefinovanými funkcemi v PHP, které slouží pro práci s polem a kalendářním datem a časem. Jazyk PHP umožňuje vytvářet spojitě nebo nespojitě indexovaná pole nebo asociativní pole. Při práci s polem můžeme použít funkce pro procházení polem, vyhledávání v poli nebo řazení pole. Práce s kalendářním datem a časem je možné procedurálním způsobem (pomocí funkcí) nebo objektově (pomocí třídy DateTime). Zadané datum je možné naformátovat různými způsoby pomocí speciálních formátovacích znaků. Jazyk PHP obsahuje mnoho funkcí, jejich popis v této kapitole není úplný, jsou zmíněny jen některé z funkcí.

11 Objektově orientované programování v PHP

V této kapitole se dozvíte:

- Jak vytvářet aplikace v PHP pomocí objektového přístupu.
- Jak vytvořit odvozenou třídu.
- Jak přetěžovat proměnné a metody.

Po jejím prostudování byste měli být schopni:

- Umět vytvořit třídu v PHP.
- Umět použít dědičnost při tvorbě tříd.
- Umět přetěžovat proměnné a metody.

Klíčová slova této kapitoly:

class, konstruktor, destruktor, dědičnost, public, protected, private.

Doba potřebná ke studiu: 2 hodiny

Průvodce studiem



Studium této kapitoly předpokládá alespoň základní znalosti objektově orientovaného programování. Kapitola se nezabývá vysvětlováním principů objektově orientovaného programování, ale vysvětluje tvorbu tříd v jazyce PHP.

Vývoj programů byl v minulosti realizován pomocí strukturovaného programování, které se vyznačuje tím, že striktně odděluje data a funkce. Během 90. let 20. století se začalo rozšiřovat objektově orientované programování (OOP). V tomto přístupu jsou základním stavebním kamenem všech programů objekty, které obsahují jak data, tak funkce. Definici typu objektu označujeme jako třídu, objekty jsou pak jednotlivé instance této třídy. Jako jednoduchý příklad uveďme situaci, kdy bychom pracovali s geometrickými tvary. Jedním ze základních geometrických tvarů je obdélník. K reprezentaci obdélníku vytvoříme třídu *Obdélník*. Data v této třídě budou tvořit rozměry obdélníku, tj. jeho *šířka* a *výška*. Kromě toho můžeme ve třídě definovat funkce (metody), které vypočítají *obvod* a *obsah* obdélníku. Vytvořením instance třídy vytvoříme jeden konkrétní obdélník o zadaných rozměrech.

V jazyce PHP verze 5 je již k dispozici kompletní podpora objektově orientovaného programování. Používání objektového přístupu ale není v PHP nutností, jako je tomu například v jazyce Java. Pokud nechceme, můžeme používat klasický strukturovaný přístup. Volba přístupu při tvorbě aplikace závisí na tom, jak složitá je naše aplikace. Pokud vytváříme pouze jednoduchou webovou aplikaci skládající se z několika PHP skriptů, je použití

objektového přístupu zbytečné. V případě rozsáhlých aplikací, které se stále rozšiřují je již použití objektů téměř nezbytné.

Tato kapitola se nebude zabývat principy OOP, ale bude se zabývat syntaxí v PHP, která souvisí s tvorbou tříd, objektů apod.

11.1 Vytvoření třídy a její instance



Pokud používáme při tvorbě webové aplikace v PHP objektový přístup, bude naše aplikace obsahovat definici tříd, vytvoření instancí tříd a volání jejich metod. Aby byly zdrojové kódy aplikace přehledné, je vhodné vytvořit pro každou třídu samostatný soubor. Pokud potřebujeme v nějaké skriptu použít určitou třídu, stačí vložit do tohoto skriptu soubor obsahující definici třídy. Definice třídy začíná klíčovým slovem `class`, za kterým následuje její název. Za názvem může následovat definice rodičovských objektů. Ta se zde ale vyskytuje pouze v případě, že chceme, aby objekt zdědil vlastnosti nadřazené třídy. Dědičnost (inheritance) je jedním ze základních pilířů OOP – nové třídy můžeme vytvářet rozšiřováním existujících tříd. Pokud chceme při definici třídy použít nadřazenou třídu, použijeme k tomu klíčové slovo `extends`. Dále již následují složené závorky, ve kterých je vlastní definice třídy.

Definice třídy se skládá ze dvou částí. První částí je definování členských proměnných třídy, druhou částí je definování jejích metod. Definice metody začíná klíčovým slovem `function`, stejně jako definice funkcí. V části obsahující metody mohou být umístěny dvě speciální metody – konstruktor a destruktor.

Konstruktor je metoda, která se automaticky volá při vytváření instance třídy. V konstruktoru je možné použít parametry (stejně jako v jakékoliv jiné metodě), které se předávají vytvořenému objektu. Od PHP verze 5 má konstruktor jednotný název `__construct()` – před slovem construct je uveden 2x za sebou znak _ (podržítko). V nižších verzích PHP se při definici konstruktoru používal stejný název jako je název třídy. Pokud vytváříme třídu, která dědí z jiné třídy, konstruktor nadřazené třídy se nevolá automaticky. Je nutné jej v případě potřeby zavolat, k čemuž slouží příkaz `parent::__construct()`.

Destruktor je metoda, která se volá při odstraňování objektu. Definice destruktoru není nutná, protože PHP automaticky provádí správu paměti. Doba běhu skriptů je většinou krátká, takže není potřeba odstraňovat objekty ručně.

Pokud ale i přesto potřebujeme destruktor vytvořit, použijeme metodu `__destruct()` – před slovem destruct je opět 2x znak _ (podtržítko).

Instanci třídy vytvoříme pomocí klíčového slova `new`, za kterým uvedeme název vytvářené třídy. Pokud konstruktor obsahuje parametry, je potřeba je uvést při vytváření instance třídy. Pokud je konstruktor bez parametrů, není nutné uvádět za názvem třídy prázdné kulaté závorky.

Následující výpis obsahuje ukázku vytvoření třídy a jejích potomků. Třída `OdvozenaTrida1` neobsahuje v konstruktoru volání nadřazené třídy, takže text „`Základní třída`“ se nevypíše. V konstruktoru třídy `OdvozenaTrida2` je již volání konstruktoru nadřazené třídy uvedeno, takže se vypíše i text „`Základní třída`“. Obrázek pod výpis pak zobrazuje výsledek zobrazený v prohlížeči.

Výpis 11.1 Vytvoření třídy.

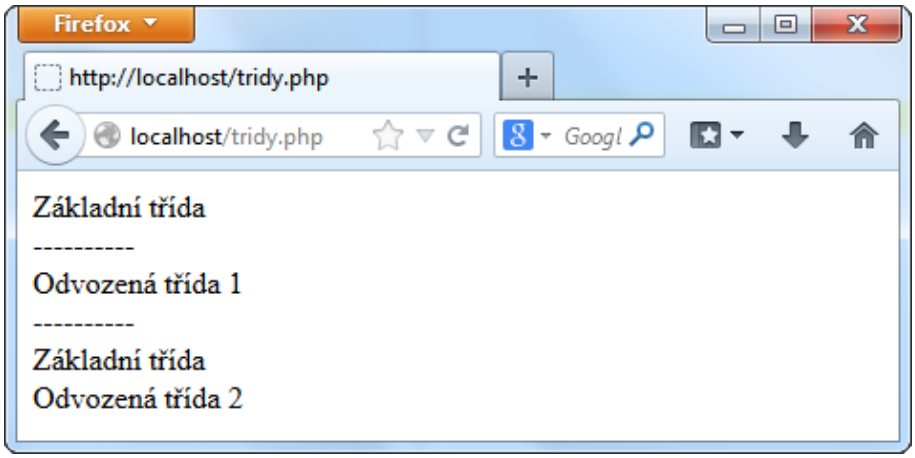
```
<?php
  class ZakladniTrida {
    function __construct() {
      echo "Základní třída<br>";
    }
  }

  class OdvozenaTrida1 extends ZakladniTrida {
    function __construct() {
      echo "Odvozená třída 1<br>";
    }
  }

  class OdvozenaTrida2 extends ZakladniTrida {
    function __construct() {
      parent::__construct();
      echo "Odvozená třída 2<br>";
    }
  }

  $objekt = new ZakladniTrida();
  echo "----------<br>";
  $objekt = new OdvozenaTrida1();
  echo "----------<br>";
  $objekt = new OdvozenaTrida2();
?>
```





Obrázek 11.1 Vytvoření třídy.

11.2 Členské proměnné a metody, zapouzdření tříd

Definice každé třídy může obsahovat proměnné nebo metody (funkce), které bývají nazývány také jako členské proměnné a členské metody. Po vytvoření instance objektu můžeme přistupovat k členským proměnným nebo metodám pomocí operátoru `->`. Při přístupu k členské proměnné se za operátorem `->` již neuvádí znak `$` (dolar). Pokud potřebujeme v rámci některé členské metody pracovat s aktuálním objektem, použijeme k tomu proměnnou `$this`, která je k dispozici v každé třídě a představuje odkaz na aktuální objekt.



Dalším pilířem objektově orientovaného programování je zapouzdření (encapsulation). Pomocí zapouzdření můžeme při definici třídy určit, komu jsou přístupné její členské proměnné a metody. V jazyce PHP existují tři možnosti definice přístupu, které jsou stejné jako ve většině jiných programovacích jazyků. Jedná se o přístup `private`, `protected` a `public`. Přístup `private` znamená, že k proměnné nebo k metodě smí přistupovat pouze třída, ve které jsou definovány. Přístup `protected` znamená, že k proměnné nebo metodě smí přistupovat třída, ve které jsou definovány, a nebo potomci třídy (odvozené třídy). Přístup `public` znamená, že k proměnné nebo metodě smí přistupovat kdokoliv. Následující výpis zobrazuje ukázku použití různých přístupových práv k proměnným.

Výpis 11.2 Ukázka použití private, protected a public.



```
<?php
  class Trida {
    private $priv = 1;
    protected $prot = 2;
    public $publ = 3;
```

```
  }

  $o = new Trida();
  echo $o->publ; // v poradku
  echo $o->prot; // chyba
  echo $o->priv; // chyba
?>
```

S přístupem k členským proměnným a metodám souvisí statické proměnné a metody. Statické proměnné nebo metody můžeme používat aniž bychom vytvořili instanci třídy. Pokud chceme označit některou z členských proměnných nebo metod jako statickou, uvedeme před název proměnné nebo metody klíčové slovo `static`. Přístup ke statickým proměnným nebo metodám provádíme pomocí operátoru `::` (znak dvojtečky 2x za sebou). Ve statických metodách nelze použít proměnnou `$this`. Pokud potřebujeme ve statické metodě pracovat s aktuálním objektem, je potřeba použít popisovat `self::`.



V některých případech je žádoucí, aby potomci třídy nepřepisovali některé z metod. Takové metody označíme jako finální, a to pomocí klíčového slova `final`. Pokud označíme jako finální celou třídu, nelze vytvořit jejího potomka. Přepsání konečné metody nebo vytvoření potomka konečné třídy vede ke kritické chybě. Následující výpis ukazuje použití statických proměnných nebo metod a konečných metod nebo tříd.

Výpis 11.3 Statické proměnné a metody, konečné metody.

```
<?php
  class Trida {
    public static $cislo = 10;
    public static function tiskCisla() {
      echo "Číslo: " . self::$cislo;
    }
  }

  echo Trida::$cislo . "<br>";
  Trida::tiskCisla();

  class TridaFinal {
    private $cislo = 20;
    public final function tiskCisla() {
      echo "Číslo: " . $this->cislo;
    }
  }
```



```
  class TridaFinal2 extends TridaFinal {
    public function tiskCisla2() {  // zpusobi chybu
      echo "Číslo: " . $this->cislo;
    }
  }
?>
```

11.3 Přetěžování proměnných a metod

Pojem přetěžování má v PHP jiný význam než je obvyklé. Jako přetěžování bývá v programovacích jazycích označován postup, kdy jedna metoda se stejným názvem existuje v různých variantách, které se liší svými parametry (jejich počtem nebo datovým typem) nebo svou návratovou hodnotou.

Přetěžování v PHP poskytuje způsob jak dynamicky vytvářet členské proměnné a metody. K tomuto účelu jsou určeny speciální funkce. Přetěžující metody se použití v případě, že pracujeme s proměnnými nebo metodami, které zatím nebyly vytvořeny nebo nejsou v aktuálním místě přístupné (např. proto, že jsou definovány jako `private`). Všechny přetěžující metody musí být definovány jako public.



Pro přetěžování proměnných jsou důležité dvě funkce, a to `__set()` a `__get()`. Stejně jako v případě metod konstruktoru a destruktoru začíná název metody dvěma znaky _ (podtržítka) za sebou. Funkce `__set()` slouží k nastavení hodnoty a má dva parametry. Prvním je název proměnné, druhým hodnota, kterou do ní chceme uložit. Funkce nic nevrací, resp. vrací typ `void`. Funkce `__get()` má pouze jeden parametr a tím je název proměnné, jejíž hodnotu chceme získat. Metoda vrací obsah proměnné, která může být různého typu, takže ani návratový typ není pěvně dán.

Jestliže jako první parametr metody `__set()` použijeme název proměnné, která neexistuje, je tato proměnná vytvořena. Metody `__get()` a `__set()` je možné použít pouze při práci s objektem. Nelze je použít na statické proměnné a nemohou být deklarovány jako static.

Výpis 11.4 Použití metod __get() a __set().



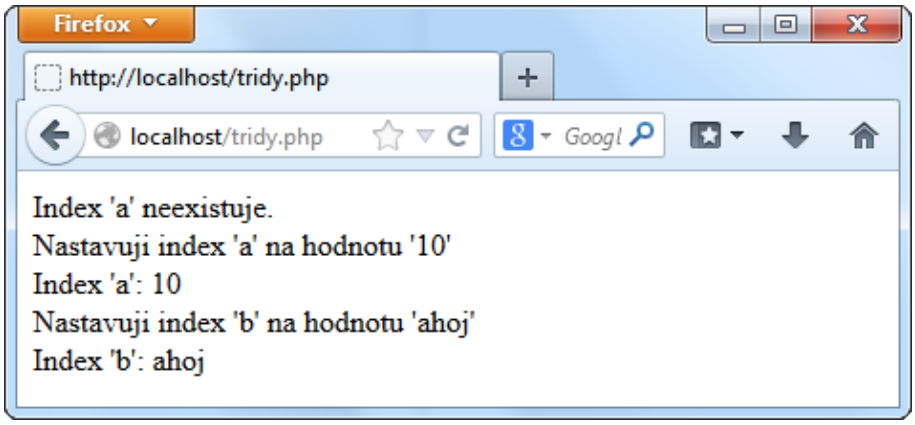
```
<?php
  class Trida {
    private $data = array();

    public function __set($name, $value) {
```

```
      echo "Nastavuji index '$name' na hodnotu
       '$value'<br>";
      $this->data[$name] = $value;
    }

    public function __get($name) {
      if (array_key_exists($name, $this->data)) {
        echo "Index '$name': ";
        return $this->data[$name];
      }
      else {
        echo "Index '$name' neexistuje.";
      }
    }
  }

  // pri vytvareni objektu nemusi byt uvedeny kulate
   zavorky
  $test = new Trida;
  echo $test->a . "<br>";
  $test->a = 10;
  echo $test->a . "<br>";
  $test->b = "ahoj";
  echo $test->b . "<br>";
?>
```



Obrázek 11.2 Použití metod __get() a __set().

Jazyk PHP umožňuje přetěžovat nejen proměnné, ale také metody. Pro přetěžování metod jsou v PHP určeny dvě metody - `__call()` a `__callStatic()`. Metoda `__call()` slouží pro přetěžování nestatických metod, metoda `__callStatic()` pro statické metody. Obě metody mají dva parametry. Prvním je název metody a druhým je pole, ve kterém jsou uloženy parametry metody.

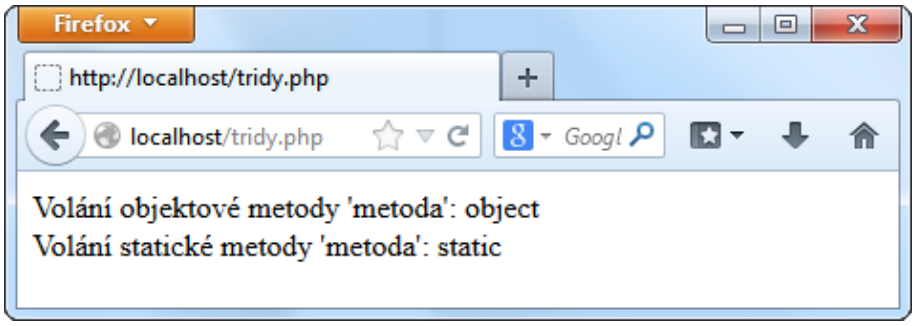


Výpis 11.5 Použití metod __call() a __callStatic().

```
<?php
  class Trida {
    public function __call($name, $arguments) {
      echo "Volání objektové metody '$name': " .
       implode(", ", $arguments) . "<br>";
    }

    public static function __callStatic($name,
     $arguments) {
      echo "Volání statické metody '$name': " .
       implode(', ', $arguments). "<br>";
    }
  }

  $arg = array(100, 200, 300, 400);
  $test = new Trida;
  $test->metoda('object');
  Trida::metoda('static');
?>
```



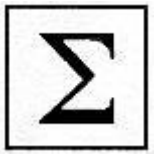
Obrázek 11.3 Použití metod __call() a __callStatic().

Kontrolní otázky:

1. Jakým vytvoříme třídu v PHP?
2. Jaký je rozdíl mezi `public`, `protected` a `private`?
3. Jak je možné v PHP vytvořit konstruktor a jak destruktor?
4. Co je to přetěžování proměnných?
5. Co je to přetěžování metod?

INVESTICE DO ROZVOJE VZDĚLÁVÁNÍ

Shrnutí obsahu kapitoly



V této kapitole jste se seznámili s tvorbou tříd v jazyce PHP. Třídy mohou obsahovat členské proměnné a členské metody. Speciálními metodami jsou konstruktor a destruktor, které slouží k vytvoření nebo zrušení instance třídy. Přístup ke členským proměnným nebo metodám může být nastaven jako `public`, `protected` nebo `private`. Jazyk PHP umožňuje dynamicky vytvářet členské proměnné nebo metody. Tato činnost bývá označována jako přetěžování.

12 Databáze MySQL

V této kapitole se dozvíte:

- Základní informace o databázích MySQL.
- Jak se připojit k databázovému serveru.
- Jak spouštět SQL příkazy na databázovém serveru.
- Jak uvolnit výsledky a uzavřít připojení databázi.

Po jejím prostudování byste měli být schopni:

- Umět používat rozhraní mysqli pro práci s databází MySQL.
- Umět spouštět libovolné příkazy SQL.
- Umět zpracovat výsledky, které vrátil příkaz SELECT.

Klíčová slova této kapitoly:

MySQL, mysqli, mysqli_real_connect, mysqli_select_db mysqli_query, mysqli_close mysqli_fetch_row, mysqli_fetch_array, mysqli_fetch_assoc, mysqli_fetch_object, mysqli_free_result.

Doba potřebná ke studiu: 4 hodiny



Průvodce studiem

Studium této kapitoly není náročné. V úvodu kapitoly se krátce seznámíte s databázovým serverem MySQL. Dále se již naučíte pracovat s tímto serverem pomocí jazyka PHP. Práce s databází spočívá ve čtyřech krocích – připojení k databázovému serveru, výběr databáze, spouštění příkazů a uzavření připojení k databázovému serveru.

Databázový server MySQL slouží k vytváření relačních databází. Je k dispozici v bezplatné verzi pro nekomerční a soukromé použití. Pro komerční použití existuje i placená varianta databázového serveru MySQL. Databáze se vždy skládá z tabulek, které jsou navzájem propojeny relacemi. V této opoře se nebudeme zabývat teoretickými východisky relačních databází. Zaměříme se pouze na tvorbu databází v MySQL a na práci s databází MySQL prostřednictvím jazyka PHP. Práce s databázovým serverem je možná pomocí různých nástrojů. Jedním z nich je phpMyAdmin, který je vytvořen v jazyce PHP a je k dispozici zdarma. Prostřednictvím internetového prohlížeče můžeme pomocí tohoto nástroje vytvářet databáze, tabulky, plnit je daty, dotazovat se apod.

12.1 Typy tabulek a datové typy

Specialitou MySQL je možnost zadání typu tabulky v okamžiku jejího vytvoření. Jednotlivé typy tabulek se liší různými vlastnostmi. Mezi nejdůležitější typy tabulek patří MyISAM, InnoDB a HEAP.

Tabulka typu MyISAM představuje v MySQL standardní typ tabulek. Jedná se o stabilní, vyspělý a jednoduše spravovatelný typ tabulek. Pokud není žádný speciální důvod pro použití jiného typu tabulky, je tento typ ideální.



Tabulka typu InnoDB má všechny vlastnosti typu MyISAM a navíc má dvě odlišnosti. První odlišností je, že databázové operace v tabulkách typu InnoDB je možné spouštět jako transakce. Druhou odlišností je, že tabulky InnoDB podporují používaní pravidel integrity. Kromě toho mají ale tabulky InnoDB i nevýhody. První nevýhodou je, že nelze vytvořit fulltextový index. Druhou je, že správa tabulek InnoDB je o něco složitější. Poslední nevýhodou je, že komerční licence MySQL s podporou InnoDB je dražší než verze bez ní.

Posledním typem jsou tabulky typu HEAP, které se vytváří pouze v operační paměti RAM. Neukládají se tedy na pevný disk. Používají tzv. index typu hash, který umožňuje rychlý přístup k záznamům.

Každý sloupec tabulky v relační databázi musí mít určený datový typ, který je stejný pro celý sloupec. Všechny záznamy mají v jednom sloupci hodnotu stejného typu. Databáze v MySQL nejsou výjimkou a platí pro ně totéž. Datový typ není závislý na typu tabulky. V dalších odstavcích si představíme nejdůležitější datové typy v MySQL

12.1.1 Celá čísla



V MySQL se standardně po všechna celá čísla používá znaménko. Pokud chceme do sloupce ukládat pouze kladná čísla, je nutné u definice sloupce zadat atribut UNSIGNED. V MySQL je k dispozici 6 datových typů pro celá čísla. Podle velikosti od nejmenšího po největší jsou to TINYINT, SMALLINT, MEDIUMINT, INT (nebo INTEGER) a BIGINT. Kromě toho existuje ještě typ SERIÁL, který slouží pro sloupec představující primární klíč. Typ TINYINT umožňuje uložit hodnoty -127 až +128 a zabírá jeden byte. Další datové typy mají vždy dvojnásobný rozsah než předchozí.

MySQL umožňuje sloupci, který je číselného typu, nastavit vlastnost AUTO_INCREMENT. Nastavení této vlastnosti je vhodné zejména u sloupců, které tvoří primární klíč (často se takový sloupec nazývá ID). Nastavením vlastnosti docílíme toho, že hodnoty ve sloupci se budou u každého nového záznamu zvyšovat o jedničku – tím pádem je každá hodnota jedinečná.

12.1.2 Desetinná čísla



Pro uložení desetinných čísel je možné použít čísla s pevnou nebo s plovoucí desetinnou čárkou. Rozdíl mezi nimi spočívá v jejich vnitřní interpretaci. Čísla s plovoucí desetinnou čárkou se v MySQL ukládají v binárním formátu, čísla s pevnou desetinnou čárkou se ukládají jako řetězce. U čísel s pevnou desetinnou čárkou jsou vyloučeny chyby způsobené zaokrouhlováním. Nevýhodou ale je, že jsou vyšší nároky na paměť.

MySQL obsahuje 3 datové typy pro desetinná čísla. Datové typy FLOAT a DOUBLE slouží pro uložení čísel s plovoucí desetinnou čárkou. Datový typ DECIMAL pak slouží k uložení čísel s pevnou desetinnou čárkou. Kromě toho existují typy REAL (synonymum pro DOUBLE), NUMERIC a DEC (oba představují synonymum pro DECIMAL). Pomocí parametrů můžeme u všech těchto typů nastavit počet cifer a počet desetinným míst.

12.1.3 Datum a čas



Pro reprezentaci kalendářního data a času se nejčastěji používá typ DATETIME. Pomocí tohoto typu můžeme ukládat libovolné časové údaje. Kromě toho existují typy DATE pro reprezentaci data (bez času) a TIME pro reprezentaci času (bez data). Dalšími typy jsou pak YEAR (letopočet) a TIMESTAMP. Obsah sloupce s typem TIMESTAMP se při každé změně v MySQL automaticky aktualizuje. Jeho použití je vhodné v případě, že chceme sledovat datum a čas poslední změny.

Datum v MySQL je uloženo ve formátu YYYY-mm-dd, tj. jako první je uložen rok, dále měsíc (vždy na dvě místa) a nakonec den (také na dvě místa). Oddělovačem je znak – (pomlčka). Datum 17.6.2010 bude tedy uloženo jako 2010-06-17, datum 6.11.2012 jako 2012-11-06.

12.1.4 Řetězce znaků



Pro ukládání řetězců znaků jsou v MySQL k dispozici datové typy CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT a LONGTEXT. U typů CHAR a VARCHAR můžeme nastavit maximální délku řetězce. U ostatních datových typů je maximální délka určena přímo datovým typem.

12.1.5 Binární data

Pro uložení binárních dat, např. obrázků, můžeme v MySQL použít 4 datové typy. Rozdíl mezi nimi je pouze ve velikosti. Pro binární data jsou určeny datové typy TINYBLOB, BLOB, MEDIUMBLOB a LONGBLOB.

12.2 Přístup k databázi MySQL prostřednictvím PHP

Pro přístup k databázi MySQL jsou v PHP k dispozici dvě různá rozhraní. Prvním způsobem je funkce `mysql`, která byla k dispozici už v dřívějších verzích PHP. Od PHP verze 5 je k dispozici také objektově orientované rozhraní `mysqli`. Rozhraní `mysqli` můžeme použít ale i procedurálním stylem. V této studijní opoře se zaměříme na rozhraní `mysqli`, ale zmíníme i funkce `mysql`. Rozhraní `mysqli` je novější a funkce `mysql` už začínají být zastaralé.

Obě rozhraní nejsou standardní součástí PHP. K jejich fungování je potřeba nainstalovat potřebné knihovny. Pokud potřebujeme zjistit, zda je v PHP nainstalována podpora MySQL, můžeme tak učinit pomocí funkce `phpinfo()`, kterou zavoláme v libovolném souboru.

12.2.1 Připojení k databázi a výběr databáze



Prvním krokem při práci s databází je připojení k databázovému serveru MySQL. Připojení pomocí `mysqli` je možné dvěma způsoby. Prvním způsobem je vytvoření instance objektu `mysqli`. Konstruktor může obsahovat až 6 volitelných parametrů. Jsou to adresa databázového serveru (pokud máme PHP i MySQL nainstalovány lokálně, je adresa `localhost`), jméno uživatele, heslo uživatele, název databáze, port a socket. Pokud je pokus o připojení k databázovému serveru úspěšný, je vytvořen objekt, která představuje připojení k MySQL serveru. Druhou možností připojení je pomocí funkce `mysqli_real_connect()`.

Ve starším rozhraní `mysql` jsou k dispozici funkce `mysql_connect()` a `mysql_pconnect()`. Všechny parametry těchto funkcí jsou volitelné. Pro obě jsou důležité první tři parametry. Prvním parametrem je adresa databázového serveru, druhým uživatelské jméno a třetím heslo. Obě funkce vracejí odkaz na připojení k MySQL, v případě neúspěchu pak vracejí `false`.



Po připojení k databázovému serveru je ještě potřeba vybrat databázi, se kterou budeme pracovat. Pokud databáze již existuje, pro její výběr použijeme funkci `mysql_select_db()`, jejímž parametrem je název databáze. Pokud databáze doposud neexistuje, můžeme ji vytvořit pomocí funkce `mysql_create_db()`. Parametrem je opět název databáze. Obě funkce mají druhý volitelný parametr a tím je odkaz na aktivní připojení k databázi. Návratovou hodnotou obou funkcí je logická hodnota, pokud je operace

úspěšná, funkce vracejí `true`, v případě neúspěchu vracejí `false`. Při použití rozhraní `mysqli` je název funkce pro výběr databáze `mysqli_select_db()`. Návratová hodnota je stejná jako u funkcí `mysql_select_db()` a `mysql_create_db()`. Parametry se liší, funkce má dva povinné parametry. První parametrem je odkaz na aktuální připojení k databázovému serveru, druhým parametrem je pak název databáze.

Následující výpis ukazuje připojení k databázovému serveru a výběr databáze pomocí obou rozhraní. Pro připojení je použit účet `root` (výchozí účet po nainstalování MySQL serveru), který je v aktuální konfiguraci bez hesla. Po připojení je jako aktuální zvolena databáze `moje_db`.

Výpis 12.1 Připojení k databázovému serveru.



```
<?php
  $pripojeni1 = mysql_pconnect("localhost", "root")
  or exit ("Není možné připojit databázový server");
  $db = mysql_select_db("moje_db")
  or exit ("Není možné vybrat databázi");

  $pripojeni2 = mysqli_init();
  if (!$pripojeni2) {
    exit("mysqli_init selhalo");
  } else {
    echo "mysql_init proběhlo v pořádku";
  }
  if (!mysqli_real_connect($pripojeni2, "localhost",
   "root")) {
    exit ("Není možné připojit databázový server");
  } else {
    echo "Připojení k databázovému serveru proběhlo
     úspěšně";
  }
  mysqli_select_db($pripojeni2, "moje_db")
  or exit ("Není možné vybrat databázi");
?>
```

12.2.2 Spouštění příkazů SQL



Po připojení k databázovému serveru a zvolení aktuální databáze již můžeme spouštět SQL příkazy. Může se jednat o jakékoliv příkazy z jazyka SQL. Můžeme použít příkazy z částí DDL (Data Definition Language), DML (Data Manipulation Language) i DQL (Data Query Language). Do části DDL patří příkazy pro vytvoření, úpravu nebo smazání databázových objektů, např.

tabulky, indexu nebo pohledu. V části DML jsou k dispozici příkazy pro manipulaci se záznamy tabulek. Můžeme vložit nový záznam (řádek) do tabulky, upravit jej nebo jej smazat. Pro část DQL je pak k dispozici pouze jeden příkaz a tím je příkaz SELECT, který slouží k vybrání záznamů z databáze, které splňují zadaná kritéria. Z hlediska spouštění dotazů pomocí PHP pak mohou nastat dvě situace. První situací je použití příkazu z části DDL nebo DML. Spuštění takového dotazu má jako efekt změnu databázového objektu nebo úpravu sady záznamů v tabulkách. Taková změna tedy nevrací žádný výsledek (i když z hlediska použití v PHP to není úplně přesně řečeno, viz. dále). Použití příkazu SELECT má naopak jako efekt načtení sady záznamů z tabulky nebo tabulek, které odpovídají zadaným kritériím. Tato data jsou pak vrácena ve formě jakési virtuální tabulky, která obsahuje sloupce definované v příkazu SELECT. Vrácená tabulka může být samozřejmě prázdná, pokud neexistuje záznam, který by odpovídal zadaným kritériím.



Pro spuštění libovolného SQL příkazu pomocí rozhraní mysqli slouží funkce `mysqli_query()`, která má dva parametry. Prvním parametrem je odkaz na připojení k databázovému serveru, druhým parametrem je dotaz, který chceme spustit. Dotaz se zadává ve formě textového řetězce. Návratová hodnota se liší podle toho, jaký SQL příkaz byl spuštěn. Jestliže byl spuštěn příkaz SELECT, funkce vrací objekt `mysqli_result`, který představuje již zmíněnou virtuální tabulku. Pro ostatní příkazy SQL funkce vrací `true` v případě úspěchu. V případě neúspěchu vrací funkce `false` a je jedno, jaký SQL příkaz byl spuštěn. Rozhraní `mysql` obsahuje funkci `mysql_query()`, která funguje velmi podobně.

Pokud v aktuální databázi spustíme některý z příkazů DDL nebo DML, stačí otestovat, zda funkce `mysqli_query()` vrátila hodnotu `true`. Následující výpis zobrazuje ukázku použití funkce `mysqli_query()`, v proměnné `$pripojeni` je uložen odkaz na aktuální připojení k databázovému serveru.

Výpis 12.2 Vytvoření tabulky a vložení záznamu pomocí funkce mysqli_query().



```
<?php
  $dotaz = "CREATE TABLE IF NOT EXISTS osoba (
   id_osoba int(11) NOT NULL AUTO_INCREMENT,
   jmeno varchar(50) NOT NULL,
   prijmeni varchar(50) NOT NULL,
   PRIMARY KEY (id_osoba)
   ) ENGINE=InnoDB DEFAULT CHARSET=utf8
   COLLATE=utf8_czech_ci;";
```

```
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Tabulka byla úspěšně vytvořena.<br>";
  } else {
    echo "Chyba při vytváření tabulky.<br>";
  }

  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('Rostislav', 'Miarka');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('Jan', 'Novák');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('Petra', 'Dvořáková');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('David', 'Král');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $posledniId = mysqli_insert_id($pripojeni);
  echo "Poslední ID: " . $posledniId;
?>
```

V uvedeném výpisu je použit postup, kdy je prováděný SQL příkaz uložen do samostatné proměnné a při volání funkce `mysqli_query` je tato proměnná předána jako parametr. První příkaz vytvoří tabulku osoba, která má 3 sloupce.

Sloupec `id_osoba` tvoří primární klíč a má nastavenu vlastnost AUTO_INCREMENT (při každém vložení záznamu je vygenerována nová hodnota v tomto sloupci o jedničku vyšší než předchozí hodnota, první záznam má ve sloupci hodnotu 1). Všimněte si, že název tabulky nebo názvy sloupců nemusí být uvedeny v uvozovkách ani v apostrofech. Pokud bychom chtěli použít apostrof, je potřeba použít znak ` (obrácený apostrof), jehož kód v ASCII tabulce je 96. S obyčejným apostrofem nebo s klasickými uvozovkami nebude příkaz fungovat. Při vkládání záznamů do tabulky stačí vložit hodnoty do sloupců `jmeno` a `prijmeni`, do sloupce `id_osoba` se hodnota vygeneruje automaticky.

Pokud používáme u primárního klíče vlastnost AUTO_INCREMENT a potřebujeme zjistit hodnotu ID u posledního vloženého záznamu, můžeme použít funkci `mysqli_insert_id()`, jejímž parametrem je aktuální připojení k databázovému serveru. Ukázka použití této funkce je na posledních dvou řádcích výpisu.

12.2.3 Zpracování výsledků příkazu SELECT



Základem práce s každou databází je dotazování. Těžko bychom používali databázi pouze k tomu, abychom do ní uložili data a nijak s nimi nepracovali. Potřebujeme načítat data z tabulek a dále s nimi pracovat. K dotazování slouží příkaz SELECT. Jeho základní syntaxe je následující:

SELECT seznam_sloupců

FROM seznam_tabulek

WHERE podmínka

ORDER BY sloupce

Za slovem SELECT následuje seznam sloupců databázových tabulek, které chceme v dotazu zobrazit. Dále následuje slovo FROM a za ním seznam tabulek, ze kterých načítáme data. Další částí je stanovení podmínek, tato část následuje za slovem WHERE. Podmínek může být více, může se jednat o podmínky pro spojení tabulek nebo pro omezení výsledků. Poslední částí je část následující za slovy ORDER BY, v této části můžeme definovat řazení výsledků.

Pokud jako parametr funkce `mysqli_query()` použijeme dotaz definovaný pomocí příkazu SELECT, vrací tato funkce objekt `mysqli_result`. Tento

objekt představuje virtuální tabulku, kterou tvoří sloupce definované v dotazu a která obsahuje záznamy splňující zadaná kritéria. S výsledkem dotazu můžeme dále pracovat. Jsou v zásadě dvě možnosti. Buď můžeme pracovat přímo s objektem `mysqli_result` a nebo s jeho částmi (řádky nebo sloupce). V případě zpracování po částech jen nejtypičtější zpracování po řádcích. Pro zpracování po řádcích je možné použít čtyři různé funkce. Všechny funkce mají jako parametr objekt, který reprezentuje výsledek dotazu. Funkce vracejí jeden řádek výsledku ve formě pole nebo objektu. Kromě toho se provede posun ukazatele v rámci výsledku na další záznam. Pokud už bylo dosaženo posledního záznamu, funkce vrací hodnotu `false`. K procházení výsledků můžeme použít následující funkce:



- `mysqli_fetch_row()` – vrací záznam (řádek) výsledku ve formě jednoduchého pole. Jednotlivé sloupce výsledku jsou indexovány číslem, první sloupec má index `0`.

- `mysqli_fetch_assoc()` – vrací záznam (řádek) výsledku ve formě asociativního polo. K jednotlivým sloupcům výsledku můžeme přistupovat pomocí indexu tvořeného řetězcem – indexem je název sloupce tabulky.

- `mysqli_fetch_array()` – vrací záznam (řádek) výsledku ve formě jednoduchého a asociativního polo. K jednotlivým sloupcům výsledku můžeme přistupovat buď pomocí číselného indexu (první sloupec má opět index `0`) nebo pomocí indexu tvořeného řetězcem – indexem je název sloupce tabulky.

- `mysqli_fetch_object()` – vrací záznam (řádek) výsledku ve formě objektu. Jednotlivé sloupce výsledku pak tvoří atributy (členské proměnné) objektu. K hodnotám ve sloupcích pak můžeme přistupovat pomocí názvu sloupců. Pokud je již dosaženo konce výsledku, vrací funkce hodnotu `null` (na rozdíl od předchozích třech funkcí).

Při zpracování výsledků dotazu je někdy užitečné znát počet záznamů ve výsledku. Ke zjištění počtu řádku v objektu `mysqli_result` slouží funkce `mysqli_num_rows()`, jejímž parametrem je instance objektu `mysqli_result`. Funkce vrací číslo – počet řádků ve výsledku.

Kromě uvedených funkcí můžeme u instance objektu `mysqli_result` zjišťovat počet sloupců, zjišťovat metadata o sloupcích, testovat, zda sloupec

existuje apod. Nejčastěji ale použijeme výše zmíněné funkce, protože při dotazování bychom měli znát strukturu databáze, takže víme, jaké sloupce jsou v tabulkách. Bez této informace bychom těžko vytvářeli složitější příkazy SELECT.

Následující výpis ukazuje načtení obsahu tabulky osoba. Vytvoření této tabulky a vložení 4 záznamů je uvedeno v předchozím výpisu. Ve výpisu je opět použita proměnná `$pripojeni`, která obsahuje odkaz na aktuální připojení k databázovému serveru. Obrázek pod výpisem pak zobrazuje výsledek v prohlížeči.

Výpis 12.3 Zpracování výsledku dotazu SELECT.

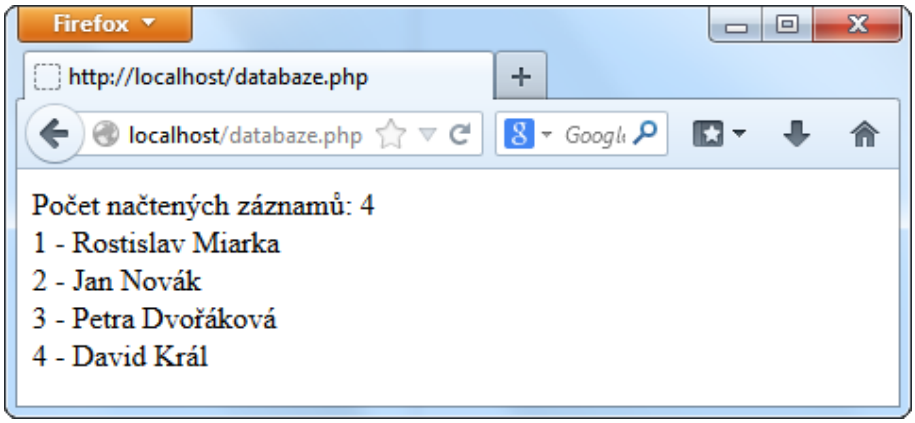
```
<?php
  $dotaz = "SELECT * FROM osoba";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  $pocet = mysqli_num_rows($vysledek);
  echo "Počet načtených záznamů: " . $pocet . "<br>";
  // klasicke pole
  $radek = mysqli_fetch_row($vysledek);
  echo $radek[0] . " - " . $radek[1] . " " .
   $radek[2] . "<br>";

  // asociativni pole
  $radek = mysqli_fetch_assoc($vysledek);
  echo $radek["id_osoba"] . " - " . $radek["jmeno"] .
   " " . $radek["prijmeni"] . "<br>";

  // klasicke a asociativni pole
  $radek = mysqli_fetch_array($vysledek);
  echo $radek["id_osoba"] . " - " . $radek[1] . " " .
   $radek["prijmeni"] . "<br>";

  // objekt
  $radek = mysqli_fetch_object($vysledek);
  echo $radek->id_osoba . " - " . $radek->jmeno . " "
   . $radek->prijmeni . "<br>";
?>
```





Obrázek 12.1 Zpracování výsledku dotazu SELECT.

12.2.4 Uvolnění výsledku a uzavření připojení k databázi.



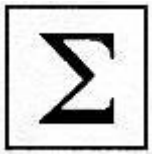
Po ukončení práce s jakýmkoliv zdrojem je potřeba uzavřít veškerá existující připojení. Pokud pracujeme se souborem, po skončení práce musíme soubor uzavřít. Totéž platí pro práci s databází. PHP umožňuje uvolnit (zrušit) výsledek, který byl vrácen po provedení příkazu SELECT. K tomuto účelu je určena funkce `mysqli_free_result()`, která má jako parametr instanci objektu `mysqli_result`. Funkce nic nevrací, tj. vrací datový typ `void`.

Po uvolnění výsledku je připojení k databázovému serveru stále aktivní. Ke korektnímu ukončení práce je potřeba připojení uzavřít. K uzavření připojení slouží funkce `mysqli_close()`, jejímž parametrem je odkaz na aktivní připojení k databázi. Funkce vrací logickou hodnotu. Pokud vše proběhne bez problémů, vrátí hodnotu `true`, v případě neúspěchu pak hodnotu `false`.



Kontrolní otázky:

1. Pomocí jakých rozhraní je možné pracovat v PHP s databází MySQL?
2. Jak provedete připojení k databázi a výběr databáze?
3. Jaké funkce je možné použít pro zpracování výsledků dotazu SELECT?



Shrnutí obsahu kapitoly

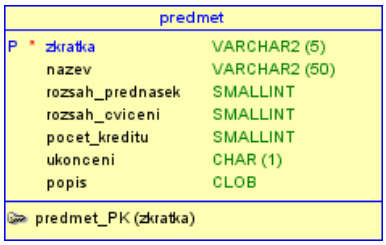
V této kapitole jste se seznámili se základními principy práce s databází MySQL pomocí jazyka PHP. K práci s databází je možné použít dvě rozhraní `mysql` a `mysqli`. Rozhraní `mysqli` je novější, proto je vhodnější. Základní funkcí pro spouštění příkazů SQL je `mysqli_query()`. Pomocí této funkce můžeme spouštět libovolné SQL příkazy. Ke zpracování výsledků vrácených

po spuštění příkazu SELECT slouží objekt `mysqli_result`. Procházení záznamů ve výsledku je možné pomocí čtyř různých funkcí. Po ukončení práce s databází bychom měli uvolnit načtené výsledky a uzavřít připojení k databázi.

Korespondenční úkol č. 2



Na databázovém serveru si vytvořte tabulku předmětů, která bude mít strukturu jako na následujícím obrázku (pro jednotlivé sloupce použijte vhodné datové typy MySQL):

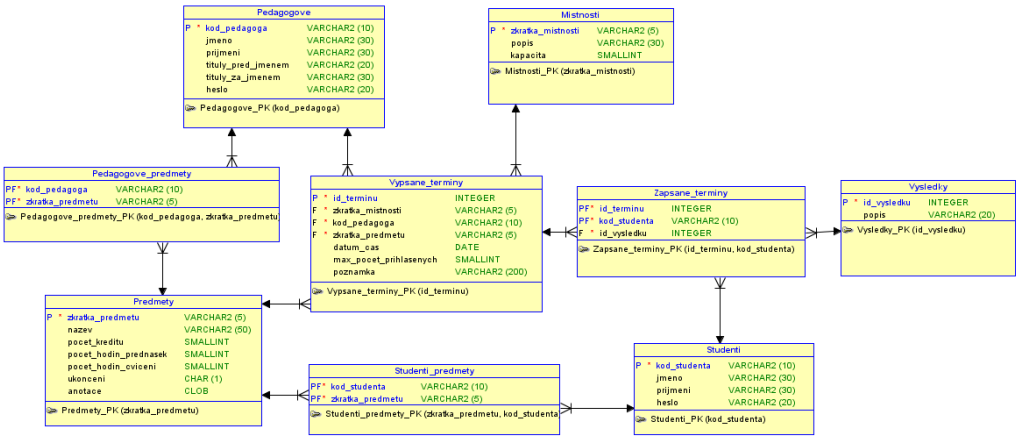


Dále vytvořte PHP skript, který bude obsahovat formulář pro vložení záznamu do tabulky `predmet`. Ve formuláři použijte vhodné formulářové prvky. Pod formulářem bude ve formě tabulky vypsán obsah tabulky `predmet`. Pokud tabulka neobsahuje žádný záznam, bude o tom zobrazena zpráva.

Formulář se bude odesílat na stejný PHP skript. Po odeslání formuláře se provede kontrola zadaných dat (pokud bude nutná) a uloží se nový záznam do tabulky.

13 Semestrální projekt

Vytvořte webovou aplikaci, která bude sloužit k práci se zkouškovými termíny na vysoké škole. V aplikaci budou dvě uživatelské role – pedagog a student. Pedagog bude moci vytvářet termíny zápočtů nebo zkoušek pro předmět, které vyučuje. Vytvořené termíny bude moci upravovat nebo mazat. Dále bude moci u termínů zadávat hodnocení. U zápočtu, zda student vyhověl nebo nevyhověl. U zkoušky bude moci zadat známku. Student se bude moci zapsat na termín a nebo se odepsat z termínu. Struktura databáze je uvedena na následujícím obrázku:



Do aplikace bude fungovat přihlašování, nepřihlášený uživatel nemůže v systému dělat žádné operace. V rámci aplikace se budou měnit pouze záznamy v tabulkách vypsane_terminy a zapsane_terminy. Ostatní tabulky si naplňte zkušebními daty a v aplikaci z nich budete pouze načítat záznamy.

Literatura



Castagnetto, J., Rawat, H., Schumann, S., Scollo, C., & Veliath, D. (2002). *Programujeme PHP profesionálne.* Computer Press.

IANA | MIME media types. (nedatováno). Načteno z http://www.iana.org/assignments/media-types

Kofler, M., & Öggl, B. (2011). *PHP 5 a MySQL 5 Průvodce webového programátora.* Computer Press.

Leiss, O., & Shmidt, J. (2010). *PHP v praxi.* Grada Publishing, a.s.

Uniform Resource Identifier (URI). (nedatováno). Načteno z http://www.ietf.org/rfc/rfc3986.txt

W3C. (nedatováno). *HTML 4.01 specification*. Načteno z http://www.w3.org/TR/html4/

W3C. (nedatováno). *RFC2616 - HyperText Transfer Protocol -- HTTP/1.1*. Načteno z http://tools.ietf.org/html/rfc2616

Vysvětlivky k používaným symbolům

	**Průvodce studiem** – vstup autora do textu, specifický způsob, kterým se studentem komunikuje, povzbuzuje jej, doplňuje text o další informace
	**Příklad** – objasnění nebo konkretizování problematiky na příkladu ze života, z praxe, ze společenské reality, apod.
	**Pojmy k zapamatování.**
	**Shrnutí** – shrnutí předcházející látky, shrnutí kapitoly.
	**Literatura** – použitá ve studijním materiálu, pro doplnění a rozšíření poznatků.
	**Kontrolní otázky a úkoly** – prověřují, do jaké míry studující text a problematiku pochopil, zapamatoval si podstatné a důležité informace a zda je dokáže aplikovat při řešení problémů.
	**Úkoly k textu** – je potřeba je splnit neprodleně, neboť pomáhají dobrému zvládnutí následující látky.
	**Korespondenční úkoly** – při jejich plnění postupuje studující podle pokynů s notnou dávkou vlastní iniciativy. Úkoly se průběžně evidují a hodnotí v průběhu celého kurzu.
	**Úkoly k zamyšlení.**
	**Část pro zájemce** – přináší látku a úkoly rozšiřující úroveň základního kurzu. Pasáže a úkoly jsou dobrovolné.
	**Testy a otázky** – ke kterým řešení, odpovědi a výsledky studující najdou v rámci studijní opory.
	**Řešení a odpovědi** – vážou se na konkrétní úkoly, zadání a testy.